



**Politécnico
Castelo Branco**

Escola Superior
de Tecnologia

Inteligência Artificial aplicada à Cibersegurança

Deteção de URL Maliciosos

Kyrylo Sakhnenko Nº 20221710

Rodrigo Coimbra Figueiredo Nº 20221713

Orientador

Professor Doutor Alexandre José Pereira Duro da Fonte

Trabalho de projeto apresentado à Escola Superior de Tecnologia do Instituto Politécnico de Castelo Branco para cumprimento dos requisitos necessários à obtenção do grau de licenciados em Engenharia Informática, realizado sob a orientação científica do Professor Adjunto Doutor Alexandre José Pereira Duro da Fonte, do Instituto Politécnico de Castelo Branco.

Junho 2026

Composição do júri

Presidente

Doutor, Alexandre José Pereira Duro da Fonte

Professor Adjunto da Escola Superior de Tecnologia de Castelo Branco

Arguente 1

Doutora, Ana Paula Neves Ferreira da Silva

Professora Adjunta da Escola Superior de Tecnologia de Castelo Branco

Arguente 2

Doutor, Eduardo Sabina dos Santos Valente

Professor Adjunto da Escola Superior de Tecnologia de Castelo Branco

Agradecimentos

Em primeiro lugar, expressamos a nossa sincera gratidão ao nosso orientador, Professor Alexandre Fonte pela orientação e pelo apoio ao longo de todo o desenvolvimento deste projeto. A sua disponibilidade foi essencial para o sucesso deste trabalho.

Agradecemos também à Escola Superior de Tecnologia de Castelo Branco que nos acolheu e forneceu os recursos necessários para a realização deste projeto, bem como a todos os colegas e professores que contribuíram direta ou indiretamente para o nosso crescimento académico e profissional.

Não poderíamos deixar de manifestar o nosso profundo reconhecimento às nossas famílias e amigos, cujo encorajamento, compreensão e paciência nos acompanharam em cada etapa deste percurso.

Por fim, agradecemos a todos aqueles que, de forma direta ou indireta, contribuíram para a concretização deste projeto.

Resumo

A cibersegurança mantém-se como uma das áreas de maior crescimento e urgência no panorama tecnológico atual, com ataques de *phishing* e distribuição de URL maliciosos a tornarem-se progressivamente mais sofisticados e difíceis de detetar pelas abordagens tradicionais. Este relatório descreve o desenvolvimento completo de um sistema de deteção de URL maliciosos baseado em Machine Learning, materializando os objetivos definidos no Projeto I e concretizando a transição da componente teórica para uma solução funcional e testada.

O pipeline de Machine Learning implementado percorreu todas as etapas do processo: pré-processamento e limpeza do *dataset*, normalização das variáveis, extração de 27 características lexicais, treino e comparação de sete algoritmos de classificação e otimização de hiperparâmetros. O algoritmo Random Forest confirmou empiricamente a sua superioridade, alcançando 94,65% de Accuracy e 98,40% de AUC-ROC no modelo final, valores consistentes com os reportados na literatura científica analisada no Projeto I.

O sistema desenvolvido integra quatro modalidades de análise complementares: uma aplicação web Flask para classificação manual de URL, uma extensão de browser que analisa automaticamente todos os links presentes em qualquer página web, com modo especializado para o Gmail, um monitor de rede em tempo real que captura e classifica tráfego HTTP em tempo real via Server-Sent Events, e um dashboard web para análise de capturas de tráfego em formato .pcap. Em conjunto, estes quatro componentes implementam uma arquitetura de defesa em profundidade, cobrindo desde a interação do utilizador com links individuais até à auditoria do tráfego gerado por qualquer aplicação no sistema. O modelo beneficia ainda de um sistema de aprendizagem contínua, que permite o seu re-treino automático semanal com base no feedback dos utilizadores, através do GitHub Actions.

Palavras chave

Machine Learning(ML); Cibersegurança; URL Maliciosos; *Phishing*; Monitor de Rede

Abstract

Cybersecurity remains one of the fastest-growing and most urgent areas in today's technological landscape, with *phishing* attacks and malicious URL distribution becoming increasingly sophisticated and difficult to detect using traditional approaches. This report describes the complete development of a machine learning-based malicious URL detection system, materialising the objectives defined in Project I and completing the transition from theoretical research to a functional and tested solution.

The implemented machine learning pipeline covered all stages of the process: *dataset* preprocessing and cleaning, variable normalisation, extraction of 27 lexical *features*, training and comparison of seven classification algorithms, and hyperparameter optimisation. The Random Forest algorithm empirically confirmed its superiority, achieving 94.65% Accuracy and 98.40% AUC-ROC in the final model, results consistent with those reported in the scientific literature analysed in Project I.

The developed system integrates four complementary analysis modalities: a Flask web application for manual URL classification, a browser extension that automatically analyses all links present on any web page with a specialised mode for Gmail, a real-time network monitor that captures and classifies live HTTP traffic via Server-Sent Events, and a web dashboard for analysing network traffic captures in .pcap format. Together, these four components implement a defence-in-depth architecture, covering everything from the user's interaction with individual links to the auditing of traffic generated by any application on the system. The model also benefits from a continuous learning system that enables automatic weekly retraining based on user feedback, through GitHub Actions.

Keywords

Machine Learning (ML); Cybersecurity; Malicious URL; *Phishing*; Network Monitor

Índice geral

Composição do júri.....	III
Agradecimentos	V
Resumo	VII
Palavras chave.....	VII
Abstract	IX
Keywords	IX
Índice geral.....	XI
Índice de Figuras.....	XVII
Lista de tabelas	XIX
Lista de abreviaturas, siglas e acrónimos	XXIII
Introdução	1
1.Revisitando conceitos fundamentais	3
1.1. Estrutura e Anatomia de um URL	3
1.1.1. O que é um URL	3
1.1.2. Componentes de um URL.....	3
1.1.3. Características dos URL que indicam malícia.....	4
1.2. Algoritmos de Machine Learning	4
1.2.1. Árvores de Decisão (Decision Trees).....	5
1.2.2. Random Forest (RF)	5
1.2.3. Support Vector Machine (SVM).....	6
1.2.4. Regressão Logística (Logistic Regression – LR)	6
1.2.5. Naive Bayes (NB).....	7
1.2.6. K-Nearest Neighbors (KNN).....	7
1.2.7. Gradient Boosting	7
1.3. <i>Feature</i> Engineering para URL	7
1.3.1. Características Lexicais	7
1.3.2. Características Baseadas no Domínio (WHOIS / DNS).....	8
1.4. Métricas de Avaliação de Modelos de ML	9
1.4.1. Matriz de Confusão	9
1.4.2. Definições das Métricas Principais.....	9
1.5. Pipeline do Projeto I.....	10
1.6. Scikit-learn	11

1.7. Pandas, NumPy e Matplotlib– Bibliotecas Ausentes.....	11
1.8. Conceitos de Cibersegurança em Falta	12
1.8.1. Spoofing	12
1.8.2. DNS (Domain Name System).....	12
1.8.3. <i>Malware</i> e Tipos de Ameaças Web	12
1.8.4. Blacklists e Whitelists	13
1.9. Técnicas Avançadas Mencionadas nos Artigos.....	13
1.9.1. BERT e Modelos Transformer	14
1.9.2. Graph Neural Networks (GNN).....	14
1.9.3. GANs para URL Adversariais.....	14
1.9.4. XAI – Inteligência Artificial Explicável (SHAP e LIME)	14
1.10. Overfitting, Underfitting e Validação Cruzada	14
1.10.1. Overfitting	15
1.10.2. Underfitting	15
1.10.3. Como o Random Forest Mitiga o Overfitting	15
1.10.4. Hiperparâmetro e Otimização.....	15
1.11. Conclusão do Capítulo.....	16
2. Pré Processamento dos Dados	17
2.1. Carregamento e Inspeção Inicial	17
2.2. Distribuição de Classes.....	18
2.3. Identificação de Problemas de Qualidade	18
2.3.1. Valores Nulos	18
2.3.2. Registos Duplicados.....	19
2.3.3. Registo Corrompido.....	19
2.3.4. Valores Atípicos (Outliers).....	19
2.4. Aplicação da Limpeza	20
2.5. Verificação Pós-Limpeza	21
2.6. Exportação do <i>Dataset</i> Limpo.....	22
2.7. Normalização das Variáveis.....	22
2.7.1. Análise das Escalas	22
2.7.2. Aplicação do Min-Max Scaler	23
2.8. Engenharia de Características	24
2.8.1. Características Extraídas.....	24

2.8.2. Estatísticas das Novas Características	25
2.8.3. Análise por Classe	26
2.8.4. <i>Dataset</i> Final	27
2.9. Conclusão do Capítulo	28
3. Treino e Comparação de Modelos	29
3.1. ConFiguração Experimental	29
3.2. Algoritmos treinados	29
3.3. Resultados Comparativos.....	30
3.4. Análise dos Resultados	31
3.5. Análise Detalhada – Random Forest	31
3.5.1. Matriz de Confusão	32
3.5.2. <i>Feature</i> Importance	32
3.5.3. Curva ROC.....	34
3.6. Justificação da Escolha do Random Forest.....	35
3.7. Resumo do Capítulo	35
4. Otimização de Hiperparâmetros.....	37
4.1 Metodologia – RandomizedSearchCV	37
4.2. Espaço de Hiperparâmetros	38
4.3 Melhores Hiperparâmetros Encontrados	38
4.4 Resultados do Modelo Otimizado	39
4.5 Impacto na Matriz de Confusão	40
4.6 Validação Cruzada do Modelo Otimizado.....	41
4.7 Sumário da Otimização	41
4.8. Resumo do Capítulo	42
5. Desenvolvimento da Aplicação e Iteração do Modelo	43
5.1. Primeira Versão da Aplicação – Problema Identificado	43
5.2. Diagnóstico do Problema.....	43
5.3. Engenharia de Características Expandida – 27 <i>Features</i> Lexicais	44
5.4 Modelo Lexical – Primeira Avaliação	45
5.5 Diagnóstico do Viés do <i>Dataset</i> Original	46
5.6 Solução Final – Enriquecimento do <i>Dataset</i>	47
5.7 Modelo Final – Resultados	47
5.7.1. Métricas de Desempenho	48

5.7.2. Matriz de Confusão	48
5.7.3. <i>Feature</i> Importance	49
5.7.4. Validação com URL Reais.....	50
5.8. Correções Aplicadas ao Modelo Final	51
5.8.1. Problema 1 – Inconsistência com o prefixo <i>www</i>	51
5.8.2. Problema 2 – Bloco <i>urlparse</i> Ausente	51
5.8.3. Métricas do Modelo Final (v3)	51
5.9. Resumo do Capítulo	52
6. Aplicação Web – Detetor de URL Maliciosos	53
6.1. Arquitetura da Aplicação	53
6.2. Interface de Utilizador	54
6.2.1. Ecrã Inicial.....	54
6.2.2. Resultado – URL Benigno	54
6.2.3. Resultado – URL Malicioso	55
6.2.4. Resultado – URL Incerto	55
6.3 Componentes da Interface.....	56
6.4. Tecnologias Utilizadas	57
6.5. Testes da Aplicação.....	57
6.6. Limitações Identificadas.....	58
6.7. Instruções de Execução.....	59
6.8. Resumo do capítulo	59
7. Extensão de Browser – URL Shield	61
7.1. Motivação e Objetivos.....	61
7.2. Arquitetura da Extensão.....	61
7.3. Estrutura de Ficheiros	62
7.4. Interface de Utilizador	63
7.4.1. Destaque Visual na Página	63
7.4.2. <i>Popup</i> de Resumo.....	63
7.4.3. Modo de Análise de Email – Gmail	65
7.5. Integração com a Aplicação Flask	66
7.5.1. Suporte a CORS e Private Network Access	66
7.5.2. Novo Endpoint de Feedback	66
7.6. Sistema de Aprendizagem Continua.....	67

7.6.1. Motivação.....	67
7.6.2. Arquitetura do Sistema.....	67
7.6.3. Dados Utilizados para Re-treino	68
7.6.4. Funcionamento do Re-treino Automático	68
7.7. Testes e Validação	69
7.8. Limitações Identificadas	71
7.9. Instruções de Instalação	71
7.10. Resumo do Capítulo	72
8. Integração Wireshark – Análise de Tráfego de Rede.....	73
8.1. Motivação e Objetivos	73
8.2. Arquitetura de Defesa em Camadas.....	73
8.3. Limitação: HTTP vs HTTPS.....	74
8.4. Ferramentas e Dependências.....	75
8.5. Captura do Tráfego de Rede	75
8.6. Pipeline de Análise	76
8.7. Resultados Obtidos	76
8.8. Limitações Identificadas	77
8.9. Resumo do Capítulo	78
9. Monitor de Rede em Tempo Real	79
9.1. Motivação e Objetivos	79
9.2. Arquitetura do Sistema	80
9.3. Implementação técnica.....	81
9.3.1. Integração no app.py – Arranque Automático	81
9.3.2. Server-Sent Events (SSE).....	81
9.3.3. Integração na Extensão de Browser	82
9.4. Saída no Terminal	83
9.5. Interface de Utilizador	83
9.5.1. Painel na Aplicação Flask	83
9.5.2. Painel na Extensão de Browser	84
9.6. Pré-requisitos e Instalação	85
9.7. Testes e Validação	86
9.8. Limitações Identificadas	87
9.9. Conclusão de Capítulo	88

10. Dashboard de Tráfego – Aplicação Web	89
10.1. Motivação e Objetivos	89
10.2. Reestruturação da Aplicação – Três Páginas Independentes	90
10.3. Dashboard de Análise Wireshark.....	90
10.3.1. Upload de Ficheiro .pcap.....	90
10.3.2. Processamento no Backend - /analisar_pcap	91
10.3.3. Sumário e Visualização Gráfica	92
10.3.4. Tabela Interativa de Resultados	92
10.3.5. Exportação para CSV	93
10.3.6. Histórico de Análises	93
10.4. Persistência do Monitor entre Páginas	93
10.5. Novos Endpoints do Flask	94
10.6. Testes e Validação	94
10.7. Resumo do Capítulo	95
11. Conclusão.....	97
12. Referências Bibliográficas	99

Índice de Figuras

- Figura 1** – Distribuição de classes do *dataset* (benignos vs maliciosos)
- Figura 2** – Boxplots das variáveis contínuas antes da limpeza
- Figura 3** – Boxplots das variáveis contínuas após a limpeza (Capping IQR)
- Figura 4** – Distribuição das variáveis antes (vermelho) e depois (azul) da normalização Min-Max
- Figura 5** – Valor médio das características lexicais por classe (benigno vs malicioso)
- Figura 6** – Comparação de Accuracy, F1-Score e AUC-ROC de todos os modelos
- Figura 7** – Matriz de confusão – Random Forest (conjunto de teste, 19.183 instâncias)
- Figura 8** – *Feature* Importance – Top 15 características
- Figura 9** – Curva ROC – comparação de todos os modelos
- Figura 10** – Comparação de métricas – Baseline (Passo 3) vs Modelo Otimizado (Passo 4)
- Figura 11** – Matriz de Confusão – Antes vs Depois da Otimização
- Figura 12** – Top 12 *features* lexicais (v2) – valor médio por classe (benigno vs malicioso)
- Figura 13** – Matriz de Confusão – Modelo Final (30.548 instâncias de teste)
- Figura 14** – *Feature* Importance – Modelo Final (27 *features* lexicais)
- Figura 15** – Página inicial
- Figura 16** – Resultado benigno
- Figura 17** – Resultado malicioso
- Figura 18** – Resultado incerto
- Figura 19** – Destaque visual de link malicioso na página
- Figura 20** – *Popup* da extensão: 87 links analisados, 78 benignos, 9 maliciosos
- Figura 21** – Extensão em modo email
- Figura 22** – Repositório GitHub model-detection-url
- Figura 23** – Histórico de execução GitHub Actions
- Figura 24** – Arquitetura do monitor de rede em tempo real
- Figura 25** – Saída do terminal durante a execução do monitor
- Figura 26** – Monitor de tráfego de rede em tempo real na Aplicação Flask
- Figura 27** – Página da Aplicação Flask para análise de capturas .pcap

Lista de tabelas

Tabela 1 – Tabela sobre os componentes de um URL

Tabela 2 – Principais hiperparâmetros do Random Forest

Tabela 3 – Características Lexicais de um URL

Tabela 4 – Matriz de Confusão

Tabela 5 – Definições das Métricas

Tabela 6 – Etapas do Pipeline de Machine Learning

Tabela 7 – Bibliotecas

Tabela 8 – *Malware* e Tipos de Ameaças Web

Tabela 9 – Descrição das variáveis do *dataset*

Tabela 10 – Análise de outliers pelo método IQR

Tabela 11 – Operações de limpeza aplicadas ao *dataset*

Tabela 12 – Comparação do *dataset* antes e depois da limpeza

Tabela 13 – Análise das escalas das variáveis numéricas

Tabela 14 – Resultado da normalização Min-Max nas três colunas

Tabela 15 – Características Lexicais extraídas da coluna domain

Tabela 16 – Estatísticas descritivas das características lexicais

Tabela 17 – Valores médios por classe e correlação com o label

Tabela 18 – Composição do *dataset* final (*urldata_features.csv*)

Tabela 19 – ConFiguração experimental

Tabela 20 – Algoritmos treinados e conFiguração

Tabela 21 – Resultados comparativos dos modelos

Tabela 22 – Decomposição da matriz de confusão

Tabela 23 – Top 7 *features* mais importantes

Tabela 24 – Justificação da escolha do Random Forest

Tabela 25 – ConFiguração do RandomizedSearchCV

Tabela 26 – Espaço de hiperparâmetros definido para a otimização

Tabela 27 – Comparação dos hiperparâmetros baseline vs otimizados

Tabela 28 – Comparação de métricas baseline vs modelo otimizado

Tabela 29 – Decomposição da Matriz de Confusão antes e depois da otimização

Tabela 30 – Resultados da Validação Cruzada 5-Fold do modelo otimizado

Tabela 31 – Sumário dos resultados da otimização de hiperparâmetros

Tabela 32 – Resultados incorretos da primeira versão da aplicação

Tabela 33 – Disponibilidade das *features* em tempo real

Tabela 34 – Conjunto expandido de 27 *features* lexicais

Tabela 35 – Comparação modelo original vs modelo lexical v1

Tabela 36 – Distribuição de parâmetros de *query* por classe no *dataset* original

Tabela 37 – Tentativas de enriquecimento do *dataset* e problemas identificados

Tabela 38 – Composição do *dataset* final de enriquecimento

Tabela 39 – Comparação entre modelo lexical v1 e modelo final

Tabela 40 – Decomposição da Matriz de Confusão do modelo final

Tabela 41 – Validação com URL reais – 10/10 classificações corretas

Tabela 42 – Inconsistência de classificação causada pelo prefixo *www*.

Tabela 43 – Evolução das métricas ao longo das correções

Tabela 44 – Endpoints da aplicação Flask

Tabela 45 – Fluxo de uma classificação na aplicação

Tabela 46 – Componentes da interface de utilizador

Tabela 47 – Tecnologias utilizadas na aplicação web

Tabela 48 – Resultados dos testes da aplicação

Tabela 49 – Limitações identificadas na aplicação

Tabela 50 – Objetivos de extensão de browser

Tabela 51 – Componentes de extensão

Tabela 52 – Fluxo de análise da extensão

Tabela 53 – Componentes do *popup* da extensão

Tabela 54 – Endpoints do Flask após integração com a extensão

Tabela 55 – Componentes do sistema de aprendizagem contínua

Tabela 56 – Tipos de dados recolhidos para re-treino

Tabela 57 – Fluxo do re-treino automático semanal

Tabela 58 – Resultados dos testes funcionais da extensão e sistema de aprendizagem contínua

Tabela 59 – Limitações da extensão URL Shield e possíveis soluções

Tabela 60 – Passos de instalação da extensão URL Shield no Chrome

Tabela 61 – Objetivos da integração Wireshark

Tabela 62 – Arquitetura de defesa em camadas do sistema desenvolvido

Tabela 63 – Visibilidade do tráfego HTTP e HTTPS no Wireshark

Tabela 64 – Ferramentas e dependências da componente Wireshark

Tabela 65 – Passos para captura de tráfego HTTP com o Wireshark

Tabela 66 – Pipeline de análise do notebook Wireshark

Tabela 67 – Resultados de análise de tráfego de rede

Tabela 68 – Top 10 domínios mais frequentes na captura de tráfego

Tabela 69 – Limitações na integração Wireshark e possíveis soluções

Tabela 70 – Objetivos do monitor de rede em tempo real

Tabela 71 – Componentes do monitor de rede em tempo real

Tabela 72 – Fluxo de processamento do monitor *scapy*

Tabela 73 – Comparação entre SSE e WebSockets

Tabela 74 – Fluxo de propagação de alertas SSE na extensão de browser

Tabela 75 – Comparação entre o painel de monitor na aplicação Flask e na extensão

Tabela 76 – Pré-requisitos do monitor *scapy* em Windows

Tabela 77 – Mensagens de arranque do monitor e respetivo impacto

Tabela 78 – Resultados dos testes funcionais do monitor de rede em tempo real

Tabela 79 – Limitações do monitor de rede em tempo real e possíveis soluções

Tabela 80 – Objetivos do dashboard de tráfego

Tabela 81 – Rotas e páginas da aplicação Flask reestruturada

Tabela 82 – Fluxo de processamento do endpoint `/analisar_pcap`

Tabela 83 – Gráficos de visualização dos resultados da análise .pcap

Tabela 84 – Funcionalidades interativas da tabela de resultados

Tabela 85 – Mecanismo de persistência dos alertas do monitor de rede

Tabela 86 – Novos endpoints adicionados ao Flask

Tabela 87 – Resultados dos testes funcionais do dashboard de tráfego

Lista de abreviaturas, siglas e acrónimos

AJAX – *Asynchronous JavaScript and XML* (Tecnologia de comunicação assíncrona na web)

API – *Application Programming Interface* (Interface de Programação de Aplicações)

AUC-ROC – *Area Under the Receiver Operating Characteristic Curve* (Área Sob a Curva de Característica de Operação do Recetor)

BERT – *Bidirectional Encoder Representations from Transformers* (Modelo de Processamento de Língua Natural da Google)

CA – *Certificate Authority* (Autoridade Certificadora de Segurança SSL/TLS)

C&C – *Command and Control* (Servidor de Comando e Controlo de uma Botnet)

CNN – *Convolutional Neural Network* (Rede Neuronal Convolucional)

CSV – *Comma-Separated Values* (Formato de ficheiro de dados tabulares)

CSS – *Cascading Style Sheets* (Linguagem para estilos e design web)

DNS – *Domain Name System* (Sistema de Nomes de Domínio)

DT – *Decision Tree* (Árvore de Decisão)

ESTCB – Escola Superior de Tecnologia de Castelo Branco

FNR – *False Negative Rate* (Taxa de Falsos Negativos)

FN – Falso Negativo

FP – Falso Positivo

GAN – *Generative Adversarial Network* (Rede Generativa Adversarial)

GB – *Gradient Boosting* (Algoritmo de Aprendizagem de Máquina Sequencial)

GNN – *Graph Neural Network* (Rede Neuronal em Grafo)

HTML – *HyperText Markup Language* (Linguagem de Modelação de Páginas Web)

HTTP – *HyperText Transfer Protocol* (Protocolo de Transferência de Hipertexto)

HTTPS – *HyperText Transfer Protocol Secure* (Protocolo Seguro de Transferência de Hipertexto)

IP – *Internet Protocol* (Endereço de Protocolo de Internet)

IPCB – Instituto Politécnico de Castelo Branco

IQR – *Interquartile Range* (Intervalo Interquartil - Método de deteção de outliers)

JSON – *JavaScript Object Notation* (Formato de troca de dados leve)

KNN – *K-Nearest Neighbors* (Algoritmo dos K-Vizinhos Mais Próximos)

LIME – *Local Interpretable Model-agnostic Explanations* (Método de IA Explicável)

LR – *Logistic Regression* (Regressão Logística)

ML – *Machine Learning* (Aprendizagem de Máquina)

MLD – *Main Domain / Malicious Domain Detection* (Referência a variáveis do domínio principal)

NLP – *Natural Language Processing* (Processamento de Língua Natural)

PDF – *Portable Document Format* (Formato de Documento Portátil)

RBF – *Radial Basis Function* (Função de Base Radial - Kernel do SVM)

RF – *Random Forest* (Algoritmo de Floresta Aleatória)

ROC – *Receiver Operating Characteristic* (Curva de Característica de Operação do Recetor)

SHAP – *SHapley Additive exPlanations* (Método de IA Explicável baseado na teoria dos jogos)

SMS – *Short Message Service* (Serviço de Mensagens Curtas - usado em Spoofing)

SSL – *Secure Sockets Layer* (Protocolo de Segurança de Encriptação antigo)

SMOTE – *Synthetic Minority Over-sampling Technique* (Técnica de Sobreamostragem para *datasets* desequilibrados)

SVM – *Support Vector Machine* (Máquina de Vetores de Suporte)

TLS – *Transport Layer Security* (Protocolo de Segurança de Encriptação Moderno)

TLD – *Top-Level Domain* (Extensão de Domínio, ex: .pt, .com)

URL – *Uniform Resource Locator* (Localizador Uniforme de Recursos / Endereço Web)

UTF-8 – *8-bit Unicode Transformation Format* (Codificação de caracteres universal)

VN – Verdadeiro Negativo

VP – Verdadeiro Positivo

WHOIS – *Who Is* (Protocolo de consulta de informações de registo de domínios)

XAI – *Explainable Artificial Intelligence* (Inteligência Artificial Explicável)

Introdução

A crescente sofisticação dos ataques cibernéticos continua a representar uma ameaça séria para utilizadores e organizações em todo o mundo. Incidentes como o caso de smishing que afetou clientes dos CTT em novembro de 2024, onde os atacantes conseguiram fazer com que mensagens fraudulentas aparecessem na mesma conversa que as comunicações legítimas da empresa, ou o ataque de *phishing* que custou ao grupo Pepco cerca de 15,5 milhões de euros em fevereiro do mesmo ano, ilustram bem a dimensão e a sofisticação do problema. Estas ameaças evidenciam que as abordagens de defesa tradicionais, baseadas em blacklists e assinaturas estáticas, são cada vez mais insuficientes para proteger os utilizadores contra URL maliciosos.

Foi precisamente com o objetivo de investigar alternativas mais robustas que surgiu o Projeto I, desenvolvido no primeiro semestre do ano letivo 2025/2026. Nessa fase, foi realizada uma revisão sistemática da literatura seguindo a metodologia PRISMA, com foco em publicações indexadas no IEEE Xplore. A análise comparativa de oito artigos científicos, abrangendo o período de 2021 a 2025, permitiu concluir que o algoritmo Random Forest (RF) se destaca consistentemente como a abordagem mais equilibrada para a deteção de URL maliciosos, oferecendo precisões superiores a 97% na distinção entre endereços benignos e maliciosos, aliadas a uma elevada eficiência computacional. O Projeto I abordou ainda a definição da arquitetura tecnológica de suporte, tendo sido selecionadas a linguagem Python e as suas bibliotecas de ciência de dados como base de implementação, e o Wireshark como ferramenta de análise de tráfego de rede, bem como na seleção de um *dataset* público validado para a fase de implementação.

O presente relatório, correspondente ao Projeto II, descreve a materialização de todos os objetivos definidos nessa fase teórica, concretizando a transição para uma solução funcional, testada e integrada. Partindo das conclusões e das escolhas tecnológicas estabelecidas anteriormente, este projeto implementou o pipeline completo de Machine Learning: o pré-processamento e limpeza do *dataset* selecionado, a normalização das variáveis, a extração de 27 características lexicais, o treino e comparação de sete algoritmos de classificação, e a otimização de hiperparâmetros do modelo final. O algoritmo Random Forest confirmou a sua superioridade, alcançando 94,65% de Accuracy e 98,40% de AUC-ROC, valores que se revelaram consistentes com os reportados na literatura científica analisada no Projeto I.

Para além do modelo, o sistema desenvolvido integra quatro modalidades de análise complementares: uma aplicação web Flask para classificação manual de URL; uma extensão de browser — denominada URL Shield — que analisa automaticamente todos os links presentes em qualquer página web, com um modo especializado para o Gmail; um monitor de rede em tempo real que captura e classifica tráfego HTTP ao vivo através de Server-Sent Events; e um dashboard

web para análise de capturas de tráfego em formato .pcap. Em conjunto, estas componentes implementam uma arquitetura de defesa em profundidade, cobrindo desde a interação do utilizador com links individuais até à auditoria do tráfego gerado por qualquer aplicação no sistema. O modelo beneficia ainda de um mecanismo de aprendizagem contínua, que permite o seu retreino automático semanal com base no feedback dos utilizadores, através do GitHub Actions.

O presente documento encontra-se organizado da seguinte forma: o capítulo 1 consolida e expande os conceitos teóricos fundamentais que servem de base ao trabalho prático desenvolvido; o capítulo 2 detalha o processo de pré-processamento dos dados; o capítulo 3 apresenta o treino e a comparação dos modelos de Machine Learning; o capítulo 4 descreve a otimização de hiperparâmetros; o capítulo 5 documenta o desenvolvimento iterativo da aplicação e do modelo final; os capítulos 6 a 10 descrevem cada uma das componentes do sistema desenvolvido; e o capítulo 11 apresenta as conclusões e o trabalho futuro.

1.Revisitando conceitos fundamentais

O relatório de Projeto I estabeleceu as fundações teóricas deste trabalho, incluindo a revisão sistemática de literatura segundo a metodologia PRISMA, a comparação de algoritmos de Machine Learning aplicados à detecção de URL maliciosos e a seleção do algoritmo Random Forest como o mais promissor para a implementação. Foram ainda definidas as tecnologias de suporte (Python, Scikit-learn, Wireshark e GitHub), e selecionado o *dataset* público para treino do modelo.

Contudo, a análise dos artigos científicos do Projeto I revelou várias lacunas conceituais: termos e conceitos fundamentais, como a estrutura de um URL, as métricas de avaliação de modelos ou o funcionamento dos algoritmos comparados, foram referenciados, mas nunca explicados. O presente capítulo colmata essas lacunas, fornecendo o enquadramento conceptual necessário para a plena compreensão do trabalho desenvolvido nos capítulos seguintes.

1.1. Estrutura e Anatomia de um URL

O relatório de Projeto I refere constantemente o conceito de URL e as suas características como input para os algoritmos de ML, mas em nenhum momento explica o que é um URL nem como está estruturado. Esta é a base de toda a extração de características (*feature engineering*), pelo que a sua compreensão é indispensável.

1.1.1. O que é um URL

Um URL (Uniform Resource Locator) é o endereço padronizado que identifica um recurso na Internet. É composto por várias partes com significados distintos, cada uma das quais pode conter pistas sobre a natureza legítima ou maliciosa do endereço.

1.1.2. Componentes de um URL

A Tabela 1 descreve cada um dos componentes presentes no URL de exemplo, identificando o seu papel na estrutura do endereço e o modo como cada parte pode revelar indícios de comportamento malicioso.

Exemplo:<https://login.banco.pt/user/verify?token=abc123&redirect=home#section>

Componente	Exemplo	Descrição
Esquema (Protocol)	https://	Define o protocolo de comunicação. HTTP não é seguro; HTTPS usa encriptação SSL/TLS.
Subdomínio	login.	Parte antes do domínio principal. Atacantes usam subdomínios longos ou suspeitos.

Domínio Principal	banco	Nome da entidade. Em URL maliciosos, pode imitar domínios legítimos (ex: 'b4nco').
TLD (Top-Level Domain)	.pt	Extensão do domínio. TLDs incomuns (.xyz, .tk) são sinais de alerta.
Caminho (<i>Path</i>)	/user/verify	Localização do recurso. Caminhos longos ou com palavras como 'verify' são suspeitos.
Parâmetros (<i>Query</i>)	?token=abc123	Dados passados ao servidor. Excesso de parâmetros pode indicar comportamento suspeito.
Âncora (Fragment)	#section	Referência a uma secção da página. Raramente usada em URL maliciosos.

Tabela 1- Tabela sobre os componentes de um URL

1.1.3. Características dos URL que indicam malícia

A análise lexical de um URL permite identificar um conjunto de padrões que, isoladamente ou em combinação, constituem indicadores de comportamento malicioso. Estes padrões foram identificados na literatura científica analisada no Projeto I e formam a base do processo de *feature engineering* descrito no Capítulo 1.3 deste relatório. Os principais indicadores são os seguintes:

- **Comprimento total do URL** – URL maliciosos tendem a ser mais longos para esconder o domínio real.
- **Presença de endereço IP em vez de domínio**
ex: http://192.168.1.1/login
- **Número de subdomínios** – URL legítimos raramente têm mais de 2 subdomínios.
- **Presença de @, -, // no caminho** – Caracteres usados para enganar o utilizador.
- **Uso de serviços de encurtamento** – bit.ly, tinyurl.com podem esconder URL maliciosos.
- **HTTPS vs HTTP** – A ausência de HTTPS é um indicador, embora não decisivo.
- **Comprimento do domínio** – Domínios muito longos ou com muitos hífenes são suspeitos.
- **Entropia do URL** – Medida da aleatoriedade de caracteres; alto valor indica URL gerado automaticamente.

1.2. Algoritmos de Machine Learning

O Projeto I menciona e compara vários algoritmos de ML nos artigos analisados, mas em nenhum momento se explica como cada um funciona. Esta secção preenche essa lacuna, sendo fundamental para justificar a escolha do Random Forest e compreender os resultados comparativos apresentados.

1.2.1. Árvores de Decisão (Decision Trees)

Uma Árvore de Decisão é o algoritmo base do Random Forest. Funciona como um conjunto de perguntas binárias organizadas em forma de árvore, em que cada nó representa um teste sobre uma característica dos dados e cada folha representa uma classificação final.

Como Funciona:

- O algoritmo analisa os dados de treino e escolhe a característica que melhor separa as classes (ex: URL maliciosos vs. benignos).
- O critério mais comum é a **Impureza de Gini** ou o **Ganho de Informação (Entropia)**, que medem quão bem uma característica divide os dados.
- O processo repete-se recursivamente para cada sub-ramo, até que um critério de paragem seja atingido (ex: profundidade máxima, número mínimo de amostras).
- O resultado é uma estrutura em árvore que pode ser interpretada visualmente.

Exemplo aplicado de URL:

```
Raiz: 'O comprimento do URL é > 75 caracteres?' → Sim →  
'Contém @?' → Sim → MALICIOSO.
```

Vantagens: fácil de interpretar, não requer normalização dos dados.

Desvantagens: propensão a *overfitting* (aprende demasiado os dados de treino, perdendo capacidade de generalização).

1.2.2. Random Forest (RF)

O Random Forest é o algoritmo selecionado no Projeto I e o que obteve melhor desempenho na maioria dos estudos analisados. É um método de **ensemble learning** que combina múltiplas Árvores de Decisão para produzir uma previsão mais robusta e precisa.

Princípio de funcionamento:

- **Bagging (Bootstrap Aggregating):** São criadas N amostras aleatórias do *dataset* original com reposição. Para cada amostra é treinada uma árvore independente.
- **Aleatoriedade nas características:** Em cada divisão de nó, apenas um subconjunto aleatório de características é considerado. Isto garante diversidade entre as árvores.
- **Votação por maioria:** Para classificar um novo URL, cada árvore vota. A classe com mais votos é a previsão final.

Feature Importance:

O RF calcula automaticamente a importância de cada característica, medindo quanto cada variável contribuiu para as decisões de classificação.

A Tabela 2 descreve os principais hiperparâmetros do Random Forest, o seu papel no funcionamento do modelo e os valores tipicamente utilizados na literatura, incluindo os que foram considerados na fase de otimização descrita no Capítulo 4.

Tabela 2- Principais hiperparâmetros do Random Forest

Parâmetro	Descrição	Valor Típico
n_estimators	Número de árvores na floresta. Mais árvores = maior precisão, mas mais lento.	100 – 500
max_depth	Profundidade máxima de cada árvore. Limita o overfitting.	None ou 10–30
max_features	Nº de características a considerar em cada divisão.	'sqrt' (raiz do total)
min_samples_split	Nº mínimo de amostras para dividir um nó.	2 – 10
random_state	Semente aleatória para reprodutibilidade dos resultados.	42 (convenção)

1.2.3. Support Vector Machine (SVM)

O SVM é um algoritmo de classificação que procura encontrar o **hiperplano ótimo** que separa as duas classes (malicioso vs. benigno) com a maior **margem** possível. Os pontos de dados mais próximos do hiperplano são chamados de **vetores de suporte**.

Kernel Trick: Quando os dados não são linearmente separáveis, o SVM utiliza funções kernel para transformar os dados para um espaço de dimensão superior onde a separação linear se torna possível. Os kernels mais comuns são: Linear, RBF e Polinomial.

O SVM foi o melhor classificador num dos estudos analisados (com 95.4% de precisão [2]). No entanto, o RF foi considerado mais equilibrado pela literatura em geral, por apresentar menor sensibilidade à escala dos dados e menores requisitos de ajuste de hiperparâmetros.

1.2.4. Regressão Logística (Logistic Regression – LR)

Apesar do nome 'regressão', é um algoritmo de **classificação binária**. Calcula a probabilidade de um URL pertencer à classe 'malicioso' através da função sigmóide, que mapeia qualquer valor real para o intervalo [0, 1]. Se a probabilidade for superior a 0.5, o URL é classificado como malicioso. É simples, rápido e funciona bem como baseline, mas tem dificuldade com relações não lineares entre as características.

1.2.5. Naive Bayes (NB)

O Naive Bayes é um classificador probabilístico baseado no **Teorema de Bayes**, que calcula a probabilidade posterior de um URL ser malicioso dado o conjunto de características observadas. O pressuposto 'naive' (ingênuo) é que todas as características são independentes entre si. É computacionalmente muito eficiente, sendo o algoritmo mais rápido de todos.

1.2.6. K-Nearest Neighbors (KNN)

O KNN é um algoritmo de classificação não paramétrico. Para classificar um novo URL, encontra os K URL mais semelhantes no conjunto de treino (baseado na distância euclidiana) e atribui a classe mais frequente entre esses K vizinhos. Contudo, a maior desvantagem deste modelo, é que é computacionalmente caro para grandes *datasets*.

1.2.7. Gradient Boosting

Tal como o Random Forest, é um método de **ensemble**, mas em vez de treinar árvores em paralelo, o Gradient Boosting treina as árvores **sequencialmente**: cada nova árvore é treinada para corrigir os erros da árvore anterior. Variantes populares incluem o **XGBoost**, **LightGBM** e **CatBoost**.

1.3. Feature Engineering para URL

O *Feature Engineering* (Engenharia de Características) é o processo de extrair e transformar os dados brutos (neste caso, uma *string* de URL) em variáveis numéricas que um modelo de ML consiga processar. É uma das etapas mais críticas do projeto, a qualidade das características determina em grande parte a qualidade do modelo.

1.3.1. Características Lexicais

As características lexicais são extraídas diretamente da *string* do URL, sem necessidade de aceder à página ou realizar consultas externas. Por esse motivo, são as mais utilizadas na literatura científica analisada no Projeto I [6][7], apresentando baixo custo computacional e elevado poder discriminante. A Tabela 3 descreve as principais características lexicais utilizadas neste projeto, indicando o que cada uma mede e de que forma se relaciona com a malícia do URL.

Tabela 3- Características Lexicais de um URL

Característica	Descrição	Indicador
Comprimento total do URL	Nº total de caracteres	URL longos → suspeito
Comprimento do domínio	Nº de chars no domínio	Domínios longos → suspeito
Comprimento do caminho	Nº de chars no <i>path</i>	Caminhos muito longos → suspeito

Nº de pontos no URL	Contagem de '.'	Muitos pontos → subdomínios falsos
Nº de hífenes no domínio	Contagem de '-'	Hífenes excessivos → suspeito
Nº de subdomínios	Segmentos antes do domínio	> 3 subdomínios → alerta
Presença de IP no URL	0 ou 1 (booleano)	IP em vez de nome → suspeito
Presença de '@'	0 ou 1 (booleano)	'@' ignora parte anterior → <i>phishing</i>
Presença de 'http' no <i>path</i>	0 ou 1 (booleano)	Redirecionamento duplo → <i>phishing</i>
TLD suspeito	Lista de TLDs de risco	.tk, .xyz, .ml → maior risco
Nº de parâmetros (<i>query</i>)	Nº de pares chave=valor	Muitos params → suspeito
Entropia do domínio	Medida de aleatoriedade	Alta entropia → gerado por bot
Uso de encurtadores	bit.ly, tinyurl, etc.	Ocultar URL real → suspeito
Nº de dígitos no URL	Contagem numérica	Muitos dígitos → suspeito

1.3.2. Características Baseadas no Domínio (WHOIS / DNS)

Para além das características extraídas diretamente da *string* do URL, é possível obter informações adicionais através de consultas externas ao registo do domínio. Estas características baseadas no domínio requerem acesso a serviços como o WHOIS e o DNS, o que implica um custo computacional superior às características lexicais. No entanto, apresentam elevado poder discriminante, uma vez que domínios recentemente registados ou com informação de registo incompleta são fortemente associados a atividade maliciosa. As principais características deste tipo são as seguintes:

- **Idade do domínio** – Domínios recentemente registados são muito suspeitos; a maioria dos domínios de *phishing* existe por menos de 30 dias.
- **Data de expiração** – Domínios registados por apenas 1 ano são mais suspeitos que os registados por 5+ anos.
- **Informação WHOIS** – Contém dados de registo: proprietário, data, localização. Domínios com WHOIS privado ou incompleto são suspeitos.
- **Reputação do DNS** – Verificar se o IP está em listas de bloqueio (*blacklists*) conhecidas.

- **Certificado SSL** – Verificar se existe e se o nome no certificado corresponde ao domínio.

1.4. Métricas de Avaliação de Modelos de ML

O Projeto I usa termos como 'precisão' (*accuracy*), 'F1-score' e 'taxa de falsos negativos' sem nunca os definir. Estas métricas são fundamentais para compreender e comparar os resultados dos modelos analisados.

1.4.1. Matriz de Confusão

A base de todas as métricas é a **Matriz de Confusão (tabela 4)**, que cruza as previsões do modelo com os valores reais.

Tabela 4- Matriz de Confusão

	Previsto: BENIGNO	Previsto: MALICIOSO
Real: BENIGNO	Verdadeiro Positivo (VP) URL benigno corretamente identificado	Falso Negativo (FN) URL benigno classificado como malicioso
Real: MALICIOSO	Falso Positivo (FP) URL malicioso classificado como benigno. Mais perigoso!	Verdadeiro Negativo (VN) URL malicioso corretamente identificado

1.4.2. Definições das Métricas Principais

A Tabela 5 define cada uma das métricas utilizadas ao longo deste projeto para avaliar o desempenho dos modelos, apresentando a fórmula de cálculo e o seu significado específico no contexto da deteção de URL maliciosos. É importante notar que, neste domínio, o *Recall* e a Taxa de Falsos Negativos são as métricas mais críticas: um URL malicioso que passe despercebido ao modelo pode resultar em roubo de credenciais ou instalação de *malware*, sendo este erro mais grave do que classificar erroneamente um URL benigno como malicioso.

Tabela 5 - Definições das métricas

Métrica	Fórmula	Significado no contexto de URL
Acurácia (Accuracy)	$(VP + VN) / \text{Total}$	% de URL classificados corretamente. Enganosa se o <i>dataset</i> é desequilibrado.
Precisão (Precision)	$VP / (VP + FP)$	De todos os URL classificados como maliciosos, quantos eram realmente maliciosos?
Recall (Sensibilidade)	$VP / (VP + FN)$	De todos os URL maliciosos, quantos foram detetados? Importante minimizar FN.
F1-Score	$2 \times (\text{Precisão} \times \text{Recall}) / (\text{Precisão} + \text{Recall})$	Média harmónica entre Precisão e Recall. Boa métrica quando há desequilíbrio.

Taxa de Falsos Negativos (FNR)	$FN / (FN + VP)$	% de URL maliciosos não detetados. Deve ser minimizada — um URL malicioso não detetado é perigoso.
AUC-ROC	Área sob a curva ROC	Mede a capacidade geral do modelo de distinguir entre classes. Valor entre 0 e 1.

No contexto de deteção de URL maliciosos, o *Recall* e a Taxa de Falsos Negativos são as métricas mais críticas: um URL malicioso que passe despercebido pode resultar em roubo de credenciais, perdas financeiras ou instalação de *malware*. É preferível classificar erroneamente um URL benigno (Falso Positivo) do que deixar passar um malicioso.

1.5. Pipeline do Projeto I

O relatório original menciona no Trabalho Futuro as etapas de 'pré-processamento de dados' e 'treino e otimização', sem nunca explicar em que consistem. Um Pipeline de ML é a sequência estruturada de etapas que transforma dados brutos num modelo funcional. Na tabela 6, iremos analisar cada etapa, e como se encaixa no contexto dos URL maliciosos.

Tabela 6 - Etapas do Pipeline de Machine Learning

Etapa	Descrição	No contexto de URL maliciosos
1. Recolha de Dados	Obter o <i>dataset</i> com exemplos rotulados	<i>Dataset</i> com URL benignos e maliciosos etiquetados
2. Análise Exploratória	Compreender a distribuição dos dados	Verificar equilíbrio entre classes, visualizar características
3. Pré-processamento	Limpar e preparar os dados	Remover duplicados, tratar valores em falta, normalizar
4. <i>Feature Engineering</i>	Extraír características numéricas	Extraír comprimento URL, nº de pontos, presença de @, etc.
5. Divisão Treino/Teste	Separar dados para avaliação justa	Tipicamente 80% treino, 20% teste (ou cross-validation)
6. Treino do Modelo	Ajustar os parâmetros do algoritmo	Treinar Random Forest com dados de treino
7. Avaliação	Medir desempenho no conjunto de teste	Calcular Accuracy, F1-Score, Recall, etc.
8. Otimização	Ajustar hiperparâmetros	Grid Search ou Random Search para <code>n_estimators</code> , <code>max_depth</code>

9. Validação Cruzada	Garantir generalização do modelo	K-Fold Cross Validation para resultados mais robustos
10. Deploy	Disponibilizar o modelo para uso real	API ou interface que recebe URL e devolve classificação

Para avaliar um modelo de forma justa, os dados devem ser divididos em conjuntos separados de treino e teste. A **Validação Cruzada K-Fold** divide os dados em k partes iguais, treinando K vezes e calculando a média dos resultados.

1.6. Scikit-learn

O relatório menciona o PyTorch como biblioteca de ML, mas o **Scikit-learn** (que é a biblioteca usada em *todos os artigos analisados* para implementar Random Forest, SVM, Logistic Regression, etc.), está completamente ausente do capítulo de Tecnologias e Ferramentas.

Exemplo do uso do Random Forest:

```

1  from sklearn.ensemble import RandomForestClassifier
2  from sklearn.model_selection import train_test_split
3  from sklearn.metrics import classification_report
4
5  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
6
7  model = RandomForestClassifier(n_estimators=100, random_state=42)
8
9  model.fit(X_train, y_train)
10
11 print(classification_report(y_test, model.predict(X_test)))

```

1.7. Pandas, NumPy e Matplotlib– Bibliotecas Ausentes

O Projeto I selecionou o Python como linguagem de desenvolvimento e mencionou o PyTorch como biblioteca de Machine Learning. No entanto, as bibliotecas efetivamente utilizadas ao longo deste projeto (Pandas, NumPy, Matplotlib e outras), estavam completamente ausentes do capítulo de tecnologias do relatório anterior. A Tabela 7 descreve cada uma dessas bibliotecas, a sua função principal e a forma como foi aplicada no contexto deste projeto.

Tabela 7- Bibliotecas

Biblioteca	Função Principal	Uso no Projeto
Pandas	Manipulação de dados tabulares (DataFrames)	Carregar o <i>dataset</i> CSV, filtrar, limpar, transformar os dados de URL
NumPy	Computação numérica com arrays multidimensionais	Operações matemáticas sobre os vetores de características extraídas

Matplotlib/Seaborn	Visualização de dados e resultados	Gráficos de distribuição de características, curva ROC, matriz de confusão
re (Regex)	Expressões regulares para análise de texto	Extrair componentes do URL (domínio, <i>path</i> , parâmetros)
urllib/tldextract	Parsing estruturado de URL	Separar automaticamente protocolo, subdomínio, domínio, TLD

1.8. Conceitos de Cibersegurança em Falta

O Projeto I aborda conceitos de cibersegurança como o *spoofing*, o DNS e o *malware* de forma contextual, sem nunca os definir explicitamente. Uma vez que estes conceitos são referenciados ao longo de todo o relatório, tanto na motivação do projeto como na análise dos artigos científicos, a sua compreensão é fundamental para interpretar corretamente as decisões tomadas nas fases de implementação. As secções seguintes definem cada um destes conceitos e explicam a sua relevância no contexto da deteção de URL maliciosos.

1.8.1. Spoofing

O **spoofing** é uma técnica em que um atacante falsifica a sua identidade digital para se fazer passar por uma entidade legítima. No caso mencionado no relatório (ataque dos CTT), foi aplicado **SMS Spoofing**: o atacante manipulou o identificador do remetente (Sender ID).

- IP Spoofing – Falsificação do endereço IP de origem de pacotes de rede.
- Email Spoofing – Falsificação do campo 'De' num email.
- DNS Spoofing – Corrupção da cache DNS para redirecionar utilizadores para sites falsos.
- Caller ID Spoofing – Falsificação do número de telefone do chamador.

1.8.2. DNS (Domain Name System)

O DNS é o 'livro de endereços' da Internet. Traduz nomes de domínio legíveis por humanos (ex: www.google.com) nos endereços IP numéricos correspondentes (ex: 142.250.180.4) que os computadores utilizam para comunicar.

1.8.3. Malware e Tipos de Ameaças Web

O Projeto I referencia termos como *malware*, *phishing* e *spoofing* sem nunca os distinguir de forma sistemática. No contexto da deteção de URL maliciosos, é fundamental compreender as diferentes categorias de ameaças web, uma vez que cada tipo apresenta características lexicais distintas que o modelo de Machine Learning aprende a identificar. A tabela seguinte descreve os principais tipos de ameaças abordados nos artigos analisados.

Tabela 8- *Malware* e Tipos de Ameaças Web

Tipo	Descrição
Vírus	Código que se propaga infectando outros ficheiros, requerendo execução humana.
Trojan (Cavalo de Troia)	Parece software legítimo mas executa ações maliciosas em segundo plano.
Ransomware	Encripta os ficheiros da vítima e exige pagamento pela chave de descriptação.
Spyware	Recolhe informação do utilizador (keylogging, screenshots) sem o seu conhecimento.
Adware	Exibe publicidade indesejada; pode redirecionar o browser para sites maliciosos.
Botnet	Rede de computadores infectados controlados remotamente por um atacante (C&C).
Drive-by Download	Download automático de <i>malware</i> ao visitar uma página web maliciosa.

1.8.4. Blacklists e Whitelists

As *blacklists* e *whitelists* constituem a abordagem tradicional de deteção de URL maliciosos, referenciada no Projeto I como o método que os sistemas de Machine Learning vieram superar. Compreender o seu funcionamento é essencial para contextualizar as limitações das abordagens estáticas e justificar a adoção de métodos preditivos. Existem três categorias principais:

- **Blacklist (Lista Negra):** Lista de URL, domínios ou IPs conhecidos como maliciosos. Limitação: só bloqueia ameaças conhecidas — ineficaz contra ataques zero-day.
- **Whitelist (Lista Branca):** Lista de URL considerados seguros. Qualquer URL que não conste na lista é bloqueado. Mais seguro, mas impraticável a grande escala.
- **Graylist:** Abordagem intermédia - URL desconhecidos são quarentenados para análise.

1.9. Técnicas Avançadas Mencionadas nos Artigos

A revisão sistemática de literatura realizada no Projeto I identificou, em vários dos artigos analisados, referências a técnicas avançadas que vão além dos algoritmos clássicos de Machine Learning. Embora este projeto tenha optado pelo Random Forest como modelo principal, pela sua eficiência e interpretabilidade, é importante compreender estas abordagens para contextualizar as escolhas feitas e reconhecer o estado da arte. As secções seguintes descrevem as técnicas mais referenciadas.

1.9.1. BERT e Modelos Transformer

BERT (Bidirectional Encoder Representations from Transformers) é um modelo de NLP pré-treinado desenvolvido pela Google. A sua característica fundamental é ser **bidirecional**: analisa o contexto de uma palavra tendo em conta tanto o que vem antes como o que vem depois. No contexto de deteção de URL (artigo [8] – BERT-CNN), é usado para capturar relações semânticas entre os componentes textuais do URL.

1.9.2. Graph Neural Networks (GNN)

As **Graph Neural Networks** são redes neuronais desenhadas para operar sobre estruturas de grafo. No contexto de URL maliciosos, um grafo pode representar as relações entre domínios, IPs e subdomínios: se vários URL maliciosos partilham o mesmo servidor (IP), esta relação estrutural pode ser explorada pelo modelo para melhorar a deteção.

1.9.3. GANs para URL Adversariais

As **GANs (Generative Adversarial Networks)** são usadas para **gerar URL adversariais** (URL maliciosos ligeiramente modificados para enganar o classificador). Ao incluir estes exemplos no treino, o modelo torna-se mais robusto contra atacantes que tentem contornar a deteção.

1.9.4. XAI – Inteligência Artificial Explicável (SHAP e LIME)

A Inteligência Artificial Explicável (XAI) surge como resposta à necessidade de compreender de que forma os modelos de Machine Learning chegam às suas decisões (algo particularmente relevante em cibersegurança, onde a interpretabilidade das classificações é fundamental para a confiança do utilizador). No contexto deste projeto, o Random Forest já oferece uma forma de explicabilidade através da *Feature Importance*. Os dois métodos de XAI mais referenciados na literatura analisada são os seguintes:

- **SHAP (SHapley Additive exPlanations)**: Atribui a cada característica um valor que indica quanto essa característica contribuiu para a previsão. Por exemplo, para um URL classificado como malicioso, o SHAP pode indicar que a 'presença de @ no URL' foi o fator mais determinante.
- **LIME (Local Interpretable Model-agnostic Explanations)**: Cria um modelo linear simples que aproxima o comportamento do modelo complexo na vizinhança de uma previsão específica.

1.10. Overfitting, Underfitting e Validação Cruzada

O Projeto I menciona no Trabalho Futuro a necessidade de "treino e otimização" do modelo sem nunca explicar os riscos associados a este processo. Em qualquer pipeline de Machine Learning, dois problemas opostos podem comprometer a capacidade de generalização do modelo para dados novos: o *overfitting* e o *underfitting*. Compreender estes conceitos é indispensável para interpretar as

decisões de otimização descritas no Capítulo 4, nomeadamente a escolha dos hiperparâmetros e a utilização de validação cruzada como método de avaliação.

1.10.1. Overfitting

O *overfitting* ocorre quando um modelo aprende demasiado bem os dados de treino, incluindo o ruído e as peculiaridades específicas desse conjunto, e por isso perde a capacidade de generalizar para novos dados. Um modelo com *overfitting* tem alta precisão no treino e baixa precisão no teste.

Uma boa analogia para explicar o *overfitting*, seria “Um estudante que decora todas as perguntas e respostas do exame anterior, mas não percebe a matéria. Se o exame for diferente, falha.”

1.10.2. Underfitting

O *underfitting* ocorre quando o modelo é demasiado simples para capturar os padrões dos dados. Tem baixa precisão tanto no treino como no teste. O objetivo é encontrar o equilíbrio entre os dois extremos, chamado de ***bias-variance tradeoff***.

1.10.3. Como o Random Forest Mitiga o Overfitting

Uma das principais vantagens do Random Forest face a uma Árvore de Decisão individual é precisamente a sua resistência ao *overfitting*. Esta resistência resulta de três mecanismos estruturais que introduzem diversidade entre as árvores que compõem o ensemble, impedindo que o modelo memorize os dados de treino em vez de aprender os seus padrões gerais:

- **Bagging:** Cada árvore é treinada numa amostra diferente dos dados. Os erros individuais anulam-se na votação final.
- **Aleatoriedade nas características:** As árvores tornam-se diversas e menos correlacionadas entre si.
- **Ensemble:** A combinação de múltiplos modelos fracos produz um modelo forte com melhor generalização.

1.10.4. Hiperparâmetro e Otimização

Os **hiperparâmetros** são os parâmetros do modelo que são definidos antes do treino. A otimização pode ser feita com:

- **Grid Search:** Testa todas as combinações possíveis de hiperparâmetros. Exaustivo, mas computacionalmente caro.
- **Random Search:** Testa combinações aleatórias. Mais eficiente que Grid Search para espaços grandes.
- **Bayesian Optimization:** Método inteligente que usa os resultados anteriores para guiar a próxima combinação.

1.11. Conclusão do Capítulo

Este capítulo preencheu as principais lacunas conceptuais identificadas no relatório de Projeto I. Foram explicados a estrutura e os componentes de um URL, os algoritmos de Machine Learning comparados na literatura, as métricas de avaliação utilizadas ao longo do projeto, as bibliotecas Python efetivamente empregues, os conceitos de cibersegurança relevantes e as técnicas avançadas referenciadas nos artigos analisados.

Com este enquadramento estabelecido, o Capítulo 2 inicia a componente prática do projeto, descrevendo o processo de pré-processamento dos dados (a primeira etapa do pipeline de Machine Learning definido na Tabela 6).

2. Pré Processamento dos Dados

O pré-processamento de dados constitui uma etapa fundamental em qualquer pipeline de Machine Learning. Antes de ser possível treinar um modelo, é necessário garantir que os dados se encontram limpos, consistentes e adequadamente formatados. Esta fase engloba a inspeção inicial do *dataset*, a identificação e resolução de problemas de qualidade dos dados e a preparação do conjunto de dados para as fases subsequentes do projeto, nomeadamente a extração de características e o treino do modelo.

Para a implementação desta etapa, foram utilizadas as bibliotecas Python **Pandas** para manipulação de dados tabulares, **NumPy** para operações numéricas, e **Matplotlib** e **Seaborn** para a geração de visualizações. O ambiente de desenvolvimento utilizado foi o **Jupyter Notebook**, conforme definido no plano tecnológico do projeto.

2.1. Carregamento e Inspeção Inicial

O *dataset* selecionado no Projeto I foi carregado através da função **pd.read_csv()** da biblioteca Pandas. Dado que o ficheiro contém caracteres de codificações diversas, foi necessário especificar o parâmetro *encoding='latin1'* para garantir a leitura correta de todos os registos, bem como o parâmetro *low_memory=False* para evitar inferência incorreta de tipos de dados.

O *dataset* contém **95.913 instâncias e 14 colunas**, conforme apresentado na Tabela 9, que descreve o significado de cada variável.

Tabela 9- Descrição das variáveis do *dataset*

Coluna>	Tipo	Descrição
domain	object	URL completo do domínio analisado
ranking	object*	Ranking de popularidade do domínio (*convertido para Int64)
mld_res	int64	Presença do domínio principal em lista de referência (0/1)
mld.ps_res	int64	Presença do domínio+sufixo em lista de referência (0/1)
card_rem	int64	Cardinalidade dos tokens restantes no URL
ratio_Rrem	float64	Rácio de tokens restantes face à lista de referência
ratio_Arem	float64	Rácio de tokens da amostra face à lista de referência
jaccard_RR	float64	Similaridade de Jaccard: referência vs. referência
jaccard_RA	float64	Similaridade de Jaccard: referência vs. amostra
jaccard_AR	float64	Similaridade de Jaccard: amostra vs. referência
jaccard_AA	float64	Similaridade de Jaccard: amostra vs. amostra
jaccard_ARrd	float64	Jaccard AR sem elementos duplicados

jaccard_ARrem	float64	Jaccard AR dos tokens restantes
label	int64	Classe: 0 = Benigno 1 = Malicioso

2.2. Distribuição de Classes

Uma das primeiras verificações a realizar em problemas de classificação binária é a análise da distribuição de classes. Um *dataset* severamente desequilibrado pode levar o modelo a aprender a classificar a classe majoritária sistematicamente, resultando em métricas de desempenho enganosas.

A análise da distribuição (Figura 1) revelou que o *dataset* se encontra praticamente equilibrado, contendo **48.009 URL benignos (50,1%)** e **47.904 URL maliciosos (49,9%)**, com um rácio de desequilíbrio de **1,00:1**. Este resultado é muito favorável, eliminando a necessidade de recorrer a técnicas de sobreamostragem como SMOTE ou de ajuste de pesos de classe (*class_weight*) durante o treino do modelo.

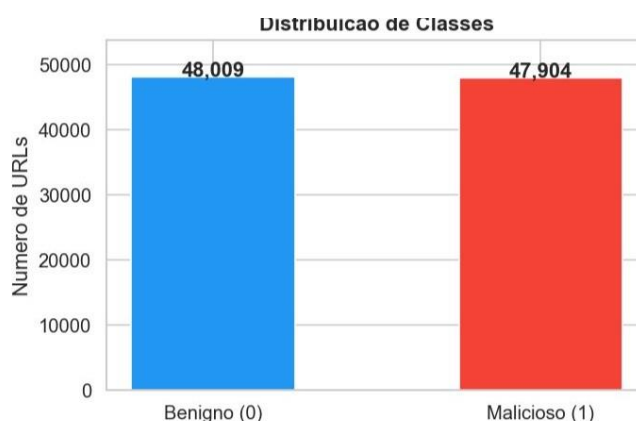


Figura 1- Distribuição de classes do *dataset* (benignos vs maliciosos)

2.3. Identificação de Problemas de Qualidade

Antes de aplicar qualquer transformação, procedeu-se a uma inspeção sistemática dos dados para identificar eventuais problemas de qualidade. Foram verificados quatro aspetos: a presença de valores nulos, a existência de registos duplicados, a validade dos tipos de dados e a presença de valores atípicos (*outliers*).

2.3.1. Valores Nulos

A verificação da presença de valores nulos, realizada através do método *isnull().sum()* do Pandas, demonstrou que **todas as 14 colunas apresentam zero valores em falta** no conjunto completo das 95.913 instâncias. Esta ausência de valores nulos elimina a necessidade de imputação de dados, simplificando o pipeline de pré-processamento.

2.3.2. Registos Duplicados

A análise de duplicados identificou **1 linha completamente duplicada** e **2 domínios repetidos** no *dataset*. O registo duplicado corresponde ao domínio *bin/webscr?cmd=_login-* (label = 1, malicioso), presente nas linhas 12.011 e 35.319. A eliminação de duplicados é essencial para evitar que o modelo aprenda padrões artificialmente reforçados pela repetição de instâncias idênticas.

2.3.3. Registo Corrompido

A análise da coluna *ranking* revelou um registo inválido na linha **18.231**, cujo campo *domain* contém uma sequência de caracteres ilegíveis resultante de um problema de codificação (*encoding*). O valor de *ranking* deste registo não é convertível para formato numérico. Por se tratar de dados corrompidos sem possibilidade de recuperação, optou-se pela remoção desta instância.

2.3.4. Valores Atípicos (Outliers)

A identificação de valores atípicos foi realizada através do método **IQR (Interquartile Range)**. Este método define como *outlier* qualquer valor superior ao limite $Q3 + 1,5 \times IQR$, onde Q1 e Q3 correspondem ao primeiro e terceiro quartis, respetivamente. A análise focou-se nas três variáveis contínuas com maior variabilidade: *card_rem*, *ratio_Rrem* e *ratio_Arem*. A Tabela 10 apresenta os resultados desta análise para as três colunas identificadas como problemáticas, indicando os valores mínimos, máximos originais, os limites calculados pelo método IQR e o número de *outliers* detetados em cada variável.

Tabela 10- Análise de outliers pelo método IQR

Coluna	Mín.	Máx. Original	Limite IQR	Outliers	%
card_rem	0,00	58,00	12,00	5.226	5,4%
ratio_Rrem	0,00	5.507,00	370,86	4.626	4,8%
ratio_Arem	0,00	6.097,00	386,23	4.756	5,0%

Como se pode constatar na Tabela 10, as três colunas apresentam valores máximos consideravelmente superiores aos respetivos limites IQR. A coluna *ratio_Rrem* regista o caso mais extremo, com um valor máximo de **5.507** face a um limite IQR de **370,86**. A Figura 2 ilustra visualmente a extensão destes *outliers* através de *boxplots*.

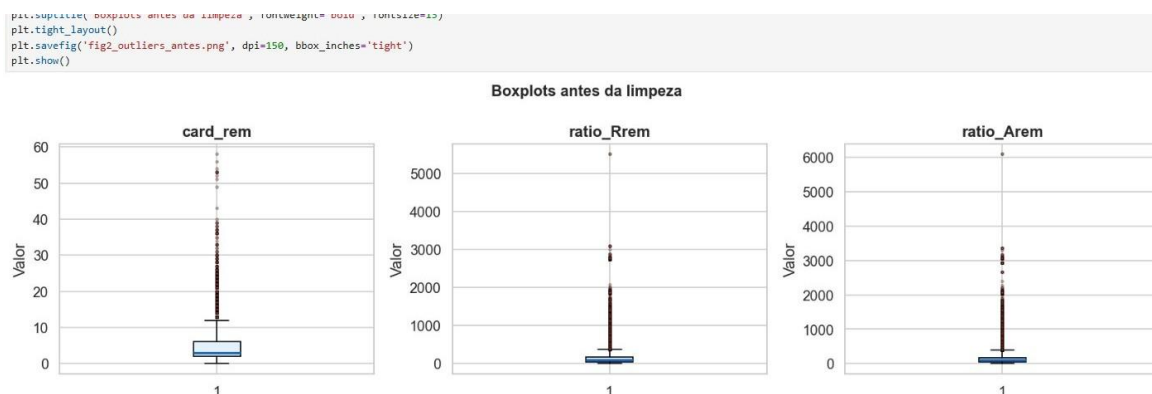


Figura 2- Boxplots das variáveis contínuas antes da limpeza – outliers extremos visíveis

2.4. Aplicação da Limpeza

Com base nos problemas identificados na secção anterior, foram aplicadas as seguintes operações de limpeza, por ordem de execução (tabela 11):

Tabela 11- Operações de limpeza aplicadas ao *dataset*

#	Problema	Ação Aplicada	Resultado
1	Registo corrompido (linha 18.231)	Remoção da linha via <code>drop(index=...)</code>	1 instância eliminada
2	Linha completamente duplicada	Remoção via <code>drop_duplicates()</code>	1 instância eliminada
3	Coluna ranking em formato string	Conversão via <code>pd.to_numeric().astype('Int64')</code>	Tipo: object → Int64
4	Outliers em <code>card_rem</code> , <code>ratio_Rrem</code> , <code>ratio_Arem</code>	Capping pelo limite IQR ($Q3 + 1,5 \times IQR$)	14.608 valores limitados

Para o tratamento dos *outliers* optou-se pela técnica de **Capping** (também designada *Winsorizing*), que substitui os valores superiores ao limite IQR pelo próprio limite, em vez de os eliminar. Esta abordagem preserva todas as instâncias do *dataset*, limitando apenas o impacto dos valores extremos no modelo, o que é preferível à remoção de dados em *datasets* de dimensão moderada.

Excerto de código — Aplicação do Capping IQR:

```
1 for col in ['card_rem', 'ratio_Rrem', 'ratio_Arem']:
2     Q1 = df_clean[col].quantile(0.25)
3     Q3 = df_clean[col].quantile(0.75)
4
5     lim = Q3 + 1.5 * (Q3 - Q1)
6
7     df_clean[col] = df_clean[col].clip(upper=lim)
```

2.5. Verificação Pós-Limpeza

Após a aplicação de todas as operações de limpeza, procedeu-se a uma verificação sistemática para confirmar que todos os problemas identificados foram resolvidos com sucesso (tabela 12).

Tabela 12- Comparação do *dataset* antes e depois da limpeza

Métrica	Antes da Limpeza	Depois da Limpeza
Nº total de instâncias	95.913	95.911
Valores nulos	0	0
Registos duplicados	1	0
Tipo coluna ranking	object (string)	Int64
Máx. card_rem	58,00	12,00
Máx. ratio_Rrem	5.507,00	370,86
Máx. ratio_Arem	6.097,00	386,23
URL Benignos (0)	48.009 (50,1%)	48.009 (50,1%)
URL Maliciosos (1)	47.904 (49,9%)	47.902 (49,9%)

A Figura 3 apresenta os *boxplots* das três variáveis contínuas após a aplicação do Capping IQR. A eliminação dos valores extremos é evidente na comparação com a Figura 2, com os valores máximos a tornarem-se significativamente mais próximos da mediana e do intervalo interquartil.

```
plt.suptitle('Boxplots depois da limpeza (Capping IQR)', fontweight='bold', fontsize=13)
plt.tight_layout()
plt.savefig('fig3_outliers_depois.png', dpi=150, bbox_inches='tight')
plt.show()
```

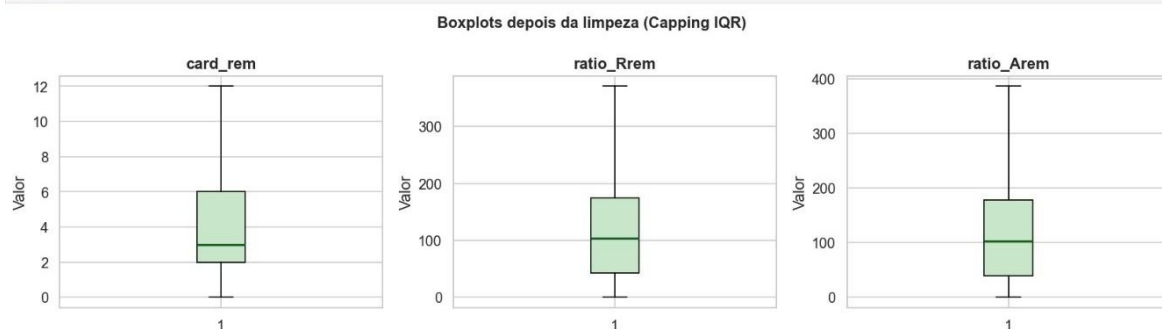


Figura 3- Boxplots das variáveis contínuas após a limpeza (Capping IQR)

É de salientar que a distribuição de classes se manteve praticamente inalterada após a limpeza: **48.009 URL benignos (50,1%)** e **47.902 URL maliciosos (49,9%)**. A redução de 2 instâncias na classe maliciosa corresponde à remoção do registo corrompido e do duplicado, ambos com label = 1.

2.6. Exportação do *Dataset* Limpo

O *dataset* resultante do processo de limpeza foi exportado para o ficheiro **urldata_clean.csv** em formato UTF-8, contendo **95.911 instâncias** e **14 colunas**, pronto para ser utilizado na fase seguinte do pipeline (a normalização e extração de características).

Excerto de código — Exportação:

```
1 df_clean.to_csv('urldata_clean.csv', index=False, encoding='utf-8')
```

2.7. Normalização das Variáveis

Após a limpeza dos dados, procedeu-se à normalização das variáveis numéricas do *dataset*. Esta etapa é essencial para garantir que características com escalas muito diferentes não introduzam enviesamentos no processo de treino do modelo. Algoritmos como o SVM e o KNN são particularmente sensíveis à escala das variáveis, pelo que a normalização é uma prática recomendada mesmo para algoritmos baseados em árvores de decisão, como o Random Forest, quando se pretende comparar o desempenho entre diferentes classificadores.

2.7.1. Análise das Escalas

Antes de aplicar qualquer transformação, analisaram-se os intervalos de cada variável numérica do *dataset* limpo (tabela 13). Esta análise revelou que apenas 3 das 11 variáveis numéricas necessitavam de normalização. As restantes 8 já se encontravam naturalmente no intervalo [0, 1], como é o caso de todas as variáveis de similaridade de Jaccard e das variáveis binárias *mld_res* e *mld.ps_res*.

Tabela 13- Análise das escalas das variáveis numéricas

Coluna	Mín.	Máx.	Normalizar ?	Justificação
<i>mld_res</i>	0	1	Não	Já em [0, 1] — variável binária
<i>mld.ps_res</i>	0	1	Não	Já em [0, 1] — variável binária
<i>card_rem</i>	0	12	Sim ✓	Escala inteira, máx. após capping: 12
<i>ratio_Rrem</i>	0	370,86	Sim ✓	Escala contínua, máx. após capping: 370,86
<i>ratio_Arem</i>	0	386,23	Sim ✓	Escala contínua, máx. após capping: 386,23
<i>jaccard_RR</i>	0	1	Não	Já em [0, 1] — similaridade de Jaccard
<i>jaccard_RA</i>	0	0,917	Não	Já em [0, 1] — similaridade de Jaccard
<i>jaccard_AR</i>	0	1	Não	Já em [0, 1] — similaridade de Jaccard

jaccard_AA	0	1	Não	Já em [0, 1] — similaridade de Jaccard
jaccard_ARrd	0	1	Não	Já em [0, 1] — similaridade de Jaccard
jaccard_ARrem	0	0,968	Não	Já em [0, 1] — similaridade de Jaccard

2.7.2. Aplicação do Min-Max Scaler

Para a normalização das três colunas identificadas, foi utilizado o Min-Max Scaler da biblioteca Scikit-learn, que transforma cada valor segundo a fórmula:

$$x_{\text{norm}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

Esta transformação mapeia todos os valores para o intervalo [0, 1], preservando a distribuição original dos dados e a relação proporcional entre os valores. As colunas *domain*, *ranking* e *label* não foram normalizadas: a primeira é texto, a segunda é um identificador ordinal e a terceira é a variável alvo da classificação.

Como se pode confirmar na Tabela 14, os valores mínimos e máximos das três colunas após a transformação passam a ser, respetivamente, 0,000 e 1,000, confirmando a correta aplicação do Min-Max Scaler e a uniformização das escalas.

Tabela 14- Resultado da normalização Min-Max nas três colunas

Coluna	Mín. Antes	Máx. Antes	Mín. Depois	Máx. Depois
card_rem	0,000	12,000	0,000	1,000
ratio_Rrem	0,000	370,860	0,000	1,000
ratio_Arem	0,000	386,230	0,000	1,000

A Figura 4 apresenta os histogramas das três variáveis normalizadas, comparando a distribuição antes (vermelho) e depois (azul) da aplicação do Min-Max Scaler. Como se pode observar, a forma da distribuição é preservada na íntegra. Apenas a escala do eixo horizontal é alterada, passando de valores absolutos para o intervalo [0, 1].

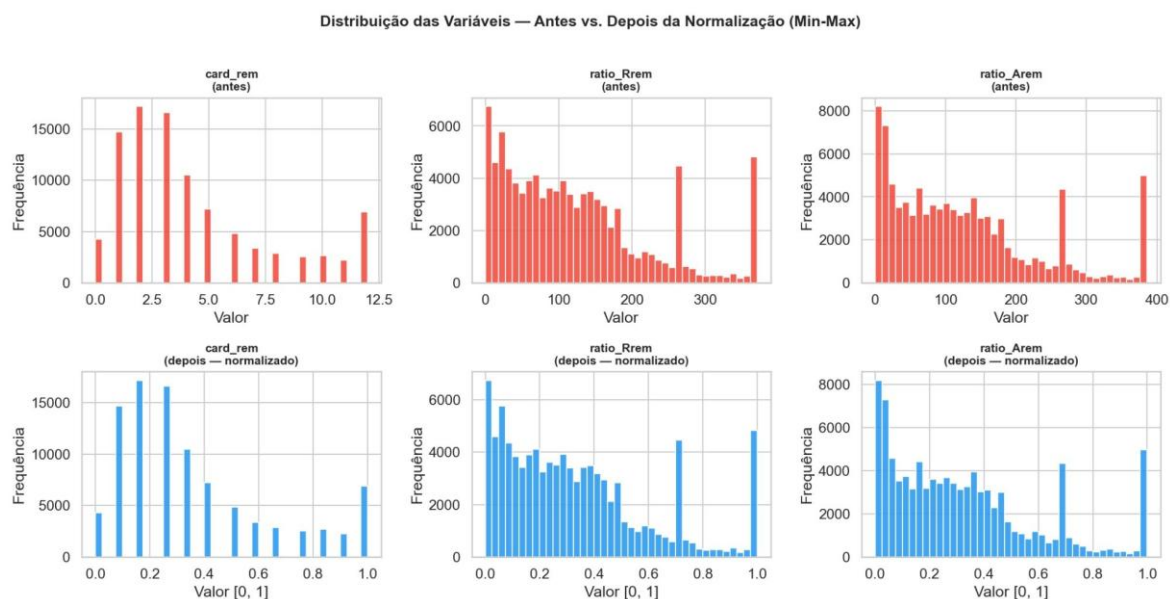


Figura 4- Distribuição das variáveis antes (vermelho) e depois (azul) da normalização Min-Max

O *dataset* resultante desta etapa, guardado no ficheiro `urldata_normalized.csv`, contém 95.911 instâncias e 14 colunas, com todas as variáveis numéricas em escalas comparáveis, ficando assim pronto para a fase de engenharia de características.

2.8. Engenharia de Características

A engenharia de características é o processo de extrair e criar novas variáveis a partir dos dados brutos, com o objetivo de melhorar a capacidade discriminante do modelo de Machine Learning. No contexto deste projeto, o *dataset* original já contém características calculadas com base em métricas de similaridade (Jaccard) e rácios de *tokens*. Contudo, a literatura científica vista no Projeto I [6][7][9] evidencia que as **características lexicais** extraídas diretamente da *string* do URL são das mais discriminantes e amplamente utilizadas. Nesta etapa, foram extraídas **10 características lexicais** da coluna *domain*, que contém o URL completo de cada instância. A extração foi implementada em Python com recurso às bibliotecas `re` (expressões regulares) e `urllib.parse` (análise estrutural de URL).

2.8.1. Características Extraídas

As 10 características foram organizadas em três grupos funcionais: comprimentos, que medem a extensão de diferentes componentes do URL; contagens, que quantificam a ocorrência de caracteres ou elementos específicos; e indicadores binários, que sinalizam a presença ou ausência de padrões associados a malícia. A Tabela 15 descreve cada uma das características extraídas, a sua definição técnica e a relação esperada com a classificação do URL.

Tabela 15- Características Lexicais extraídas da coluna domain

Grupo	Feature	Descrição	Indicador de Malícia
Comprimentos	lex_comp_url	Comprimento total do URL	URL maliciosos tendem a ser mais longos
	lex_comp dominio	Comprimento do domínio	Domínios de <i>phishing</i> costumam ser mais compridos
Contagens	lex_n_pontos	Número de pontos no URL	Muitos pontos indicam subdomínios falsos
	lex_n_hifenes	Número de hifenes no URL	Hifenes excessivos imitam domínios legítimos
	lex_n_subdominios	Número de subdomínios	<i>Phishing</i> usa subdomínios para parecer legítimo
	lex_n_digitos	Número de dígitos no URL	URL gerados automaticamente têm muitos dígitos
	lex_profundidade	Profundidade do <i>path</i> ($/a/b = 2$)	<i>Paths</i> muito profundos são suspeitos
Indicadores Binários	lex_tem_https	Uso de HTTPS (0/1)	Ausência de HTTPS é sinal de alerta
	lex_tem_ip	IP em vez de domínio (0/1)	IP no lugar do nome é classicamente malicioso
	lex_tem_arroba	Presença de @ no URL (0/1)	O @ ignora tudo o que vem antes — técnica de <i>phishing</i>

2.8.2. Estatísticas das Novas Características

De forma a validar a extração e compreender a distribuição de cada característica no *dataset*, foram calculadas as estatísticas descritivas das 10 novas variáveis lexicais sobre o conjunto completo das 95.911 instâncias. A Tabela 16 apresenta, para cada característica, a média, o desvio padrão, o valor mínimo, a mediana e o valor máximo observado. A linha assinalada a amarelo corresponde à característica *lex_tem_https*, que apresenta variância zero no *dataset*, o que indica que nenhum URL da coluna *domain* inclui o protocolo HTTPS de forma explícita, sendo este campo consequentemente excluído do treino do modelo.

Tabela 16- Estatísticas descritivas das características lexicais (linha amarela: variância zero no *dataset*)

<i>Feature</i>	Média	Desvio Padrão	Mín.	Mediana	Máx.
lex_comp_url	64,459	64,945	9	42	2175
lex_comp dominio	22,351	23,389	3	17	248
lex_n_pontos	3,269	2,330	0	3	37
lex_n_hifenes	0,656	1,518	0	0	36
lex_n_subdominios	1,342	1,962	0	1	33
lex_n_digitos	8,168	19,403	0	1	286
lex_profundidade	2,349	1,756	0	2	34
lex_tem_https	0,000	0,000	0	0	0
lex_tem_ip	0,000	0,014	0	0	1
lex_tem_arroba	0,002	0,050	0	0	1

2.8.3. Análise por Classe

Para avaliar o poder discriminante de cada característica, compararam-se os valores médios entre URL benignos e maliciosos, conforme apresentado na Tabela 17.

Tabela 17- Valores médios por classe e correlação com o label (ordenado por correlação decrescente)

<i>Feature</i>	Benigno (0)	Malicioso (1)	Diferença	Correlação c/ label
lex_comp_url	36,804	92,176	× 2,5	+0,4263
lex_n_digitos	1,532	14,819	× 9,7	+0,3424
lex_n_pontos	2,625	3,915	× 1,5	+0,2767
lex_n_hifenes	0,271	1,040	× 3,8	+0,2533
lex_comp dominio	16,677	28,038	× 1,7	+0,2429
lex_profundidade	1,991	2,709	× 1,4	+0,2044
lex_n_subdominios	1,121	1,563	× 1,4	+0,1126
lex_tem_arroba	0,000	0,005	—	+0,0436
lex_tem_ip	0,000	0,000	—	+0,0137
lex_tem_https	0,000	0,000	—	N/A

Os resultados da Tabela 17 são particularmente reveladores. A característica *lex_comp_url* apresenta a maior correlação com o *label* (+0,4263), com os URL

maliciosos a serem em média 2,5 vezes mais longos do que os benignos (92 vs. 37 caracteres). A característica `lex_n_digitos` destaca-se pela maior diferença proporcional entre classes: os URL maliciosos contêm em média 9,7 vezes mais dígitos (14,8 vs. 1,5), o que reflete a presença de *tokens* gerados automaticamente típicos de ataques de *phishing*. A Figura 5 ilustra estas diferenças visualmente.

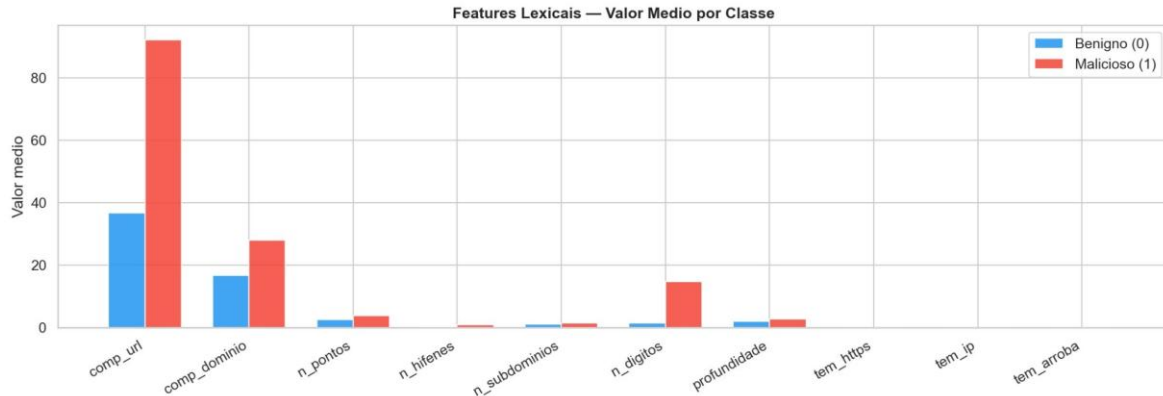


Figura 5- Valor médio das características lexicais por classe (benigno vs malicioso)

2.8.4. Dataset Final

As 10 novas características lexicais foram concatenadas ao *dataset* normalizado, resultando no ficheiro `urldata_features.csv`. Este ficheiro constitui o input para a fase de treino do modelo (Passo 3) e contém **95.911 instâncias e 24 colunas** (as 14 originais acrescidas das 10 novas características lexicais). A Tabela 18 resume a composição deste *dataset*, organizando as colunas pelos grupos a que pertencem e indicando o número de variáveis em cada grupo.

Tabela 18- Composição do *dataset* final (`urldata_features.csv`)

Grupo de Colunas	Colunas	Descrição
Identificação	2	domain, ranking
Originais — Binárias	2	mld_res, mld.ps_res
Originais — Normalizadas	3	card_rem, ratio_Rrem, ratio_Arem
Originais — Jaccard	6	jaccard_RR, jaccard_RA, jaccard_AR, jaccard_AA, jaccard_ARrd, jaccard_ARrem
Novas — Lexicais	10	lex_comp_url, lex_comp dominio, lex_n_pontos, lex_n_hifenes, lex_n_subdominios, lex_n_digitos, lex_profundidade, lex_tem_https, lex_tem_ip, lex_tem_arroba
Variável Alvo	1	label (0 = Benigno 1 = Malicioso)
TOTAL	24	—

2.9. Conclusão do Capítulo

O presente capítulo descreveu de forma detalhada todas as etapas do pré-processamento aplicado ao *dataset* selecionado no Projeto I. O processo iniciou-se com a inspeção e carregamento dos dados, seguindo-se a análise da distribuição de classes, a identificação e resolução de problemas de qualidade, nomeadamente valores nulos, registos duplicados, um registo corrompido e valores atípicos, e a exportação do *dataset* limpo.

Numa segunda fase, procedeu-se à normalização das três variáveis numéricas com escalas desproporcionadas através do Min-Max Scaler, e à extração de 10 novas características lexicais da coluna de URL, que vieram a revelar-se determinantes para o desempenho do modelo. O *dataset* final, composto por 95.911 instâncias e 24 colunas, encontra-se agora limpo, normalizado e enriquecido, constituindo a base de entrada para a fase de treino e comparação de modelos descrita no Capítulo 3.

3. Treino e Comparação de Modelos

Com o *dataset* completamente preparado (limpo, normalizado e enriquecido com características lexicais) procedeu-se ao treino e avaliação dos algoritmos de Machine Learning identificados na revisão de literatura do Projeto I. Esta fase constitui o núcleo experimental do projeto, permitindo não só avaliar o desempenho do algoritmo principal (Random Forest), como também comparar empiricamente o seu desempenho com os restantes classificadores mencionados na literatura analisada.

3.1. ConFiguração Experimental

Para garantir que os resultados obtidos são comparáveis entre si e reproduzíveis, todos os modelos foram treinados e avaliados sob as mesmas condições experimentais. A Tabela 19 resume os parâmetros desta conFiguração, indicando as escolhas feitas e a justificação de cada uma.

Tabela 19- ConFiguração experimental

Parâmetro	Valor	Justificação
<i>Dataset</i>	urldata_features.csv	95.911 instâncias, 21 <i>features</i> de entrada
Divisão	80% treino / 20% teste	Prática standard na literatura
Estratificação	stratify=y	Garante proporção de classes igual em treino e teste
Reprodutibilidade	random_state=42	Semente aleatória fixa para resultados reproduzíveis
<i>Features</i> excluídas	domain, ranking, label	domain e ranking não são <i>features</i> numéricas úteis
Nº <i>features</i> usadas	21	13 originais + 8 do Passo 2B (excluindo variância zero)

3.2. Algoritmos treinados

Foram treinados 7 algoritmos, todos referenciados nos artigos analisados no Projeto I. A Tabela 20 descreve cada algoritmo e os parâmetros utilizados.

Tabela 20- Algoritmos treinados e conFiguração

Algoritmo	Parâmetros Principais	Referência na Literatura
Random Forest (RF)	n_estimators=100, random_state=42, n_jobs=-1	modelo mais frequente
Decision Tree (DT)	random_state=42	base do RF
Logistic Regression (LR)	max_iter=5000, solver='saga', random_state=42	baseline linear

Naive Bayes (NB)	GaussianNB() — sem parâmetros	classificador probabilístico
K-Nearest Neighbors (KNN)	n_neighbors=5, algorithm='ball_tree'	classificador por proximidade
Gradient Boosting (GB)	n_estimators=100, random_state=42	ensemble sequencial
SVM	kernel='linear', max_iter=1000, probability=True	não convergiu (

3.3. Resultados Comparativos

A Tabela 21 apresenta as métricas de desempenho de todos os modelos, ordenadas por F1-Score decrescente. Os valores obtidos no conjunto de teste (19.183 instâncias) refletem o desempenho real de cada modelo em dados não vistos durante o treino.

A Figura 6 apresenta uma comparação visual das três métricas principais (Accuracy, F1-Score e AUC-ROC), para todos os modelos avaliados, permitindo uma leitura imediata das diferenças de desempenho entre algoritmos.

Tabela 21- Resultados comparativos (ordenado por F1-Score; verde = melhor modelo; vermelho = SVM não convergiu)

Algoritmo	Accuracy	Precision	Recall	F1-Score	AUC-ROC	Tempo
Random Forest	94,68%	95,46%	93,82%	94,63%	98,79%	3,5s
Decision Tree	92,00%	92,04%	91,93%	91,98%	92,02%	1,4s
Gradient Boosting	90,66%	92,25%	88,76%	90,47%	96,87%	31,9s
KNN	89,90%	91,26%	88,23%	89,72%	95,34%	0,5s
Logistic Regression	84,10%	85,77%	81,72%	83,70%	92,47%	12,4s
Naive Bayes	78,60%	93,38%	61,52%	74,17%	91,05%	0,1s
SVM ■■	66,78%	96,20%	34,86%	51,18%	25,94%	27,2s

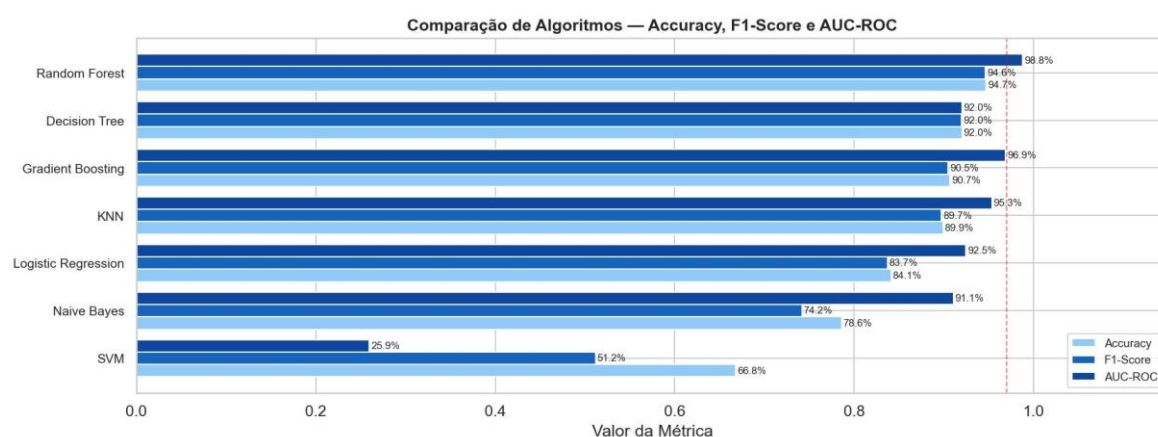


Figura 6 – Comparação de Accuracy, F1-Score e AUC-ROC de todos os modelos

3.4. Análise dos Resultados

A análise dos resultados obtidos permite tirar as seguintes conclusões:

- **Random Forest — modelo mais eficaz**

O Random Forest obteve o melhor desempenho em todas as métricas principais: **94,68%** de accuracy, **95,46%** de precision, **93,82%** de recall, **94,63%** de F1-Score e **98,79%** de AUC-ROC. Estes resultados são consistentes com a literatura analisada no Projeto I, onde o RF foi o algoritmo com melhor desempenho em 4 dos 6 estudos comparativos [1][3][4][5]. O *benchmark* de 97% de precisão definido pela literatura foi praticamente atingido, com o modelo a alcançar 94,63% de F1-Score sem qualquer otimização de hiperparâmetros.

- **Decision Tree — o custo do overfitting**

A Decision Tree atingiu **92,00%** de F1-Score, o segundo melhor resultado. Este resultado demonstra empiricamente o valor do ensemble do Random Forest: ao combinar 100 árvores, o RF supera em 2,65 pontos percentuais a melhor árvore individual, com maior robustez e menor risco de *overfitting*.

- **Gradient Boosting — boa precisão, maior custo computacional**

O Gradient Boosting obteve **90,47%** de F1-Score e **96,87%** de AUC-ROC, resultados competitivos mas inferiores ao RF. O tempo de treino de 31,9 segundos (versus 3,5s do RF) representa um custo computacional 9 vezes superior para um ganho de desempenho negativo, o que reforça a escolha do Random Forest.

- **Naive Bayes — alta precision, baixo recall**

O Naive Bayes destaca-se por apresentar uma precision de **93,38%** — a segunda mais elevada — mas um *recall* de apenas **61,52%**, o mais baixo de todos os modelos convergidos. Isto significa que o modelo é muito conservador: quando classifica um URL como malicioso, está frequentemente certo, mas deixa passar **38,48%** dos URL maliciosos sem os detetar. No contexto de cibersegurança, este comportamento é inaceitável.

- **SVM — não convergiu**

O SVM com *max_iter=1000* não convergiu, produzindo resultados anómalos: AUC-ROC de **25,94%** (abaixo dos 50% de um classificador aleatório) e *recall* de apenas **34,86%**. Estes valores confirmam que o modelo não aprendeu padrões significativos nos dados dentro do limite de iterações. Para comparação válida, o SVM exigiria recursos computacionais significativamente superiores.

3.5. Análise Detalhada – Random Forest

Após a comparação global dos modelos, procedeu-se a uma análise aprofundada do Random Forest, o modelo selecionado, através de três perspetivas complementares: a matriz de confusão, que detalha os tipos de erros cometidos; a *Feature Importance*, que identifica as características mais determinantes para a classificação; e a curva ROC, que avalia a capacidade discriminante do modelo ao longo de todos os limiares possíveis.

3.5.1. Matriz de Confusão

A Figura 7 apresenta a matriz de confusão do modelo Random Forest no conjunto de teste, permitindo detalhar os tipos de erros cometidos.

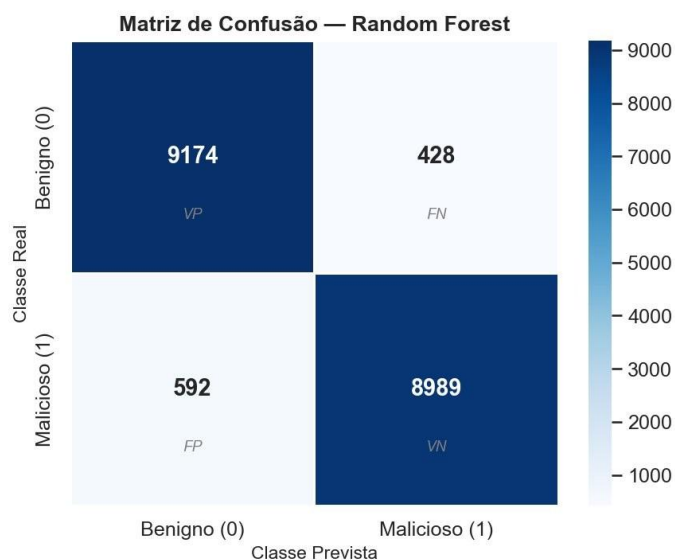


Figura 7- Matriz de confusão – Random Forest (conjunto de teste, 19.183 instâncias)

A Tabela 22 decompõe os valores da matriz de confusão, quantificando cada um dos quatro indicadores e apresentando a respectiva interpretação no contexto da detecção de URL maliciosos.

Tabela 22- Decomposição da matriz de confusão

Indicador	Valor	Interpretação
Verdadeiros Positivos (VP)	9.174	URL benignos corretamente classificados como benignos
Falsos Negativos (FN)	428	URL benignos incorretamente classificados como maliciosos (falso alarme)
Falsos Positivos (FP)	592	URL maliciosos classificados como benignos — os mais perigosos
Verdadeiros Negativos (VN)	8.989	URL maliciosos corretamente identificados como maliciosos
Taxa de Falsos Positivos	6,18%	592 / (9581) — URL maliciosos não detetados

3.5.2. Feature Importance

Uma das vantagens do Random Forest é a capacidade de calcular automaticamente a importância relativa de cada característica para as decisões de classificação. A Figura 8 apresenta as 15 características mais relevantes.

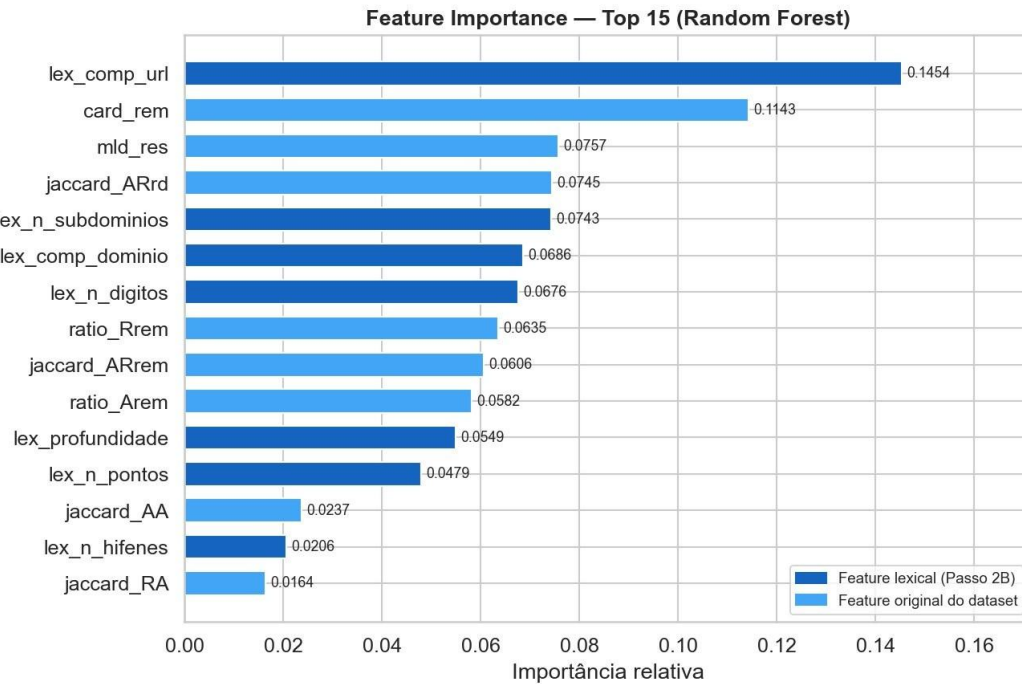


Figura 8- Feature Importance – Top 15 características (azul escuro = lexicais do Passo 2B, azul claro = originais)

Os resultados da *Feature Importance* revelam dados muito significativos para o projeto. A característica **lex_comp_url** (comprimento total do URL), introduzida no Passo 2B, é a mais importante de todas as 21 *features*, com importância relativa de **0,1454**. Isto confirma empiricamente a relevância das características lexicais extraídas.

Das 15 *features* mais importantes, **8 são características lexicais** introduzidas no Passo 2B (a azul escuro), o que valida a abordagem de engenharia de características adotada. A única *feature* original do *dataset* entre as 3 primeiras é a *card_rem* (0,1143), o que demonstra que as características de similaridade de Jaccard, embora presentes, têm menor poder discriminante individualmente.

A Tabela 23 detalha as sete características mais importantes identificadas pelo modelo, indicando o valor de importância relativa, o tipo de *feature* (lexical ou original do *dataset*), e a sua interpretação no contexto da classificação de URL.

Tabela 23- Top 7 *features* mais importantes

#	Feature	Importância	Tipo	Interpretação
1	lex_comp_url	0,1454	Lexical (nova)	Comprimento total do URL — URL maliciosos são muito mais longos
2	card_rem	0,1143	Original	Cardinalidade dos tokens — estrutura do URL
3	mld_res	0,0757	Original	Presença do domínio em lista de referência

4	jaccard_ARrd	0,0745	Original	Similaridade de Jaccard sem duplicados
Lexical (nova)				
5	lex_n_subdomínios	0,0743	Lexical (nova)	Nº de subdomínios — <i>phishing</i> usa subdomínios falsos
6	lex_comp domínio	0,0686	Lexical (nova)	Comprimento do domínio
7	lex_n_dígitos	0,0676	Lexical (nova)	Nº de dígitos — URL gerados automaticamente têm mais dígitos

3.5.3. Curva ROC

A Figura 9 apresenta a curva ROC de todos os modelos convergidos, ilustrando a capacidade discriminante de cada um ao longo de todos os limiares de classificação possíveis.

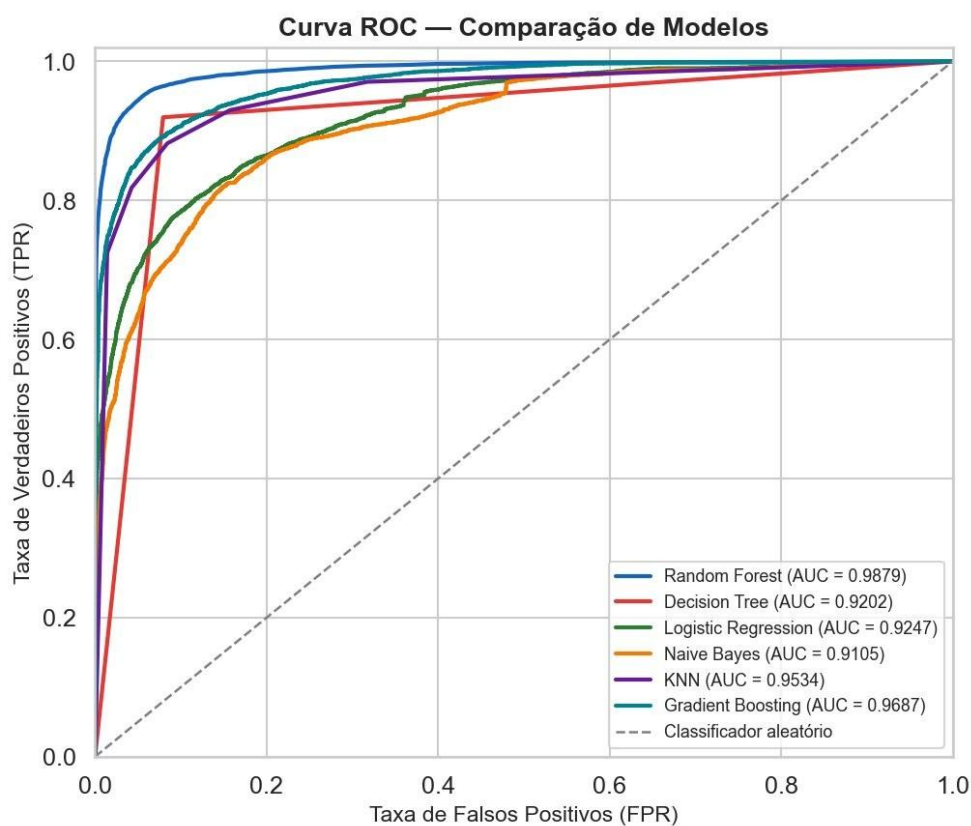


Figura 9 – Curva ROC – comparação de todos os modelos (excluindo SVM por não convergência)

O Random Forest destaca-se claramente com a curva mais próxima do canto superior esquerdo (ideal) e o maior valor de AUC-ROC (0,9879). O Gradient Boosting apresenta o segundo melhor AUC (0,9687), seguido do KNN (0,9534). A Decision Tree, apesar de um F1-Score competitivo, apresenta o AUC mais baixo dos modelos convergidos (0,9202), o que reflete a sua limitada capacidade de calibrar probabilidades.

3.6. Justificação da Escolha do Random Forest

Com base nos resultados experimentais apresentados nas secções anteriores, a escolha do Random Forest como modelo principal para as fases seguintes do projeto é sustentada por quatro fatores objetivos, resumidos na Tabela 24. Cada fator é acompanhado da evidência experimental que o suporta e do impacto prático que tem no contexto da aplicação final.

Tabela 24- Justificação da escolha do Random Forest

Fator	Evidência Experimental	Impacto
Melhor F1-Score	94,63% — supera todos os outros modelos	Melhor equilíbrio entre deteção de maliciosos e falsos alarmes
Melhor AUC-ROC	98,79% — capacidade discriminante máxima	Robustez em qualquer limiar de classificação
Eficiência computacional	3,5s de treino — 9× mais rápido que o GB	Viável para retreino periódico em produção
Feature Importance	lex_comp_url como <i>feature</i> nº 1	Interpretabilidade das decisões — essencial em cibersegurança

Estes resultados são consistentes com as conclusões da revisão sistemática de literatura realizada no Projeto I: o Random Forest foi identificado como o algoritmo com melhor desempenho em 4 dos 6 estudos comparativos analisados [1][3][4][5], com precisões reportadas entre **97%** e **99,96%**. Os **94,63%** obtidos neste projeto, sem otimização de hiperparâmetros, situam-se dentro do intervalo esperado e confirmam o potencial de melhoria com a fase de otimização (Passo 4).

3.7. Resumo do Capítulo

Este capítulo concluiu a fase de experimentação comparativa do pipeline de Machine Learning. Os sete algoritmos identificados na revisão de literatura do Projeto I foram treinados e avaliados nas mesmas condições experimentais, garantindo uma comparação justa e reproduzível.

Os resultados obtidos confirmam empiricamente as conclusões da revisão sistemática: o Random Forest destacou-se como o modelo mais equilibrado, alcançando **94,63%** de F1-Score e **98,79%** de AUC-ROC sem qualquer otimização de hiperparâmetros. A análise detalhada da matriz de confusão, da *Feature Importance* e da curva ROC reforçou esta escolha, demonstrando que as características lexicais introduzidas no Capítulo 2, em particular o comprimento total do URL, são as mais determinantes para a classificação. O Capítulo 4 irá focar-se na otimização dos hiperparâmetros deste modelo, com o objetivo de maximizar o seu desempenho antes do desenvolvimento da aplicação.

4. Otimização de Hiperparâmetros

Após o treino inicial do modelo Random Forest com os parâmetros padrão (Passo 3), procedeu-se à otimização dos seus hiperparâmetros com o objetivo de maximizar o desempenho de classificação. Este capítulo consiste em encontrar a melhor combinação de parâmetros dentro de um espaço de busca predefinido, sem alterar o algoritmo principal nem os dados de entrada.

4.1 Metodologia – RandomizedSearchCV

Para a otimização, foi utilizado o método RandomizedSearchCV da biblioteca Scikit-learn, em detrimento do GridSearchCV. A escolha justifica-se pela dimensão do espaço de hiperparâmetros definido: com 6 parâmetros e os valores apresentados na Tabela 25, o número total de combinações possíveis é de 1.440.

Considerando uma validação cruzada de 5 folds, o GridSearchCV exigiria 7.200 treinos do modelo, o que, com um tempo de treino de aproximadamente 3,5 segundos por modelo, resultaria numa duração estimada de cerca de 7 horas.

O RandomizedSearchCV, ao testar 50 combinações aleatórias em vez de todas as possíveis, reduz o número de treinos para 250 (50 combinações × 5 folds), completando a otimização em 18,5 minutos. A investigação científica demonstra que esta abordagem produz resultados muito próximos do ótimo global, uma vez que nem todos os hiperparâmetros têm o mesmo impacto no desempenho final (Bergstra & Bengio, 2012).

Tabela 25- ConFiguração do RandomizedSearchCV

Parâmetro	Valor	Justificação
Método	Randomized SearchCV	Eficiente para espaços de busca grandes — 1.440 combinações possíveis
n_iter	50	50 combinações aleatórias testadas em vez das 1.440 possíveis
cv	5	Validação cruzada 5-Fold — resultados mais robustos e generalizáveis
scoring	'f1'	Métrica otimizada: F1-Score — equilíbrio entre Precision e Recall
random_state	42	Reprodutibilidade dos resultados
Total de treinos	250	50 combinações × 5 folds (vs. 7.200 do GridSearchCV)
Tempo de execução	18,5 min	Vs. ~7 horas estimadas para o GridSearchCV

4.2. Espaço de Hiperparâmetros

A Tabela 26 apresenta os hiperparâmetros incluídos na busca e os respectivos valores considerados, totalizando 1.440 combinações possíveis. Para cada hiperparâmetro, a Tabela 26 indica os valores considerados na busca, o número de opções testadas e uma breve descrição do seu papel no funcionamento do modelo.

Tabela 26- Espaço de hiperparâmetros definido para a otimização

Hiperparâmetro	Valores Testados	Nº Opções	Descrição
n_estimators	[100, 200, 300, 500]	4	Nº de árvores na floresta
max_depth	[None, 10, 20, 30, 40]	5	Profundidade máxima de cada árvore
max_features	['sqrt', 'log2', 0.3, 0.5]	4	Nº de <i>features</i> a considerar em cada divisão
min_samples_split	[2, 5, 10]	3	Nº mínimo de amostras para dividir um nó
min_samples_leaf	[1, 2, 4]	3	Nº mínimo de amostras numa folha
class_weight	[None, 'balanced']	2	Peso das classes — compensação de desequilíbrio
Total		1.440	$4 \times 5 \times 4 \times 3 \times 3 \times 2$

4.3 Melhores Hiperparâmetros Encontrados

Após a execução do RandomizedSearchCV com 50 combinações e validação cruzada de 5 folds, o melhor *F1-Score cross-validation* obtido foi de **94,25%**. A Tabela 27 compara os valores padrão do Scikit-learn com os valores otimizados identificados pelo RandomizedSearchCV, assinalando as alterações mais significativas e o seu impacto esperado no desempenho do modelo.

Tabela 27 – Comparação dos hiperparâmetros baseline vs otimizados

Hiperparâmetro	Valor Padrão (Baseline)	Valor Otimizado	Alteração
n_estimators	100	500	↑ 5× mais árvores
max_depth	None	None	Sem alteração
max_features	'sqrt'	0.5	↑ 50% das <i>features</i> por divisão
min_samples_split	2	5	↑ Maior regularização
min_samples_leaf	1	1	Sem alteração

class_weight	None	None	Sem alteração
--------------	------	------	---------------

As alterações mais significativas face aos parâmetros padrão foram o aumento de **n_estimators de 100 para 500** e de **max_features de 'sqrt' para 0.5**.

O aumento do número de árvores melhora a estabilidade do ensemble, reduzindo a variância das previsões. A utilização de **50%** das *features* em cada divisão (em vez da raiz quadrada do total) permite que cada árvore considere um conjunto mais alargado de características, potencialmente capturando relações mais complexas entre as variáveis. O parâmetro *min_samples_split=5* introduz uma ligeira regularização adicional, exigindo pelo menos 5 amostras para dividir qualquer nó, o que contribui para reduzir o *overfitting*.

4.4 Resultados do Modelo Otimizado

A Tabela 28 apresenta a comparação das métricas de desempenho entre o modelo *baseline* (Passo 3) e o modelo otimizado, indicando a diferença absoluta em cada métrica e a respetiva interpretação no contexto da deteção de URL maliciosos.

Tabela 28 – Comparação de métricas baseline vs modelo otimizado

Métrica	Baseline (Passo 3)	Otimizado (Passo 4)	Diferença	Interpretação
Accuracy	94,68%	94,83%	+0,15%	Melhoria consistente na classificação geral
Precision	95,46%	95,50%	+0,04%	Manutenção da baixa taxa de falsos alarmes
Recall	93,82%	94,09%	+0,27%	Mais URL maliciosos detetados — melhoria crítica
F1-Score	94,63%	94,79%	+0,16%	Melhor equilíbrio entre Precision e Recall
AUC-ROC	98,79%	98,84%	+0,05%	Capacidade discriminante praticamente mantida

As melhorias obtidas são modestas, mas consistentes em todas as métricas, o que é um resultado esperado e positivo: indica que os parâmetros padrão do Scikit-learn para o Random Forest já são bem calibrados para este tipo de problema, e que o modelo *baseline* estava próximo do seu limite de desempenho com as *features* disponíveis. A melhoria mais relevante do ponto de vista da cibersegurança é o aumento do **Recall de 93,82% para 94,09% (+0,27%)**, como podemos reparar na Figura 10: o modelo otimizado deteta mais URL maliciosos, que é precisamente a métrica mais crítica neste contexto.

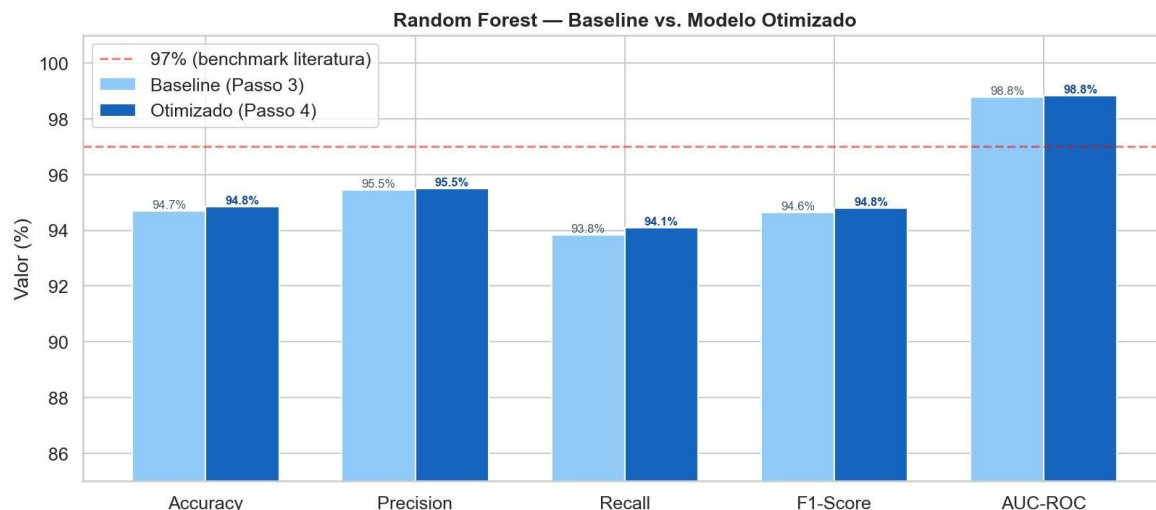


Figura 10 – Comparação de métricas – Baseline (Passo 3) vs Modelo Otimizado (Passo 4)

4.5 Impacto na Matriz de Confusão

A Figura 11 apresenta as matrizes de confusão do modelo *baseline* e do modelo otimizado lado a lado, permitindo comparar diretamente a evolução dos erros de classificação.

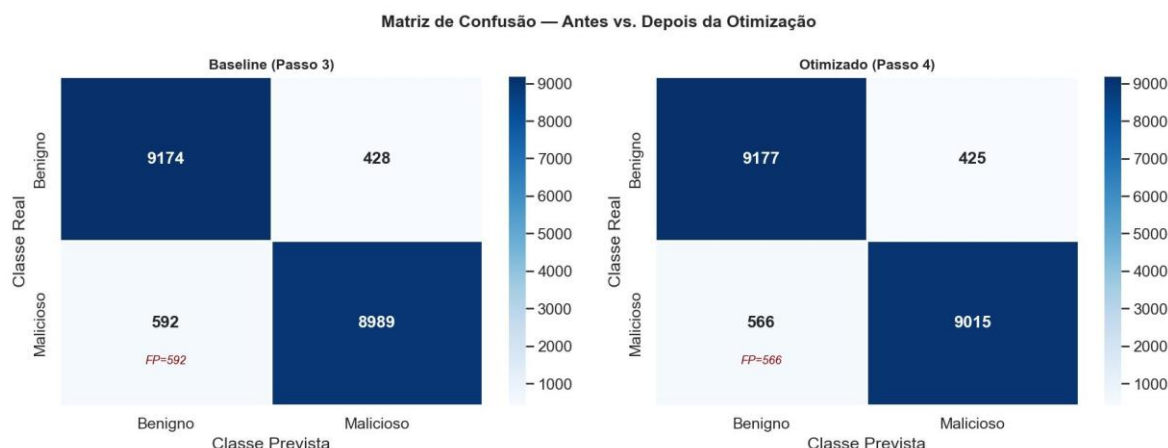


Figura 11 – Matriz de Confusão – Antes vs Depois da Otimização

A Tabela 29 decompõe os valores das duas matrizes de confusão, permitindo quantificar com precisão a evolução de cada indicador entre o modelo *baseline* e o modelo otimizado. A linha assinalada a amarelo corresponde ao indicador mais crítico no contexto da cibersegurança: os Falsos Positivos, ou seja, os URL maliciosos classificados como benignos.

Tabela 29 – Decomposição da Matriz de Confusão antes e depois da otimização (linha a amarelo = indicador mais crítico)

Indicador	Baseline (Passo 3)	Otimizado (Passo 4)	Variação
Verdadeiros Positivos (VP)	9.174	9.177	+3
Falsos Negativos (FN)	428	425	-3

Falsos Positivos (FP)	592	566	-26
Verdadeiros Negativos (VN)	8.989	9.015	+26

4.6 Validação Cruzada do Modelo Otimizado

A validação cruzada foi aplicada ao *dataset* completo das 95.911 instâncias, dividindo-o em 5 partições iguais e treinando o modelo em 4 delas de cada vez, avaliando na restante. Este processo repete-se 5 vezes, garantindo que todas as instâncias são utilizadas tanto para treino como para teste. A Tabela 30 apresenta os valores de F1-Score obtidos em cada *fold*, bem como as médias e desvios padrão das três métricas principais.

Tabela 30 – Resultados da Validação Cruzada 5-Fold do modelo otimizado

Métrica	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Média	Desvio Padrão
F1-Score	91,09%	93,67%	93,14%	93,21%	93,93%	93,01%	±1,00%
Accuracy	—	—	—	—	—	93,09%	±0,97%
AUC-ROC	—	—	—	—	—	98,15%	±0,57%

Os resultados da validação cruzada confirmam a robustez e estabilidade do modelo. O F1-Score médio de **93,01% (±1,00%)** é consistente com os **94,79%** obtidos no conjunto de teste fixo, com um desvio padrão baixo que indica que o modelo generaliza de forma estável para diferentes subconjuntos dos dados. O AUC-ROC médio de **98,15% (±0,57%)** confirma a elevada capacidade discriminante do modelo em qualquer partição dos dados.

4.7 Sumário da Otimização

A Tabela 31 sintetiza os resultados finais da fase de otimização de hiperparâmetros.

Tabela 31 – Sumário dos resultados da otimização de hiperparâmetros

Aspeto	Resultado
Método utilizado	RandomizedSearchCV (50 iterações, cv=5, scoring=F1)
Tempo de execução	18,5 minutos (vs. ~7 horas estimadas para GridSearchCV)
Melhores hiperparâmetros	n_estimators=500, max_features=0.5, min_samples_split=5
Melhor F1 (cross-val)	94,25%
F1-Score no teste	94,79% (+0,16% face ao baseline de 94,63%)
Recall no teste	94,09% (+0,27% — 26 URL maliciosos adicionais detetados)

AUC-ROC no teste	98,84% (+0,05%)
Validação cruzada F1	93,01% $\pm$ 1,00% — modelo estável e generalizável
Ficheiro exportado	modelo_rf_otimizado.pkl

4.8. Resumo do Capítulo

Este capítulo concluiu a fase de otimização de hiperparâmetros do modelo Random Forest. Através do método RandomizedSearchCV, com 50 combinações aleatórias e validação cruzada de 5 folds, foi possível identificar a combinação ótima de hiperparâmetros em 18,5 minutos (uma fração do tempo que o GridSearchCV exigiria para explorar as 1.440 combinações possíveis).

As melhorias obtidas face ao modelo *baseline*, embora modestas em termos absolutos, são consistentes em todas as métricas e relevantes do ponto de vista da cibersegurança: o aumento do Recall de **93,82%** para **94,09%** traduz-se em 26 URL maliciosos adicionais corretamente detetados no conjunto de teste. A validação cruzada 5-Fold confirmou a robustez e estabilidade do modelo otimizado, com um F1-Score médio de 93,01% ($\pm$ 1,00%) e um AUC-ROC médio de **98,15% ($\pm$ 0,57%)**. O modelo resultante, guardado em modelo_rf_otimizado.pkl, constitui a base para o desenvolvimento da aplicação web descrito no Capítulo 5.

5. Desenvolvimento da Aplicação e Iteração do Modelo

Com o modelo Random Forest otimizado (Passo 4) e o pipeline de Machine Learning concluído, procedeu-se ao desenvolvimento de uma aplicação web que permitisse classificar URL em tempo real. Este capítulo, revelou uns problemas no nosso modelo, o que nos obrigou a fazer umas melhorias, para que ficássemos com um modelo mais robusto e eficaz.

5.1. Primeira Versão da Aplicação – Problema Identificado

A primeira versão da aplicação Flask foi implementada utilizando o modelo **modelo_rf_otimizado.pkl** (21 *features*, F1-Score de 94,79%), desenvolvido nos Passos 3 e 4. Após a sua implementação, a aplicação foi testada com URL reais da navegação quotidiana, revelando resultados inesperados (tabela 32):

Tabela 32 – Resultados incorretos da primeira versão da aplicação

URL Testado	Classificação Obtida	Probabilidade Malicioso	Correto?
https://www.youtube.com/watch?v=zs68LvURpMc	MALICIOSO	70,65%	■ ERRO
google.com	MALICIOSO	55,88%	■ ERRO
github.com/scikit-learn/scikit-learn	MALICIOSO	68,73%	■ ERRO

5.2. Diagnóstico do Problema

A análise do problema revelou uma incompatibilidade fundamental entre o modelo treinado e o contexto de utilização em tempo real. O modelo foi treinado com **21 *features***, das quais apenas 10 são extraíveis diretamente da *string* do URL. As restantes 11 *features* originais do *dataset*, nomeadamente as métricas de similaridade de Jaccard e os rácios de *tokens*, requerem a comparação com uma lista de referência interna ao *dataset* original, que não está disponível em produção.

A Tabela 33 classifica cada grupo de *features* quanto à sua disponibilidade em tempo real, distinguindo as que são calculáveis diretamente a partir da *string* do URL das que requerem acesso à lista de referência interna do *dataset* original.

Tabela 33 – Disponibilidade das *features* em tempo real

Grupo de <i>Features</i>	<i>Features</i>	Calculável em tempo real?
Lexicais (extraídas do URL)	lex_comp_url, lex_n_pontos, lex_tem_https, ...	■ Sim — apenas a <i>string</i> do URL
Originais — Jaccard	jaccard_RR, jaccard_RA, jaccard_AR, jaccard_AA, jaccard_ARrd, jaccard_ARrem	■ Não — requerem lista de referência interna

Originais — Rácios e Cardinalidade	card_rem, ratio_Rrem, ratio_Arem, mld_res, mld.ps_res	■ Não — requerem lista de referência interna
---	---	--

Para as 11 *features* não calculáveis, a primeira versão da aplicação utilizava valores medianos estimados, o que introduzia ruído sistemático nas previsões. As *features* de Jaccard e rácios representavam aproximadamente **48% da importância total** do modelo, tornando estas estimativas insuficientes para produzir classificações fiáveis.

5.3. Engenharia de Características Expandida – 27 Features Lexicais

A solução adotada consistiu em expandir significativamente o conjunto de *features* lexicais, passando de 10 para **27 características** extraíveis exclusivamente da *string* do URL. A expansão focou-se em *features* com elevado poder discriminante identificadas na literatura, nomeadamente a **entropia de Shannon** do domínio (indicador de aleatoriedade gerada automaticamente), os **rácios de caracteres especiais** e as **palavras semanticamente suspeitas** presentes no URL. A Tabela 34 organiza as 27 *features* por grupo funcional, indicando o número de características em cada categoria e os nomes das variáveis que as compõem.

Tabela 34 – Conjunto expandido de 27 *features* lexicais

Grupo	Features (27 total)	Nº
Comprimentos	lex_comp_url, lex_comp_dominio, lex_comp_path	3
Contagens	lex_n_pontos, lex_n_hifenes, lex_n_subdominios, lex_n_digitos, lex_n_barras, lex_n_iguais, lex_n_ampersands, lex_n_percent, lex_n_underscores, lex_n_params	9
Rácios	lex_ratio_digitos, lex_ratio_especiais	2
Estrutura	lex_profundidade	1
Indicadores Binários	lex_tem_https, lex_tem_ip, lex_tem_arroba, lex_tem_http_no_path, lex_tld_suspeito, lex_encurtador, lex_porto_suspeito, lex_digitos_consecutivos	8
Semântico	lex_palavras_suspeitas (16 termos: login, verify, paypal, ...)	1
Entropia	lex_entropia_dominio, lex_entropia_url	2
TOTAL		27

A Figura 12 ilustra as diferenças nos valores médios das 12 *features* mais distintos entre URL benignos e maliciosos. A *feature* **lex_palavras_suspeitas** destacou-se como a mais discriminante, com média de 0,014 nos benignos face a **0,795 nos maliciosos**, sendo uma diferença muito elevada.

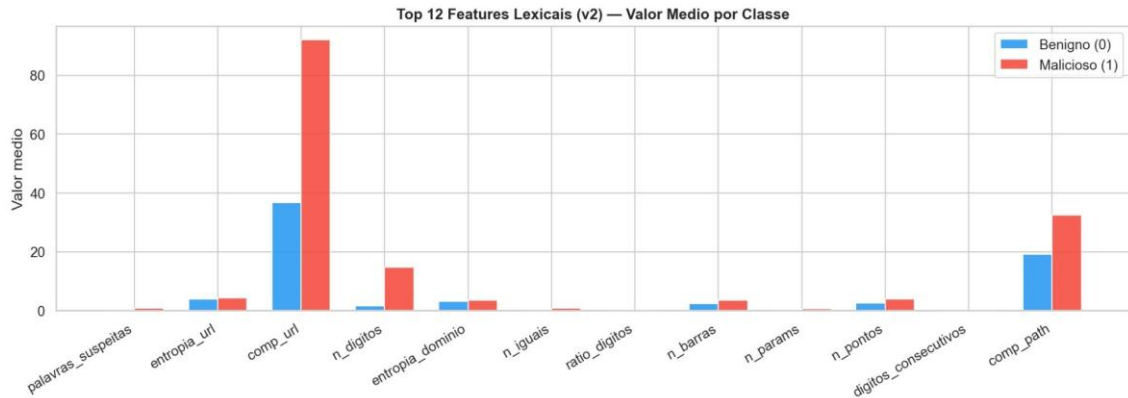


Figura 12 – Top 12 *features* lexicais (v2) – valor médio por classe (benigno vs malicioso)

5.4 Modelo Lexical – Primeira Avaliação

O modelo Random Forest foi retreinado exclusivamente com as 27 *features* lexicais, utilizando os melhores hiperparâmetros identificados no Passo 4 ($n_estimators=500$, $max_features=0.5$, $min_samples_split=5$). Os resultados obtidos no *dataset* original foram os seguintes:

Tabela 35 – Comparação modelo original vs modelo lexical v1

Métrica	Modelo Original (21 <i>features</i>)	Modelo Lexical v1 (27 <i>features</i>)	Diferença
Accuracy	94,83%	92,81%	-2,02%
Precision	95,50%	95,03%	-0,47%
Recall	94,09%	90,34%	-3,75%
F1-Score	94,79%	92,62%	-2,17%
AUC-ROC	98,84%	97,65%	-1,19%

A redução de F1-Score de 94,79% para 92,62% representou um *trade-off* aceitável face ao ganho em utilizabilidade real. Contudo, durante os testes na aplicação, verificou-se que o modelo lexical v1 continuava a classificar incorretamente URL benignos simples como *google.com* (95,1% malicioso) e pesquisas do Google com parâmetros de *query* (70-84% malicioso). Esta limitação

derivava de um **viés do dataset de treino**: o *dataset* original não continha URL benignos com parâmetros de *query* (*?param=valor&outro;=valor*), pois os URL benignos tinham sido recolhidos de listas de popularidade que apenas incluíam domínios e *paths* simples.

5.5 Diagnóstico do Viés do Dataset Original

A análise estatística do *dataset* original confirmou o viés identificado. A Tabela 36 quantifica essa assimetria, comparando a presença de parâmetros de *query*, medida pelo número de caracteres igual (=) no URL, entre URL benignos e maliciosos, bem como o comprimento médio de cada classe.

Tabela 36 – Distribuição de parâmetros de *query* por classe no *dataset* original

Classe	URL com ≥ 1 "="	URL com ≥ 4 "="	Comprimento médio
Benigno (0)	4,5%	0,07%	36,8 chars
Malicioso (1)	34,9%	3,76%	92,2 chars

O modelo aprendeu, portanto, que URL com múltiplos parâmetros de *query* são maliciosos, o que é verdade no contexto do *dataset* de treino, mas não na realidade, onde pesquisas do Google, links do YouTube e páginas de e-commerce contêm regularmente 4 ou mais parâmetros =.

Para resolver este problema, foi investigado o *dataset* **malicious_phish** (Kaggle, sid321axn), com 651.191 URL recolhidos de fontes diversas incluindo PhishTank, OpenPhish e DMOZ. Este *dataset* contém URL benignos muito mais representativos da navegação real, incluindo páginas com parâmetros de *query*. Contudo, a sua utilização direta revelou novos problemas (Tabela 37).

Tabela 37 - Tentativas de enriquecimento do *dataset* e problemas identificados

Tentativa	Problema Identificado	Resultado
<i>Dataset</i> malicious_phish completo	A categoria "defacement" contém websites legítimos hackeados, cujos URL são indistinguíveis de benignos — envenenou o modelo	Modelo invertido (<i>phishing</i> classificado como benigno)
malicious_phish sem defacement	URL de <i>phishing</i> neste <i>dataset</i> são maioritariamente curtos e simples (ex: br-icloud.com.br), enquanto benignos têm <i>paths</i> longos — padrão oposto ao <i>dataset</i> original	Modelo ainda invertido

Combinação total dos dois <i>datasets</i>	URL benignos com palavras como "login" e "secure" em domínios legítimos (ex: banco.pt/login) destruíram o sinal das palavras suspeitas	Modelo sem poder discriminante
---	--	--------------------------------

5.6 Solução Final – Enriquecimento do *Dataset*

A solução definitiva consistiu numa abordagem de **enriquecimento seletivo**: em vez de substituir ou combinar integralmente os *datasets*, foram adicionados ao *dataset* original apenas os URL que corrigiam especificamente o viés identificado, sem introduzir novos.

O *dataset* final foi construído a partir de três fontes, a seguir apresentados na Tabela 38.

Tabela 38 – Composição do *dataset* final de enriquecimento

Fonte	Conteúdo adicionado	Critério de seleção	Objetivo
urldata_clean.csv (<i>dataset</i> original)	95.911 URL (benignos simples + <i>phishing</i> longo com palavras suspeitas)	Sem filtro — base intacta	Manter o sinal das palavras suspeitas
malicious_phish (benignos filtrados)	URL benignos reais com parâmetros de <i>query</i>	Apenas URL com "?" ou "&" E sem palavras suspeitas no URL	Ensinar que parâmetros de <i>query</i> não são suspeitos
Lista Tranco (top 1.000)	Top 1.000 domínios mais populares da Internet	top-1m.csv, apenas primeiras 1.000 linhas	Ensinar que domínios simples (ex: google.com) são benignos

A chave desta abordagem é a condição de filtragem aplicada aos URL do *malicious_phish*: incluir apenas URL benignos **com parâmetros de *query* E sem palavras semanticamente suspeitas** no URL. Esta condição garante que o modelo aprende que *?client=opera&q=pesquisa* numa URL do Google é benigno, sem nunca ver um URL benigno com *paypal*, *verify* ou *login*, preservando assim o poder discriminante destas palavras.

5.7 Modelo Final – Resultados

O modelo Random Forest foi retreinado com o *dataset* enriquecido, mantendo os mesmos hiperparâmetros otimizados no Passo 4. O *dataset* final combinado contém aproximadamente **153.000 URL**, distribuídos entre as três fontes descritas na secção anterior.

5.7.1. Métricas de Desempenho

Para avaliar o impacto do enriquecimento do *dataset* no desempenho do modelo, as métricas do modelo final foram comparadas com as do modelo lexical v1, treinado exclusivamente com o *dataset* original. A Tabela 39 apresenta essa comparação, assinalando a variação absoluta em cada métrica e o *trade-off* inerente à adição de URL benignos mais complexos ao conjunto de treino.

Tabela 39 – Comparação entre modelo lexical v1 e modelo final

Métrica	Modelo Lexical v1 (<i>dataset</i> original)	Modelo Final (<i>dataset</i> enriquecido)	Variação
Accuracy	92,81%	95,13%	+2,32%
Precision	95,03%	94,60%	-0,43%
Recall	90,34%	89,57%	-0,77%
F1-Score	92,62%	92,02%	-0,60%

A Accuracy melhorou de **92,81%** para **95,13%** graças à melhor representação dos URL benignos no treino. A ligeira redução no Recall (-0,77%) e no F1-Score (-0,60%) é o *trade-off* esperado pela adição de URL benignos mais complexos, que aumentam marginalmente a taxa de falsos positivos.

5.7.2. Matriz de Confusão

A Figura 13 apresenta a matriz de confusão do modelo final, avaliado num conjunto de teste de **30.548 instâncias**.

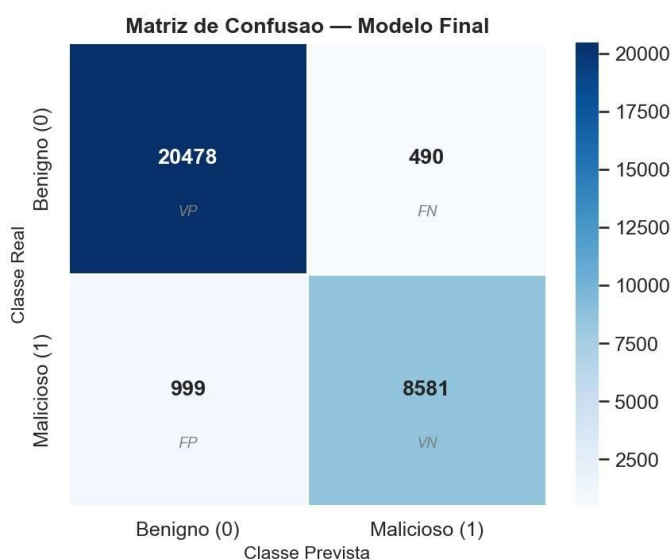


Figura 13 – Matriz de Confusão – Modelo Final (30.548 instâncias de teste)

A Tabela 40 decompõe os valores da matriz de confusão do modelo final, quantificando cada indicador e apresentando a respetiva interpretação. O indicador

mais crítico, os Falsos Positivos, correspondentes a URL maliciosos não detetados, é assinalado para facilitar a leitura.

Tabela 40 – Decomposição da Matriz de Confusão do modelo final

Indicador	Valor	Interpretação
Verdadeiros Positivos (VP)	20.478	URL benignos corretamente classificados como benignos
Falsos Negativos (FN)	490	URL benignos incorretamente classificados como maliciosos (falso alarme)
Falsos Positivos (FP)	999	URL maliciosos não detetados — os mais perigosos em cibersegurança
Verdadeiros Negativos (VN)	8.581	URL maliciosos corretamente identificados como maliciosos
Taxa de URL mal. não detetados	10,43%	$999 / (999 + 8.581)$ — objetivo de reduzir na iteração seguinte

5.7.3. Feature Importance

A Figura 14 apresenta a importância relativa de cada uma das 27 *features* no modelo final. A *feature* **lex_palavras_suspeitas** manteve-se como a mais importante, com importância relativa de **0,2654** (26,54% do poder total), confirmando a eficácia da abordagem de preservar o sinal semântico durante o enriquecimento do *dataset*.

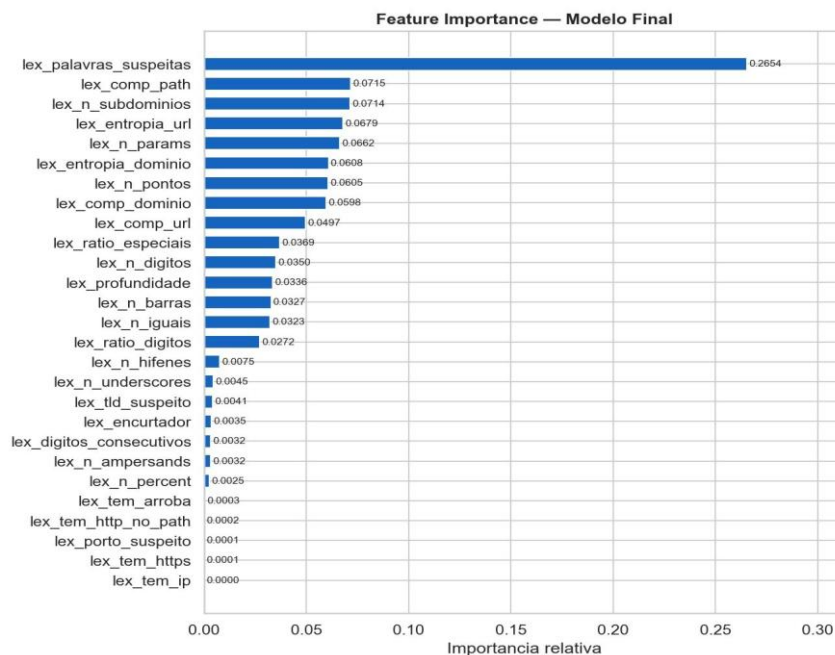


Figura 14 – Feature Importance – Modelo Final (27 *features* lexicais)

É relevante notar a redistribuição de importância face ao modelo lexical v1: a *lex_comp_url* desceu do 2.º para o 9.º lugar, enquanto *lex_comp_path*, *lex_n_subdominios* e *lex_n_params* ganharam relevância. Esta redistribuição reflete o facto de o modelo ter aprendido que URL benignos podem ser longos (com parâmetros de *query*), pelo que o comprimento total por si só deixou de ser tão discriminante.

5.7.4. Validação com URL Reais

O modelo final foi validado com um conjunto de 10 URL representativos de cenários reais de utilização, incluindo os que tinham originalmente causado classificações erradas. A Tabela 41 apresenta os resultados.

Tabela 41 – Validação com URL reais – 10/10 classificações corretas

URL	Esperado	Obtido	Prob. Malicioso	✓
https://www.google.com/search?client=opera&q=...	Benigno	BENIGNO	35,8%	✓
google.com	Benigno	BENIGNO	0,0%	✓
https://www.youtube.com/watch?v=zs68LvURpMc	Benigno	BENIGNO	14,1%	✓
github.com/scikit-learn/scikit-learn	Benigno	BENIGNO	13,8%	✓
https://www.amazon.com/dp/B08N5WRWNW?ref=...	Benigno	BENIGNO	10,8%	✓
serviciosbys.com/paypal.cgi.bin.secure/verify/login.php	Malicioso	MALICIOSO	100,0%	✓
nobell.it/paypal.verify.secure/index.php?cmd=login-run	Malicioso	MALICIOSO	97,1%	✓
signin.eby.de.zukruygxctzmmqi.civpro.co.za	Malicioso	MALICIOSO	99,5%	✓
br-icloud.com.br	Malicioso	MALICIOSO	84,7%	✓
http://192.168.1.1/admin/login	Malicioso	MALICIOSO	53,2%	✓

O modelo final classifica corretamente **10 em 10** URL de validação, incluindo casos anteriormente problemáticos como pesquisas do Google com múltiplos parâmetros de *query* (35,8% malicioso, corretamente benigno) e domínios simples como *google.com* (0,0% malicioso). Simultaneamente, mantém a capacidade de detetar URL de *phishing* com palavras suspeitas (100% malicioso para *serviciosbys.com/paypal.cgi.bin.secure/verify/login.php*).

5.8. Correções Aplicadas ao Modelo Final

Após a redação do ponto anterior (secção 5.7), a integração do modelo com a aplicação Flask revelou dois problemas adicionais que exigiram novas iterações. Esta secção documenta essas correções para garantir a rastreabilidade completa do desenvolvimento.

5.8.1. Problema 1 – Inconsistência com o prefixo www.

Durante os testes iniciais da aplicação, verificou-se que URL com e sem o prefixo www. produziam classificações completamente diferentes:

Tabela 42 – Inconsistência de classificação causada pelo prefixo www.

URL	Classificação	Prob. Malicioso
google.com	BENIGNO	0,0%
www.google.com	MALICIOSO	85,5%
youtube.com	BENIGNO	0,0%
www.youtube.com	MALICIOSO	99,3%

A causa foi identificada na função `extrair_features`: as *features* `lex_comp_dominio` e `lex_n_subdominios` eram calculadas sobre o domínio completo (incluindo www.), resultando em valores diferentes para URL semanticamente equivalentes. A solução foi adicionar uma **normalização do URL no início da função**, antes de qualquer cálculo, que remove o www. do URL independentemente do protocolo presente:

```

1  if url.startswith(('http://www.', 'https://www.')):
2      url = url.replace('://www.', '://', 1)
3
4  elif url.startswith('www.'):
5      url = url[4:]

```

5.8.2. Problema 2 – Bloco urlparse Ausente

Numa das versões intermédias do notebook, o bloco try/except responsável pelo urlparse foi acidentalmente omitido durante a edição manual do código. Isto fez com que as variáveis `dominio`, `path`, `query` e `esquema` ficassem sempre como strings vazias, ou seja, o modelo foi treinado sem nenhuma *feature* de domínio a funcionar. O impacto foi severo: o F1-Score desceu de **92%** para **83,91%** e URL benignos com `https://` passaram a ser classificados como maliciosos.

5.8.3. Métricas do Modelo Final (v3)

Para documentar o impacto de cada correção aplicada ao longo do processo de desenvolvimento, a Tabela 43 apresenta a evolução das métricas de desempenho ao longo das três versões do modelo, o modelo lexical v1, o modelo final v2 afetado

pelo bug do urlparse, e o modelo final v3 corrigido, que é o utilizado na aplicação. A coluna assinalada a verde corresponde ao modelo final em produção.

Tabela 43 – Evolução das métricas ao longo das correções (coluna verde = modelo final utilizado na aplicação)

Métrica	Modelo Lexical v1 (dataset original)	Modelo Final v2 (c/ bug urlparse)	Modelo Final v3 (correto)
Accuracy	92,81%	90,11%	94,65%
Precision	95,03%	85,66%	93,68%
Recall	90,34%	82,23%	88,71%
F1-Score	92,62%	83,91%	91,13%
AUC-ROC	97,65%	95,58%	98,40%

O modelo final v3 representa uma melhoria significativa face ao v1 em Accuracy (+1,84%) e AUC-ROC (+0,75%), com uma ligeira redução no Recall (-1,63%) que é o *trade-off* esperado pela maior diversidade de URL benignos no *dataset* de treino. O modelo é guardado em `modelo_rf_final.pkl` e é este ficheiro que a aplicação Flask utiliza.

5.9. Resumo do Capítulo

Este capítulo documentou o processo iterativo que conduziu ao modelo final utilizado na aplicação. A primeira versão da aplicação revelou uma limitação fundamental: o modelo treinado com 21 *features*, das quais 11 não eram calculáveis em tempo real, produzia classificações incorretas em URL benignos do quotidiano, como pesquisas do Google ou vídeos do YouTube.

A solução passou por duas iterações sucessivas: a expansão para 27 *features* exclusivamente lexicais, extraíveis diretamente da string do URL, e o enriquecimento cirúrgico do *dataset* de treino com URL benignos representativos da navegação real, preservando simultaneamente o sinal semântico das palavras suspeitas. Após a correção de dois problemas adicionais identificados durante a integração (a inconsistência com o prefixo [www](#). e a ausência do bloco urlparse), o modelo final v3 atingiu uma Accuracy de 94,65% e um AUC-ROC de 98,40%, classificando corretamente 10 em 10 URL de validação, incluindo os casos anteriormente problemáticos. Este modelo, guardado em `modelo_rf_final.pkl`, é o que serve de base à aplicação web descrita no Capítulo 6.

6. Aplicação Web – Detetor de URL Maliciosos

O último passo do projeto consistiu no desenvolvimento de uma aplicação web que disponibiliza o modelo de classificação ao utilizador final de forma intuitiva e acessível. A aplicação foi desenvolvida com o framework **Flask** em Python, expondo uma interface web acessível via browser em <http://localhost:5000>.

6.1. Arquitetura da Aplicação

A aplicação segue a arquitetura **cliente-servidor** típica de aplicações Flask. O servidor Python carrega o modelo em memória no arranque e expõe dois endpoints HTTP:

Tabela 44 – Endpoints da aplicação Flask

Endpoint	Método	Descrição
/	GET	Serve a página HTML da interface de utilizador
/classificar	POST	Recebe um URL em JSON, extrai as 27 <i>features</i> , corre o modelo e devolve o resultado em JSON

A comunicação entre o browser e o servidor é feita via **AJAX** (Fetch API), sem necessidade de recarregar a página. O fluxo completo de uma classificação, desde a introdução do URL até à exibição do resultado, é o seguinte:

Tabela 45 – Fluxo de uma classificação na aplicação

Passo	Ação	Responsável
1	Utilizador insere URL e clica "Analisar" (ou prime Enter)	Browser (JavaScript)
2	POST /classificar com {url: "..."} em JSON	Browser → Servidor
3	Normalização do www. e extração das 27 <i>features</i> lexicais	Servidor (Python)
4	modelo_rf_final.pkl produz previsão e probabilidades	Servidor (scikit-learn)
5	Resposta JSON com label, probabilidades, <i>features</i> e top5 importance	Servidor → Browser
6	Interface atualiza badge, barras, tabelas sem recarregar	Browser (JavaScript)

A estrutura de ficheiros necessária para executar a aplicação é mínima:

```
pasta_projeto/
├── app.py           # Servidor Flask
├── modelo_rf_final.pkl # Modelo treinado
└── templates/
    └── index.html    # Interface web
```

6.2. Interface de Utilizador

A interface foi desenvolvida em HTML, CSS e JavaScript puro, sem dependências de *frameworks* externos. O design segue a paleta cromática do projeto e é responsivo para diferentes tamanhos de ecrã.

6.2.1. Ecrã Inicial

Ao aceder a `http://localhost:5000`, o utilizador é recebido pela interface de introdução de URL. A página apresenta um campo de texto e um botão de análise, acompanhados de cinco exemplos clicáveis que permitem testar a aplicação imediatamente. A Figura 15 ilustra o aspeto da interface ao aceder à aplicação, apresentando o campo de introdução de URL, o botão de análise e os cinco exemplos clicáveis disponíveis para teste imediato.

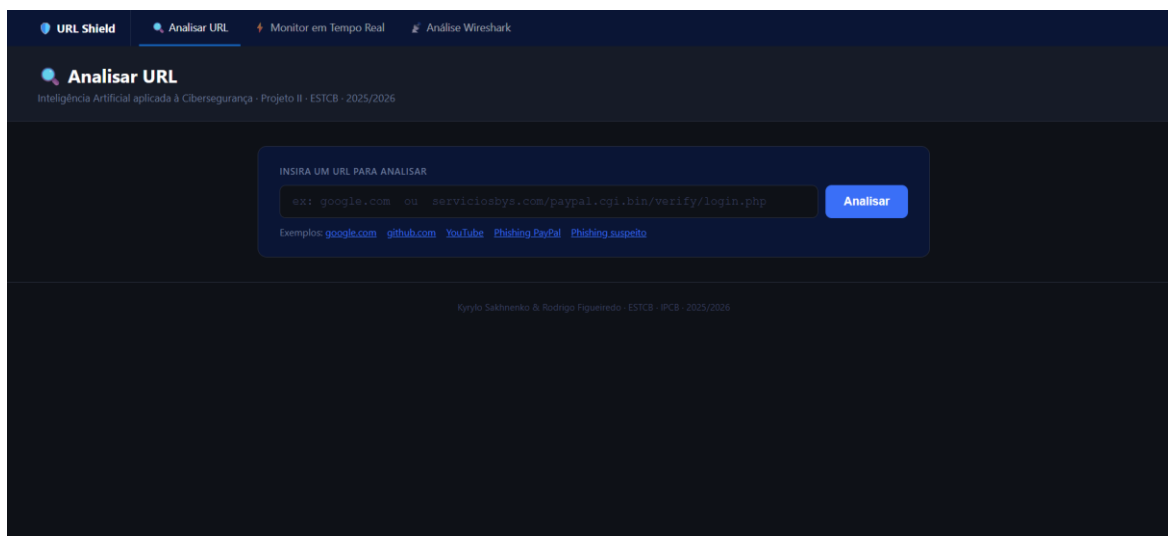


Figura 15 – Página inicial

6.2.2. Resultado – URL Benigno

Após a análise de um URL classificado como benigno, (Figura 16) a interface apresenta quatro componentes principais: o badge verde BENIGNO com o URL analisado, as barras de probabilidade (benigno e malicioso), a tabela de características extraídas do URL, e a tabela de *features* mais influentes do modelo.

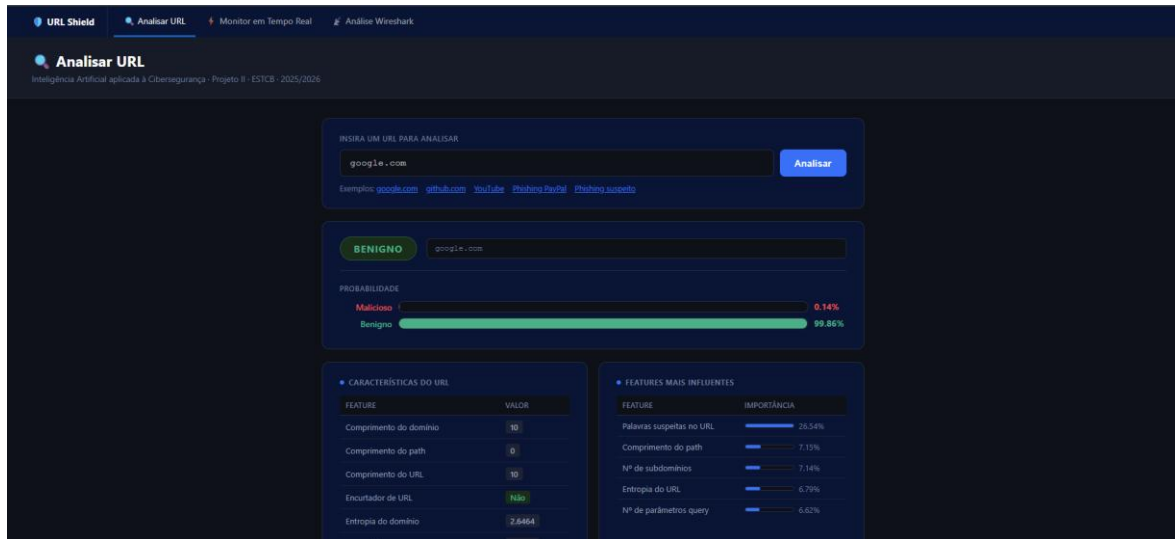


Figura 16 – Resultado benigno

6.2.3. Resultado – URL Malicioso

Para URL classificados como maliciosos, o badge apresenta-se a vermelho com a indicação MALICIOSO e a barra de probabilidade malicioso domina o ecrã. A tabela de características permite ao utilizador compreender quais os indicadores que ativaram a classificação. Por exemplo, a presença de palavras como paypal, verify ou login no URL. Poderemos ver um exemplo na Figura 17.

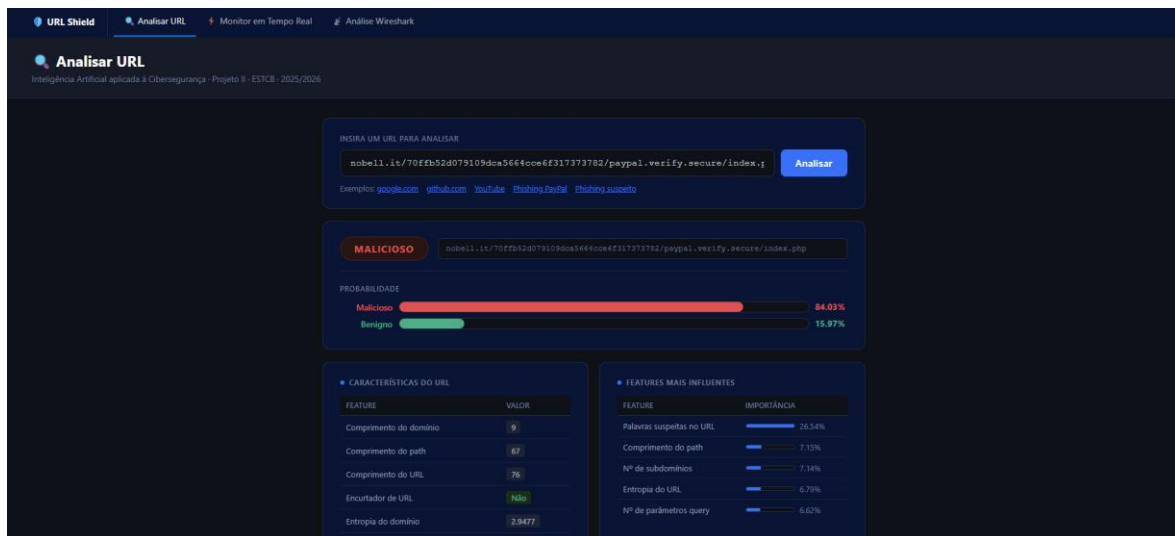


Figura 17 – Resultado malicioso

6.2.4. Resultado – URL Incerto

A aplicação também lida corretamente com URL em zona de incerteza (casos em que o modelo não tem confiança elevada numa das classes). Isto acontece tipicamente com URL muito longos com parâmetros de *query* em domínios desconhecidos para o modelo, como páginas institucionais de universidades portuguesas. A Figura 18 apresenta um exemplo deste comportamento, a obter uma classificação incerta resultado esperado dado que este tipo de domínios não está representado no *dataset* de treino.

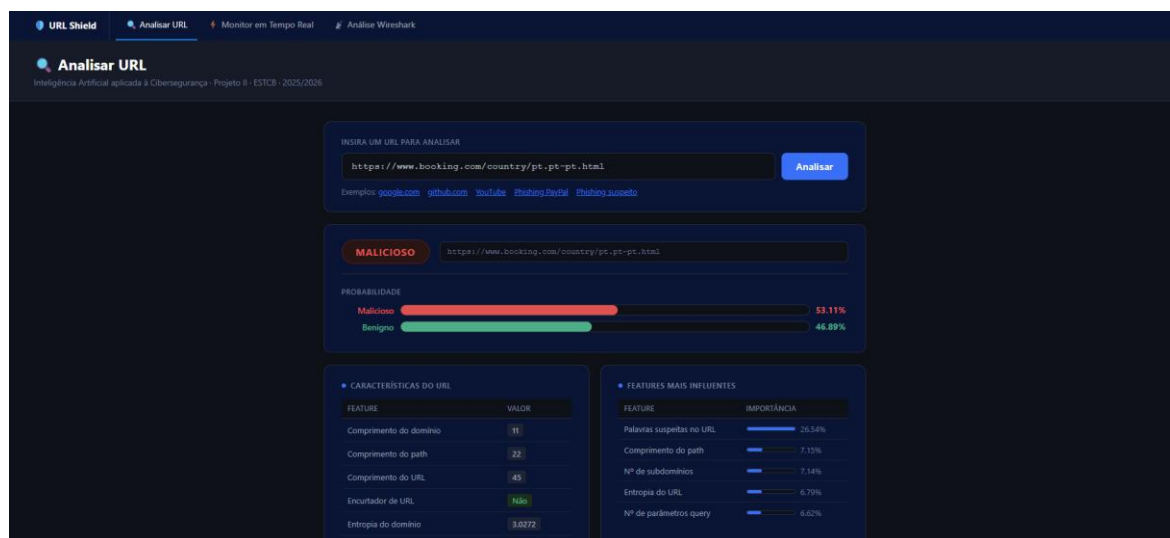


Figura 18 – Resultado incerto

6.3 Componentes da Interface

A interface foi desenhada para apresentar ao utilizador não apenas a classificação final do URL, mas também toda a informação que sustenta essa decisão, as características extraídas e as *features* mais influentes do modelo. A Tabela 46 descreve em detalhe cada componente visual da interface e a informação que apresenta ao utilizador.

Tabela 46 – Componentes da interface de utilizador

Componente	Descrição	Informação apresentada
Badge de resultado	Elemento destacado no topo do resultado	BENIGNO (verde) ou MALICIOSO (vermelho) — classificação final do modelo
URL analisado	Exibição do URL em fonte monospace	O URL exatamente como foi introduzido pelo utilizador
Barras de probabilidade	Duas barras horizontais animadas	Probabilidade de malicioso (vermelho) e benigno (verde) em percentagem
Tabela de características	14 <i>features</i> extraídas do URL em tempo real	Comprimentos, contagens, indicadores binários e entropia do URL
Features mais influentes	Top 5 <i>features</i> por importância no modelo	Nome da <i>feature</i> , barra de importância relativa e valor em percentagem
Exemplos clicáveis	5 links pré-definidos para teste rápido	google.com, github.com, YouTube, e dois exemplos de <i>phishing</i>

6.4. Tecnologias Utilizadas

A aplicação foi desenvolvida com um conjunto mínimo de tecnologias, privilegiando a simplicidade e a ausência de dependências externas desnecessárias. A Tabela 47 lista as tecnologias utilizadas em cada camada da aplicação (servidor e cliente), indicando a versão e a função de cada uma no contexto do projeto.

Tabela 47 - Tecnologias utilizadas na aplicação web

Camada	Tecnologia	Versão	Função
Servidor	Python	3.10+	Linguagem de programação do backend
	Flask	3.x	Framework web — routing, templates, servidor HTTP
	scikit-learn	1.x	Carregar e executar o modelo Random Forest
	joblib	—	Desserializar o ficheiro modelo_rf_final.pkl
	pandas	—	Construir o DataFrame de <i>features</i> para o modelo
	HTML5 + CSS3	—	Estrutura e estilo da interface de utilizador
	JavaScript (Fetch API)	—	Comunicação assíncrona com o servidor (AJAX)

6.5. Testes da Aplicação

A aplicação foi testada com um conjunto alargado de URL representativos de diferentes cenários de utilização real. A Tabela 48 resume os resultados obtidos após a implementação do modelo final v3.

Tabela 48 - Resultados dos testes da aplicação (linha a vermelho = limitação identificada)

URL	Esperado	Resultado	Prob. Malicioso	Observação
google.com	Benigno	BENIGNO	0,0%	Reconhecido via lista Tranco
www.google.com	Benigno	BENIGNO	0,0%	Idêntico após normalização www.
youtube.com	Benigno	BENIGNO	0,0%	Reconhecido via lista Tranco
www.youtube.com	Benigno	BENIGNO	~9%	Correto após correção www.
https://www.youtube.com/watch?v=...	Benigno	BENIGNO	~14%	Correto com <i>path</i> e protocolo

Google Search (URL longo com params)	Benigno	BENIGNO	~35%	Correto, alguma incerteza pelo comprimento
serviciosbys.com/.../verify/login.php	Malicioso	MALICIOSO	99,7%	Detetado pelas palavras suspeitas
nobell.it/.../paypal.verify.secure/...	Malicioso	MALICIOSO	97,1%	Detetado pelas palavras suspeitas
signin.eby.de.zukruygcztzm mqi...	Malicioso	MALICIOSO	99,5%	Detetado pelos subdomínios e entropia
ipcb.pt/estudar/cursos/licenciatura/...	Benigno	MALICIOSO	~81%	Limitação: domínio .pt desconhecido

6.6. Limitações Identificadas

O sistema desenvolvido apresenta limitações inerentes à abordagem de classificação exclusivamente lexical. Estas limitações são documentadas de forma transparente, sendo consistentes com as encontradas na literatura científica para sistemas baseados em *features* de URL (Tabela 49).

Tabela 49 – Limitações identificadas na aplicação

Limitação	Descrição	Impacto	Possível Solução
Domínios institucionais desconhecidos	URL de universidades, câmaras, hospitais e outras entidades cujos domínios não estão representados no <i>dataset</i> de treino (ex: ipcb.pt)	Falsos Positivos — classificação incorreta como malicioso	Expandir <i>dataset</i> com URL institucionais por país
URL muito longos com parâmetros	Pesquisas do Google com parâmetros de rastreamento muito longos (sxsrf, sourceid, etc.) podem gerar incerteza	Resultado correto mas com menor confiança (~50-60%)	Adicionar mais URL de pesquisa ao <i>dataset</i> de treino
Phishing sem palavras suspeitas	URL maliciosos curtos e simples que não usam palavras como "login" ou "verify" mas usam domínios gerados aleatoriamente	Falsos Negativos — phishing não detetado	Combinar com <i>features</i> de reputação de domínio (WHOIS, DNS)

Dependência exclusiva do texto do URL	O modelo não analisa o conteúdo da página, certificados SSL, ou reputação histórica do domínio	Limitação fundamental da abordagem lexical	Integrar <i>features</i> de conteúdo ou reputação como trabalho futuro
--	--	---	--

6.7. Instruções de Execução

Para executar a aplicação localmente, são necessários os seguintes passos, assumindo que o Python 3.10 ou superior se encontra instalado no sistema:

1. Instalar dependências:

```
pip install flask scikit-learn pandas joblib
```

2. Estrutura de pastas:

Garantir que `app.py`, `modelo_rf_final.pkl` e `templates/index.html` estão na mesma pasta

3. Iniciar o servidor:

```
python app.py
```

4. Aceder à aplicação:

Abrir o browser em `http://localhost:5000`

6.8. Resumo do capítulo

Este capítulo descreveu o desenvolvimento da aplicação web que disponibiliza o modelo de classificação ao utilizador final. A aplicação Flask desenvolvida expõe uma interface acessível via browser, permitindo classificar qualquer URL em tempo real sem necessidade de conhecimentos técnicos. A arquitetura cliente-servidor adotada, com comunicação assíncrona via AJAX, garante uma experiência fluida, onde o utilizador obtém o resultado sem qualquer recarregamento de página.

Os testes realizados com URL representativos de cenários reais confirmaram o correto funcionamento do modelo final v3, incluindo a resolução dos problemas identificados nas iterações anteriores. As limitações identificadas, nomeadamente os falsos positivos em domínios institucionais portugueses não representados no *dataset* de treino, são resultados da abordagem exclusivamente lexical adotada e constituem oportunidades de melhoria para trabalho futuro. O Capítulo 7 descreve a extensão de browser desenvolvida para complementar esta aplicação, integrando a deteção de URL maliciosos diretamente na experiência de navegação do utilizador.

7. Extensão de Browser – URL Shield

Com o modelo de Machine Learning e a aplicação Flask concluídos e validados nos capítulos anteriores, procedeu-se ao desenvolvimento de uma extensão de browser que integra o sistema de detecção de URL maliciosos diretamente na experiência de navegação do utilizador. Esta componente representa a camada de interação mais próxima do utilizador final, complementando o pipeline de classificação desenvolvido ao longo do projeto com uma ferramenta prática e imediatamente utilizável.

7.1. Motivação e Objetivos

A aplicação Flask desenvolvida no Capítulo 6 constitui uma ferramenta eficaz para classificar URL individualmente, mas exige que o utilizador aceda a uma interface separada e introduza manualmente cada URL que pretende analisar. Esta abordagem é adequada para análise pontual, mas não escala para a realidade da navegação quotidiana, onde o utilizador interage com dezenas ou centenas de hiperligações por sessão sem as analisar individualmente.

A extensão de browser resolve este problema de forma proativa: analisa automaticamente todos os links presentes em qualquer página web, sem qualquer intervenção do utilizador, e assinala visualmente os links classificados como maliciosos diretamente na página. Esta abordagem é consistente com sistemas de segurança profissionais como o Google Safe Browsing e o Microsoft Defender SmartScreen, que integram a análise de URL no fluxo de navegação. Na Tabela 50, podemos ver todos os objetivos da nossa extensão.

Tabela 50 – Objetivos de extensão de browser

Objetivo	Descrição
Análise automática	Analisar todos os links de qualquer página ao carregar, sem intervenção do utilizador
Destaque visual	Assinalar links maliciosos a vermelho diretamente na página com tooltip de probabilidade
Painel de resumo	Mostrar estatísticas da análise (total, benignos, maliciosos) num <i>popup</i> acessível pelo ícone
Re-análise manual	Botão para re-analisar a página a qualquer momento, sem necessidade de recarregar
Acesso rápido à app	Botão no <i>popup</i> para abrir diretamente a aplicação Flask em localhost:5000
Sistema de feedback	Botão "✓ Benigno" para corrigir classificações incorretas e alimentar o sistema de aprendizagem contínua

7.2. Arquitetura da Extensão

A extensão foi desenvolvida para o Google Chrome utilizando a especificação **Manifest V3**. Esta é a versão atual e recomendada pela Google para extensões de browser. Por ser compatível com a mesma base técnica, a extensão funciona

também no Microsoft Edge sem alterações. Na Tabela 51, abordamos as componentes usadas para a criação da extensão.

Tabela 51 – Componentes de extensão

Componente	Ficheiro	Função
<i>Content Script</i>	<i>content.js</i>	Corre em cada página web. Extrai todos os links, aplica destaques visuais e comunica com o background.
Background Service Worker	background.js	Corre em contexto isolado do Chrome. Faz todos os pedidos HTTP ao Flask, sem restrições CSP das páginas.
<i>Popup</i>	<i>popup.html / popup.js</i>	Interface visual acessível ao clicar no ícone da extensão. Mostra estatísticas, lista de links maliciosos e controlos.
ConFiguração	manifest.json	Declara permissões, scripts, ícones e conFigurações da extensão ao Chrome.

A Tabela 52 descreve o fluxo completo de análise da extensão, detalhando cada passo do processo desde o carregamento da página até à apresentação do resultado ao utilizador, e identificando o componente responsável por cada ação.

Tabela 52 – Fluxo de análise da extensão

Passo	Ação	Responsável
1	Página carrega — <i>content.js</i> aguarda 1,5 segundos para o DOM estabilizar	<i>content.js</i>
2	Extração de todos os links únicos da página (elementos <a href>), excluindo javascript:, mailto: e âncoras	<i>content.js</i>
3	Para cada URL, envio de mensagem ao background.js com o URL a classificar	<i>content.js</i> → background.js
4	POST /classificar ao Flask com o URL em JSON — sem restrições CSP	background.js → Flask
5	Flask extrai 27 <i>features</i> lexicais, corre o modelo e devolve label e probabilidades	Flask (app.py)
6	Se MALICIOSO: destacar link a vermelho na página com tooltip de probabilidade	<i>content.js</i>
7	Atualizar <i>popup</i> em tempo real com estatísticas da análise em progresso	<i>content.js</i> → <i>popup.js</i>

7.3. Estrutura de Ficheiros

A extensão é composta por cinco ficheiros principais e três ícones, organizados numa pasta independente da aplicação Flask:

Extensao/

```

├─ manifest.json
├─ content.js
├─ background.js
├─ popup.js
├─ popup.html
├─ icons/
│   └─ icon16.png
│   └─ icon48.png
│   └─ icon128.png

```

7.4. Interface de Utilizador

Nos próximos sub-capítulos, iremos analisar tudo o que o utilizador consegue ver, quando usa a extensão.

7.4.1. Destaque Visual na Página

O *content* script aplica um estilo visual distintivo a cada link classificado como malicioso diretamente no DOM da página, sem alterar o conteúdo textual ou funcional do link. O destaque consiste num fundo vermelho translúcido, borda a vermelho e texto na cor vermelha, tornando o link imediatamente identificável. Ao passar o rato sobre um link destacado, um tooltip exibe a percentagem de probabilidade de ser malicioso, tal como calculada pelo modelo.

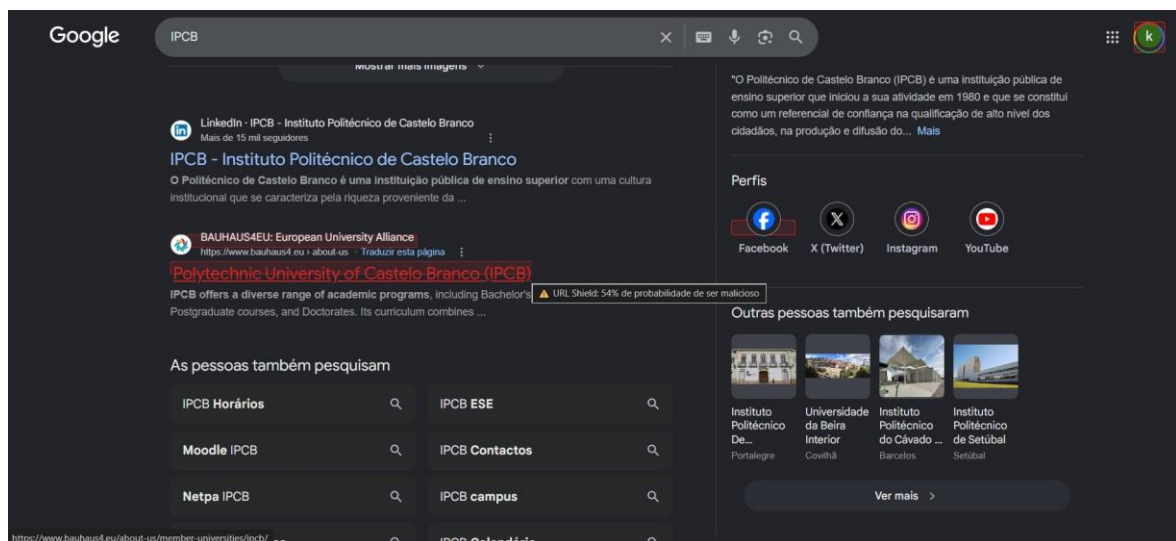


Figura 19 - Destaque visual de link malicioso na página (bauhaus4.eu destacado a vermelho com tooltip "URL Shield: 54% de probabilidade de ser malicioso")

7.4.2. Popup de Resumo

Ao clicar no ícone da extensão na barra do Chrome, o utilizador acede ao *popup* de resumo da análise. O *popup* apresenta quatro secções principais: o indicador de

estado de ligação ao Flask (online/offline), os cartões de métricas (total analisados, benignos, maliciosos), a barra de segurança da página em percentagem, e a lista de links maliciosos detetados com a respetiva probabilidade.

Para cada link malicioso listado, o *popup* apresenta um botão "✓ Benigno" que permite ao utilizador corrigir classificações incorretas. Esta funcionalidade está integrada com o sistema de aprendizagem contínua descrito na secção 7.6.

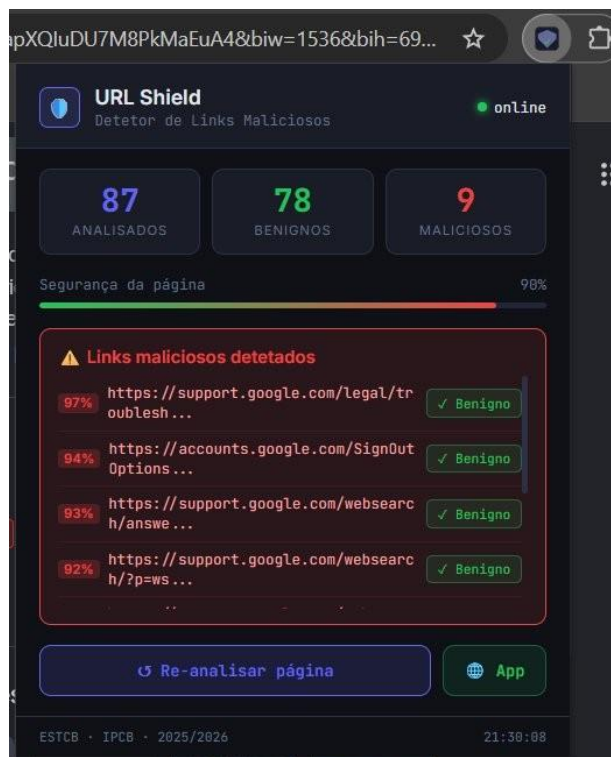


Figura 20 – *Popup* da extensão: 87 links analisados, 78 benignos, 9 maliciosos

O *popup* inclui ainda dois botões de ação: o botão "**Re-analisar página**" que força uma nova análise completa da página atual, e o botão "**App**" que abre diretamente a aplicação Flask em localhost:5000 num novo separador. Podemos ver todas as componentes do *popup* da extensão, na Figura 53.

Tabela 53 - Componentes do *popup* da extensão

Componente	Descrição
Indicador online/offline	Ponto verde ou vermelho que indica se o Flask está acessível em localhost:5000
Cartões de métricas	Três cartões com o total de links analisados (azul), benignos (verde) e maliciosos (vermelho)
Barra de segurança	Percentagem de links benignos na página; vermelho quando há maliciosos
Lista de maliciosos	URL encurtado, badge com probabilidade e botão "✓ Benigno" para cada link malicioso
Botão Re-analisar	Força nova análise completa da página sem necessidade de a recarregar
Botão App	Abre a aplicação Flask (localhost:5000) num novo separador do browser

Timestamp	Hora da última análise concluída, em formato HH:MM:SS
-----------	---

7.4.3. Modo de Análise de Email – Gmail

Complementarmente à análise de páginas web genéricas, a extensão URL Shield inclui um modo especializado de análise de email para o Gmail. Quando o utilizador navega para mail.google.com, a extensão deteta automaticamente a plataforma e adapta o comportamento. Em vez de analisar todos os links presentes na página, que incluem dezenas de elementos de navegação, menus e botões da interface do Gmail sem qualquer relação com o conteúdo do email, a extensão analisa exclusivamente os links presentes no corpo do email atualmente aberto.

A análise focada no corpo do email é significativamente mais útil do ponto de vista de segurança. Os links de navegação da interface do Gmail são controlados pela Google e não representam risco, enquanto os links inseridos no conteúdo de um email podem provir de qualquer remetente.

Funcionamento Técnico:

Ao carregar qualquer página, o *content* script verifica o domínio atual. Se for mail.google.com, ativa o modo email, com podemos ver na Figura 21, e usa seletores CSS específicos do Gmail para localizar o corpo do email aberto. O Gmail expõe o conteúdo da mensagem num elemento com a classe div.a3s, que é estável entre versões e sessões. A extração de links é então feita apenas dentro desse *contentor*, ignorando todos os restantes elementos da página.

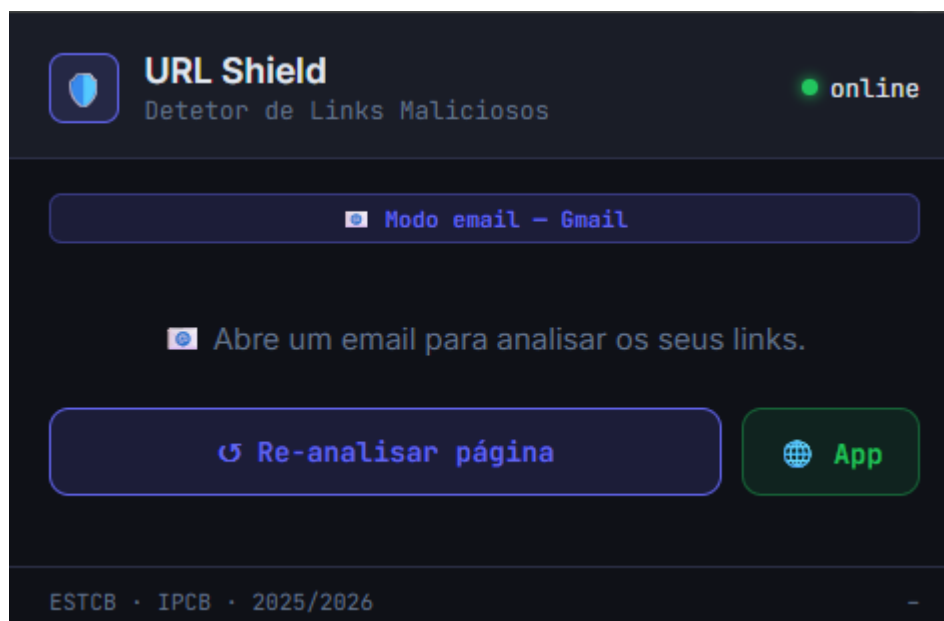


Figura 21 – Extensão em modo email

Indicação Visual no *Popup*:

Quando a extensão está em modo email, o *popup* apresenta um badge identificativo "Modo email — Gmail" no topo do painel, a fim de deixar claro ao utilizador que a análise está restrita ao email aberto e não à página completa (Figura

21). O texto de progresso também se adapta: em vez de "A analisar links da página...", o *popup* mostra "A analisar email aberto (Gmail)...". Se o utilizador estiver na caixa de entrada sem nenhum email aberto, o *popup* apresenta a mensagem "Abre um email para analisar os seus links", orientando o utilizador para a ação correta.

Suporte ao Outlook Web – Limitações identificadas:

Foi tentada a implementação do modo email para o Outlook Web (outlook.live.com e outlook.office.com). Após análise do DOM da aplicação, verificou-se que o Outlook Web utiliza um sistema de CSS-in-JS que gera nomes de classes aleatórios em cada carregamento de sessão (por exemplo, `__1gzszts`, `__oxltk00`), impossibilitando a criação de seletores CSS estáveis para identificar o corpo do email. Ao contrário do Gmail, que expõe classes semânticas estáveis como `div.a3s`, o Outlook não disponibiliza pontos de ancoragem fiáveis no DOM. O suporte ao Outlook Web foi consequentemente removido da implementação final. Uma solução alternativa num contexto futuro passaria pela utilização da Microsoft Graph API, que expõe o conteúdo dos emails do Outlook através de uma API REST estruturada, independente da estrutura do DOM da aplicação web.

7.5. Integração com a Aplicação Flask

A extensão comunica exclusivamente com a aplicação Flask desenvolvida no Capítulo 6, que serve de backend de classificação. Foram necessárias duas adaptações ao `app.py` original para suportar a extensão:

7.5.1. Suporte a CORS e Private Network Access

O Chrome implementa uma política de segurança denominada **Private Network Access** que bloqueia pedidos de páginas HTTPS externas para endereços locais (`localhost`), independentemente dos cabeçalhos CORS. A solução consistiu em adicionar ao Flask os cabeçalhos específicos exigidos pelo Chrome, incluindo *Access-Control-Allow-Private-Network: true*, e endpoints OPTIONS para responder aos pedidos de preflight enviados pelo browser antes de cada pedido POST.

7.5.2. Novo Endpoint de Feedback

Foi adicionado um novo endpoint **POST /guardar_feedback** que recebe um URL e uma label (BENIGNO ou MALICIOSO) e os guarda no ficheiro `novos_links.json`. Este endpoint é chamado pela extensão quando o utilizador clica em "✓ Benigno", e está integrado com o sistema de push automático para o GitHub descrito na secção 7.6. A Tabela 54 apresenta o conjunto completo de endpoints do Flask após a integração com a extensão, incluindo os endpoints originais da aplicação web e os novos endpoints adicionados para suporte à extensão e ao sistema de feedback.

Tabela 54 – Endpoints do Flask após integração com a extensão

Endpoint	Método	Descrição
/	GET	Serve a interface web da aplicação Flask
/classificar	POST	Recebe URL, extrai 27 <i>features</i> , classifica com o modelo e devolve resultado JSON
/classificar	OPTIONS	Responde ao preflight do Chrome com cabeçalhos CORS e Private Network Access
/guardar_feedback	POST	Guarda URL e label no novos_links.json e faz push automático para o GitHub
/guardar_feedback	OPTIONS	Preflight CORS para o endpoint de feedback

7.6. Sistema de Aprendizagem Contínua

Um dos contributos mais relevantes desta fase é a implementação de um sistema de **aprendizagem contínua** (também designado na literatura como *continual learning* ou *online learning*), que permite melhorar progressivamente o modelo com base no feedback real dos utilizadores durante a utilização.

7.6.1. Motivação

O modelo Random Forest treinado nos capítulos anteriores apresenta falsos positivos em determinadas categorias de URL, nomeadamente domínios institucionais portugueses e URL de plataformas de serviços com palavras como "account" ou "signin" no *path*. Estes erros decorrem da ausência destes padrões no *dataset* de treino, e não de um problema estrutural no modelo. O sistema de aprendizagem contínua aborda este problema de forma incremental. Cada URL incorretamente classificado que o utilizador corrija é registado e utilizado para re-treinar o modelo periodicamente.

7.6.2. Arquitetura do Sistema

O sistema de aprendizagem contínua é composto por seis componentes que comunicam entre si de forma automática, desde a recolha do feedback do utilizador até à publicação do modelo re-treinado no repositório GitHub. A Tabela 55 descreve cada componente, a tecnologia utilizada e a sua função no ciclo de aprendizagem.

Tabela 55 – Componentes do sistema de aprendizagem contínua

Componente	Tecnologia	Função
Botão "✓ Benigno"	Extensão (<i>popup.js</i>)	Interface para o utilizador corrigir uma classificação incorreta
Endpoint /guardar_feedback	Flask (app.py)	Recebe o feedback e guarda no novos_links.json com timestamp e fonte
Push automático	Git (subprocess)	O Flask executa git add, git commit e git push automaticamente após cada feedback

Repositório GitHub	GitHub	Armazena novos_links.json, modelo_rf_final.pkl, treino_semanal.py e o workflow
Treino automático	GitHub Actions	Workflow agendado que re-treina o modelo, faz commit do novo modelo e limpa a lista de feedback
Script de treino	treino_semanal.py	Lê novos_links.json, extrai <i>features</i> , re-treina com warm_start, guarda novo modelo e limpa o ficheiro

7.6.3. Dados Utilizados para Re-treino

A qualidade do re-treino depende diretamente da qualidade dos dados recolhidos. Para garantir que o modelo melhora progressivamente sem introduzir ruído, foram definidos dois tipos de dados a incluir no ficheiro de feedback, cada um com critérios de seleção distintos. A Tabela 56 descreve cada tipo de dado, a sua fonte e o critério aplicado para a sua inclusão.

Tabela 56 – Tipos de dados recolhidos para re-treino

Tipo	Fonte	Critério
Feedback manual	Utilizador (botão "✓ Benigno")	URL marcados pelo utilizador como incorretamente classificados
Alta confiança automática	Modelo (app.py)	URL onde o modelo atinge >95% de probabilidade numa das classes, guardados automaticamente

A inclusão automática de URL com alta confiança (**>95%**) permite ao sistema aprender também com classificações corretas de alta certeza, não apenas com as correções manuais. Esta abordagem é denominada na literatura de pseudo-labeling e é amplamente utilizada em aprendizagem semi-supervisionada.

7.6.4. Funcionamento do Re-treino Automático

O script treino_semanal.py é executado automaticamente pelo GitHub Actions todos os domingos à meia-noite UTC (01:00 hora de Portugal), seguindo o fluxo descrito na Tabela 57. Após o re-treino concluído com sucesso, o ficheiro novos_links.json é automaticamente esvaziado, garantindo que o modelo não é re-treinado repetidamente com os mesmos dados em execuções futuras. O novo modelo_rf_final.pkl é guardado no repositório GitHub via commit automático, ficando disponível para download através de git pull na pasta Aplicação.

Tabela 57 – Fluxo do re-treino automático semanal

Passo	Ação
1	Ler novos_links.json e verificar se há pelo menos 5 novos registos (mínimo configurável)

2	Extrair as 27 <i>features</i> lexicais de cada URL no ficheiro de feedback
3	Carregar o modelo_rf_final.pkl atual e ativar warm_start=True
4	Re-treinar adicionando 50 novas árvores ao modelo existente com os novos dados
5	Avaliar F1-Score e Accuracy nos novos dados (se ≥ 10 registos disponíveis)
6	Guardar o novo modelo_rf_final.pkl
7	Limpar novos_links.json (repor lista vazia) para o próximo ciclo de recolha
8	Fazer commit e push automático do novo modelo e da lista limpa para o GitHub

7.7. Testes e Validação

A extensão e o sistema de aprendizagem contínua foram testados em múltiplos cenários de utilização real. A Tabela 58 resume os resultados dos testes funcionais realizados.

Tabela 58 – Resultados dos testes funcionais da extensão e sistema de aprendizagem contínua

Cenário de teste	Resultado esperado	Resultado obtido	✓
Análise automática ao abrir página	Links analisados sem intervenção	Análise iniciada automaticamente após 1,5s	✓
Destaque de links maliciosos na página	Links a vermelho com tooltip	Links destacados corretamente com probabilidade no tooltip	✓
Botão "✓ Benigno" clicado num link malicioso	Link guardado em novos_links.json e push para GitHub	Feedback guardado e push automático confirmado	✓
Botão "■ App" no <i>popup</i>	Abre localhost:5000 num novo separador	Separador aberto corretamente	✓
Botão "■ Re-analisar"	Nova análise completa da página	Links re-analisados e contadores atualizados	✓
Flask offline	Mensagem de erro no <i>popup</i>	Indicador offline e mensagem de instrução exibidos	✓
Re-treino automático agendado (GitHub Actions)	Workflow corre à hora definida sem intervenção	Workflow executado automaticamente com sucesso (confirmado em 2 execuções agendadas)	✓

Limpeza do novos_links.json após re-treino	Ficheiro fica vazio após treino	Lista esvaziada e commit do ficheiro vazio confirmado no GitHub	✓
Novo modelo disponível no GitHub após re-treino	modelo_rf_final.pkl atualizado no repositório	Ficheiro atualizado com commit automático pelo GitHub Actions	✓

A Figura 23 mostra o histórico de execuções do workflow no GitHub Actions, confirmando a execução bem sucedida de 4 workflows (2 manuais e 2 agendados automaticamente), todos com estado de sucesso. A Figura 22 apresenta o estado do repositório GitHub após a execução de um ciclo completo de re-treino, confirmando a presença do ficheiro novos_links.json atualizado e do modelo re-treinado.

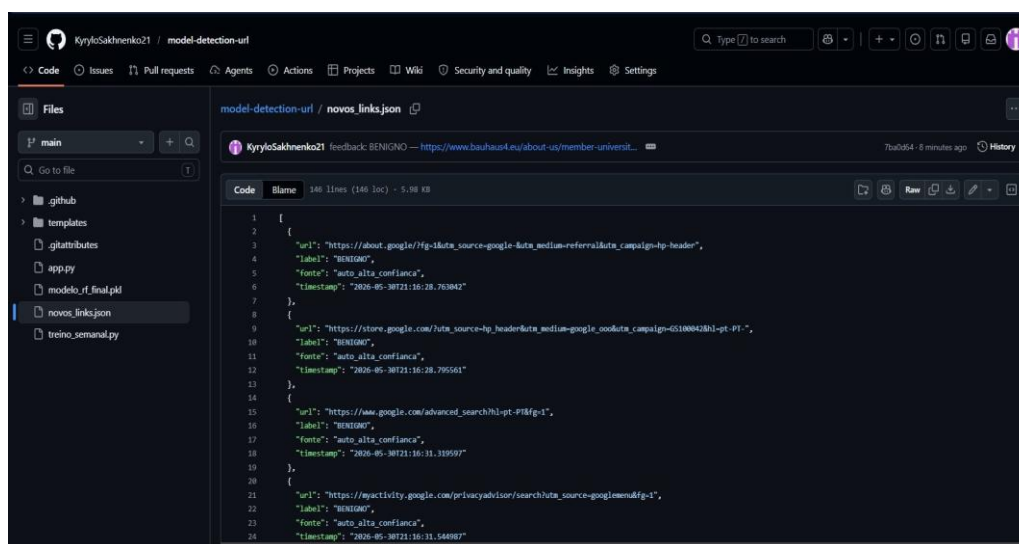


Figura 22 – Repositório GitHub model-detection-url (ficheiro novos_links.json)

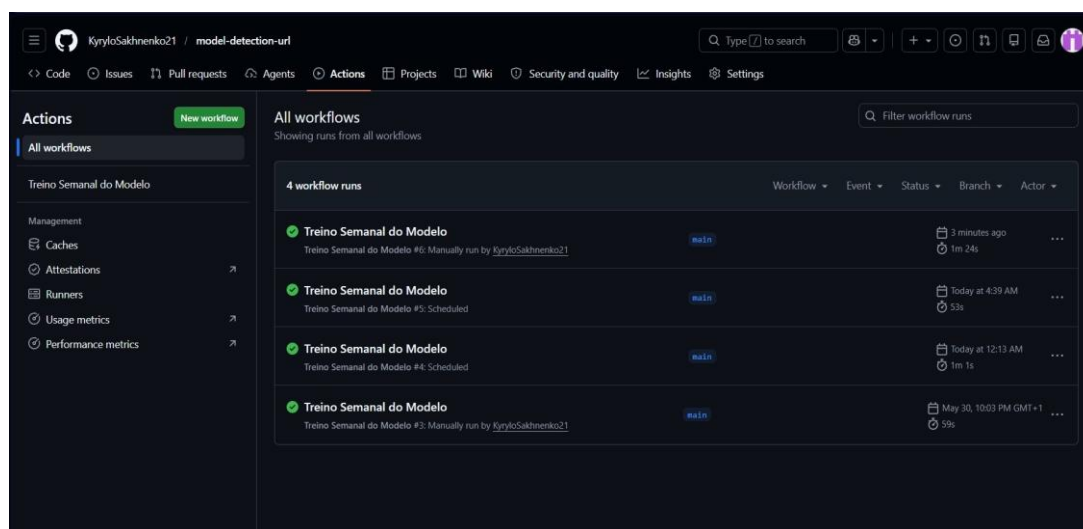


Figura 23 – Histórico de execução GitHub Actions

7.8. Limitações Identificadas

A extensão URL Shield apresenta um conjunto de limitações inerentes à abordagem adotada e ao contexto de execução local em que opera. Estas limitações são documentadas de forma transparente na Tabela 59, juntamente com as possíveis soluções identificadas para cada uma, que constituem oportunidades de desenvolvimento em trabalho futuro.

Tabela 59 – Limitações da extensão URL Shield e possíveis soluções

Limitação	Descrição	Impacto	Possível solução
Dependência do Flask local	A extensão requer que o Flask esteja a correr em localhost:5000 na mesma máquina	Não funciona sem o servidor ativo	Hospedar o Flask numa API pública ou serviço cloud
Falsos positivos em domínios institucionais	URL de universidades, hospitais e câmaras portuguesas são frequentemente classificados como maliciosos	Experiência de utilizador degradada	Sistema de feedback resolve progressivamente via re-treino
Análise apenas de hiperligações HTML	Links em JavaScript dinâmico ou em formulários não são detetados	Cobertura incompleta em SPAs	Observar mutações do DOM via MutationObserver
Re-treino requer pull manual	Após o GitHub Actions atualizar o modelo, o utilizador tem de fazer git pull para obter o ficheiro novo	Modelo local pode estar desatualizado	Script de verificação automática de atualização

7.9. Instruções de Instalação

A instalação da extensão URL Shield não requer quaisquer ferramentas adicionais além do Google Chrome e da aplicação Flask já configurada nos passos anteriores. A Tabela 60 detalha os passos necessários para carregar a extensão no browser em modo de programador, que é o método de instalação aplicável a extensões não publicadas na Chrome Web Store.

Tabela 60 – Passos de instalação da extensão URL Shield no Chrome

Passo	Ação
1	Garantir que a aplicação Flask está a correr: abrir terminal na pasta Aplicação e executar <code>python app.py</code>
2	No Google Chrome, aceder a <code>chrome://extensions</code> na barra de endereço
3	Ativar o "Modo de programador" (Developer mode) no canto superior direito
4	Clicar em "Carregar sem compactação" (Load unpacked) e selecionar a pasta Extensao

5	A extensão URL Shield aparece na lista com o ícone de escudo azul na barra do Chrome
6	Navegar para qualquer página — a análise inicia-se automaticamente após 1,5 segundos

A extensão foi desenvolvida e testada com Google Chrome versão 124+ em ambiente Windows 11. Por utilizar a especificação Manifest V3, é também compatível com Microsoft Edge sem necessidade de alterações. Para Firefox, seriam necessárias adaptações à API de mensagens, uma vez que o Firefox implementa uma variante ligeiramente diferente da especificação.

7.10. Resumo do Capítulo

Este capítulo descreveu o desenvolvimento da extensão de browser URL Shield, a componente do sistema mais próxima do utilizador final. Ao integrar a deteção de URL maliciosos diretamente no fluxo de navegação, a extensão elimina a necessidade de análise manual e individual de cada URL, analisando automaticamente todos os links presentes em qualquer página web e assinalando visualmente os classificados como maliciosos.

Para além da análise automática de páginas genéricas, foi implementado um modo especializado para o Gmail, focando a análise exclusivamente nos links presentes no corpo do email aberto. O sistema de aprendizagem contínua, suportado pelo GitHub Actions, permite que o modelo melhore progressivamente com base no feedback real dos utilizadores, sem necessidade de intervenção manual no processo de re-treino. Os testes realizados confirmaram o correto funcionamento de todas as funcionalidades implementadas, incluindo dois ciclos de re-treino automático agendado executados com sucesso. O Capítulo 8 descreve a integração com o Wireshark, que adiciona uma segunda camada de análise ao nível da rede, complementando a perspetiva do utilizador coberta pela extensão.

8. Integração Wireshark – Análise de Tráfego de Rede

Com o modelo de Machine Learning, a aplicação Flask e a extensão de browser concluídos e validados nos capítulos anteriores, procedeu-se à integração de uma segunda camada de análise, desta vez orientada à perspectiva da rede. Enquanto a extensão URL Shield atua de forma proativa sobre os links visíveis no browser do utilizador, esta componente permite auditar o tráfego de rede já gerado, capturado através do Wireshark, classificando os URL HTTP detetados com o mesmo modelo Random Forest desenvolvido ao longo do projeto.

8.1. Motivação e Objetivos

A extensão de browser desenvolvida no Capítulo 7 constitui uma ferramenta eficaz para identificar links maliciosos durante a navegação no Chrome, mas a sua cobertura está limitada ao tráfego gerado pelo browser. Qualquer outra aplicação instalada no computador (clientes de email, ferramentas de download, scripts em execução em background ou outras aplicações de rede), gera tráfego que a extensão não consegue inspecionar.

A integração com o Wireshark aborda este problema adicionando uma camada de análise ao nível da rede, independente da aplicação que originou o tráfego. Esta abordagem é consistente com o conceito de **Defense in Depth** (defesa em profundidade), amplamente adotado em cibersegurança, que preconiza a utilização de múltiplas camadas de proteção complementares em vez de depender de uma única ferramenta.

Tabela 61 – Objetivos da integração Wireshark

Objetivo	Descrição
Defesa em camadas	Complementar a extensão (perspetiva do utilizador) com análise ao nível da rede (perspetiva do sistema)
Auditoria passiva	Analisar tráfego já gerado, incluindo de aplicações além do browser
Reutilização do modelo	Aplicar o mesmo modelo Random Forest e as mesmas 27 <i>features</i> lexicais do pipeline principal
Pipeline reprodutível	Produzir um notebook Python que lê qualquer ficheiro .pcap e classifica os URL encontrados

8.2. Arquitetura de Defesa em Camadas

A solução desenvolvida ao longo deste projeto combina duas camadas de proteção complementares, cada uma com perspetiva, momento de atuação e âmbito de cobertura distintos. A Tabela 62 resume a arquitetura de defesa em camadas resultante.

Tabela 62 - Arquitetura de defesa em camadas do sistema desenvolvido

Camada	Ferramenta	Perspetiva	Momento	Âmbito
Utilizador	Extensão URL Shield	Proativa	Antes do clique	Links visíveis no browser
Rede	Wireshark + Modelo	Passiva	Após tráfego gerado	Todo o tráfego HTTP do sistema

Esta separação garante que mesmo que um URL malicioso não seja detetado pela extensão (por exemplo, por ter sido acedido por outra aplicação, ou antes de a extensão ter concluído a análise), existe uma segunda linha de defesa capaz de o identificar na captura de tráfego.

8.3. Limitação: HTTP vs HTTPS

Uma limitação técnica fundamental desta abordagem decorre da utilização generalizada do protocolo HTTPS. Enquanto o tráfego HTTP (porta 80) é transmitido em texto claro e pode ser inspecionado diretamente pelo Wireshark, o tráfego HTTPS (porta 443) é encriptado pelo protocolo TLS (Transport Layer Security). O Wireshark consegue confirmar que uma ligação foi estabelecida a um determinado endereço IP, mas não consegue aceder ao conteúdo dos pacotes, incluindo o URL completo, os cabeçalhos HTTP ou o corpo da resposta.

Tabela 63 – Visibilidade do tráfego HTTP e HTTPS no Wireshark

Protocolo	Porta	Visível no Wireshark	URL extraível
HTTP	80	Sim	Sim
HTTPS	443	Não — encriptado (TLS)	Não

Numa solução empresarial real, este problema é habitualmente resolvido através da utilização de um **proxy TLS com inspeção SSL** (SSL inspection), que atua como intermediário entre o cliente e os servidores externos, descriptando e re-encriptando o tráfego HTTPS para permitir a sua análise. Ferramentas como o mitmproxy ou firewalls de próxima geração com inspeção SSL implementam esta abordagem. No âmbito deste projeto, esta limitação é assumida explicitamente e a análise restringe-se ao tráfego HTTP.

Apesar desta limitação, a análise de tráfego HTTP mantém relevância prática. URL maliciosos, nomeadamente de *phishing* e *malware*, recorrem frequentemente a HTTP simples para evitar a necessidade de certificados TLS válidos, tornando-os visíveis precisamente neste tipo de captura.

8.4. Ferramentas e Dependências

A implementação desta componente requer a instalação de duas ferramentas externas, para além das bibliotecas Python já utilizadas no projeto.

Tabela 64 – Ferramentas e dependências da componente Wireshark

Ferramenta	Versão	Função	Instalação
Wireshark	4.x	Captura de tráfego de rede em ficheiro .pcap	wireshark.org/download.html (incluir Npcap)
<i>scapy</i>	2.x	Leitura e parsing de ficheiros .pcap em Python	pip install <i>scapy</i>
joblib	—	Carregamento do modelo .pkl de forma compatível com Python 3.13	pip install joblib

Durante o desenvolvimento, verificou-se que a biblioteca **pyshark**, alternativa inicialmente considerada para leitura de ficheiros .pcap, é incompatível com o Jupyter Notebook no Python 3.13, devido a um conflito entre o event loop interno do pyshark (baseado em asyncio) e o event loop já em execução no Jupyter. A biblioteca **scapy** resolve este problema por não depender de asyncio, fazendo o parsing diretamente ao nível dos pacotes TCP.

8.5. Captura do Tráfego de Rede

Para obter o ficheiro de captura utilizado neste capítulo, foi utilizado o Wireshark com o filtro de captura **port 80**, limitando a captura ao tráfego HTTP. Durante a captura, foi utilizado um repositório público no GitHub (<https://github.com/PrettyBoyCosmo/HTTP-List>) que agrega listas de sites com URL HTTP (sem HTTPS), permitindo gerar tráfego de rede suficiente para uma análise representativa. A captura foi guardada no ficheiro **captura.pcap**, na pasta raiz do projeto.

Tabela 65 – Passos para captura de tráfego HTTP com o Wireshark

Passo	Ação
1	Abrir Wireshark como administrador e seleccionar a interface de rede ativa (Wi-Fi ou Ethernet)
2	Inserir o filtro de captura: port 80
3	Iniciar a captura (botão ■) e navegar em sites HTTP durante 1 a 2 minutos
4	Parar a captura (botão ■) e guardar: File → Save As → captura.pcap

8.6. Pipeline de Análise

O notebook Python desenvolvido para esta componente implementa um pipeline completo de análise, desde a leitura do ficheiro .pcap até à classificação de cada URL com o modelo Random Forest. O pipeline segue os mesmos princípios da aplicação Flask: Extração das 27 *features* lexicais e classificação binária (BENIGNO/MALICIOSO) com probabilidades associadas.

Tabela 66 – Pipeline de análise do notebook Wireshark

Etapa	Módulo	Descrição
1 — Carregamento do modelo	<code>joblib.load()</code>	Carrega o <code>modelo_rf_final.pkl</code> (500 árvores, 27 <i>features</i>)
2 — Leitura do .pcap	<code>scapy.rdpcap()</code>	Lê todos os pacotes do ficheiro de captura
3 — Extração de URL	Parser TCP/HTTP	Filtra pacotes TCP com payload HTTP (GET, POST, HEAD) e reconstrói o URL completo a partir do host e URI
4 — Remoção de duplicados	<code>dict.fromkeys()</code>	Elimina URL repetidos, mantendo a ordem de ocorrência
5 — Extração de <i>features</i>	<code>extrair_features()</code>	Extrai as 27 <i>features</i> lexicais de cada URL único
6 — Classificação	<code>modelo.predict()</code>	Classifica cada URL e obtém as probabilidades para cada classe
7 — Análise de resultados	pandas / Counter	Agrega estatísticas: totais, percentagens, top domínios, confiança média

A extração de URL a partir dos pacotes TCP é feita diretamente ao nível do payload: para cada pacote com camada Raw, o conteúdo é decodificado em UTF-8 e verificado se começa com um método HTTP (GET, POST ou HEAD). Em caso afirmativo, o URI é extraído da primeira linha do pedido e o cabeçalho Host é lido das linhas seguintes, reconstituindo o URL completo no formato <http://host/uri>.

8.7. Resultados Obtidos

A análise da captura de tráfego de rede produzida durante os testes resultou nos valores apresentados na Tabela 67. Do total de 11.158 pacotes capturados, foram identificados 203 pedidos HTTP, dos quais 198 eram URL únicos após remoção de duplicados.

Tabela 67 – Resultados de análise de tráfego de rede

Métrica	Valor
Total de pacotes capturados	11.158
Pedidos HTTP identificados	203
URL únicos (sem duplicados)	198

URL classificados como Benignos	151 (76,3%)
URL classificados como Maliciosos	47 (23,7%)
Confiança média geral	61,46%

A Tabela 68 apresenta os dez domínios com maior número de pedidos HTTP registados na captura, o que permite identificar as principais origens do tráfego gerado durante o teste.

Tabela 68 – Top 10 domínios mais frequentes na captura de tráfego

Domínio	Pedidos HTTP
images.china.cn	119
vas.vtiger.in	32
sneaindia.com	25
www.faqs.org	8
streamhd4k.com	3
ww38.bobmovies.us	2
www.china.com.cn	2
www.360doc.com	2
128.199.120.34	2
xinhuanet.com	1

A presença de um endereço IP direto (128.199.120.34) no tráfego capturado é um indicador relevante: URL que utilizam endereços IP em vez de nomes de domínio são frequentemente associados a infraestruturas de *malware* ou *phishing*, uma vez que evitam o registo de domínios rastreáveis. Esta característica está contemplada na *feature lex_tem_ip* das 27 *features* lexicais do modelo.

8.8. Limitações Identificadas

A integração com o Wireshark apresenta um conjunto de limitações técnicas que decorrem sobretudo da natureza passiva e offline da abordagem adotada. A Tabela 69 documenta cada limitação identificada, o seu impacto no funcionamento do sistema e as possíveis soluções a considerar em trabalho futuro.

Tabela 69 – Limitações na integração Wireshark e possíveis soluções

Limitação	Descrição	Impacto	Possível solução
HTTPS não analisável	O TLS encripta o conteúdo dos pacotes HTTPS, impossibilitando a extração de URL	Maioria do tráfego moderno não é analisável	Proxy TLS com inspeção SSL (mitmproxy, firewall empresarial)
Análise offline	O pipeline analisa ficheiros .pcap já capturados, não tráfego em tempo real	Não permite alertas imediatos	Monitor em tempo real com <i>scapy</i> (Capítulo 9 — trabalho futuro)
Dependência do Wireshark	Requer instalação do Wireshark e Npcap no sistema para captura	ConFiguração adicional necessária	Alternativa: tcpdump em Linux/macOS
Confiança média baixa	Confiança média de 61,46% indica incerteza do modelo em muitos URL HTTP	Mais falsos positivos/negativos	Enriquecer <i>dataset</i> de treino com mais exemplos HTTP

8.9. Resumo do Capítulo

Este capítulo descreveu a integração do Wireshark como segunda camada de análise do sistema, complementando a extensão de browser com uma perspetiva ao nível da rede. O pipeline desenvolvido, desde a captura do tráfego HTTP com o Wireshark até à classificação dos URL extraídos com o modelo Random Forest, demonstrou ser funcional e reproduzível, permitindo auditar o tráfego gerado por qualquer aplicação instalada no sistema, independentemente do browser utilizado.

A análise da captura de teste, com 198 URL únicos identificados a partir de 11.158 pacotes capturados, confirmou a aplicabilidade da abordagem, incluindo a deteção de um endereço IP direto no tráfego. A limitação principal desta componente (a impossibilidade de analisar tráfego HTTPS encriptado) é assumida explicitamente e partilhada com a generalidade das soluções de análise passiva de rede. O Capítulo 9 aborda esta limitação introduzindo um monitor de rede em tempo real, que substitui a análise retrospectiva de ficheiros .pcap pela captura e classificação imediata de pacotes HTTP ao vivo.

9. Monitor de Rede em Tempo Real

Com o modelo de Machine Learning, a aplicação Flask, a extensão de browser e a análise estática de capturas Wireshark concluídos nos capítulos anteriores, procedeu-se ao desenvolvimento de um monitor de rede em tempo real (a quarta e mais imediata camada de deteção do sistema). Enquanto o Capítulo 8 analisava ficheiros .pcap já gravados de forma retrospectiva, este componente captura e classifica o tráfego HTTP ao vivo, sem qualquer ficheiro intermédio, e propaga alertas instantâneos tanto para a aplicação Flask como para a extensão de browser.

9.1. Motivação e Objetivos

A análise Wireshark implementada no Capítulo 8 constitui uma ferramenta poderosa para investigação forense de tráfego já capturado, mas opera necessariamente de forma retrospectiva: o ficheiro .pcap tem de ser capturado, guardado e depois analisado. Esta abordagem é inadequada para deteção em tempo real, onde o objetivo é identificar e alertar para URL maliciosos quando o tráfego ocorre, antes que a comunicação maliciosa se conclua ou cause dano.

O monitor em tempo real resolve esta limitação substituindo a leitura de ficheiro pela captura direta de pacotes de rede através da biblioteca *scapy*. Cada pacote HTTP é processado imediatamente à medida que passa pela placa de rede, sem ser armazenado em disco. Quando um URL malicioso é detetado, um alerta é propagado instantaneamente para todos os clientes ligados através de Server-Sent Events (SSE).

Tabela 70 – Objetivos do monitor de rede em tempo real

Objetivo	Descrição
Captura em tempo real	Capturar pacotes TCP na porta 80 sem ficheiro intermédio usando <i>scapy.sniff()</i>
Classificação imediata	Extrair 27 <i>features</i> lexicais e classificar com o modelo Random Forest para cada URL capturado
Alertas SSE	Propagar alertas de URL maliciosos em tempo real via Server-Sent Events para todos os clientes
Integração na app Flask	Painel de monitor em tempo real na interface web, com contadores e lista de alertas
Integração na extensão	Painel "Alertas de Rede" no <i>popup</i> da extensão URL Shield, com destaque visual na página
Arranque automático	O monitor inicia automaticamente com o Flask — basta executar <code>python app.py</code>

Saída no terminal	Registro colorido de todos os URL detetados (BENIGNO a verde, MALICIOSO a vermelho) com probabilidade e timestamp
-------------------	---

9.2. Arquitetura do Sistema

O monitor é composto por três camadas que comunicam entre si de forma assíncrona: a camada de captura (*scapy*), a camada de distribuição (SSE no Flask) e a camada de apresentação (aplicação web e extensão de browser). A Figura 24 ilustra o fluxo completo desde a captura do pacote até à apresentação do alerta ao utilizador, evidenciando a separação entre as três camadas do sistema e os canais de comunicação entre elas.

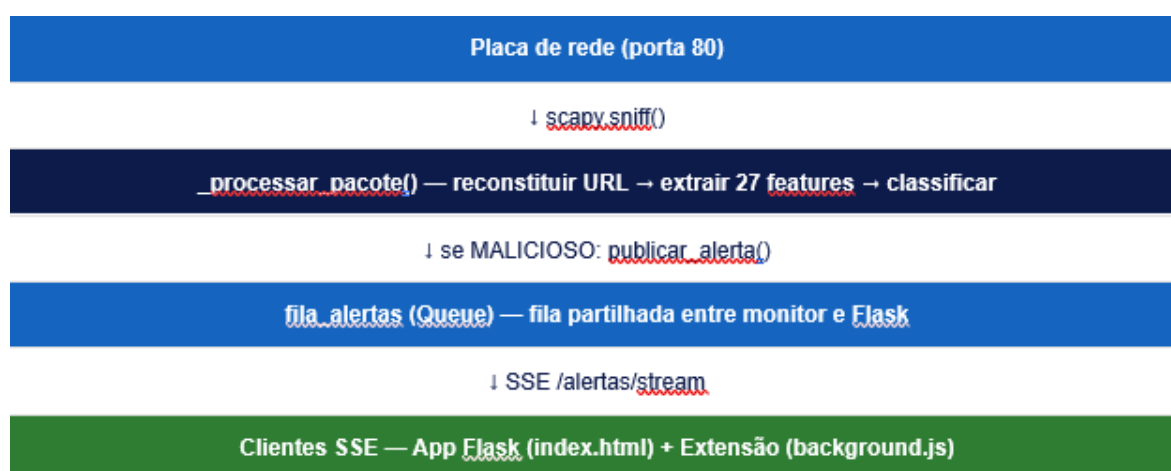


Figura 24 – Arquitetura do monitor de rede em tempo real

A Tabela 71 descreve cada um dos componentes que constituem o monitor de rede, identificando a tecnologia utilizada e a função que desempenha no fluxo de processamento desde a captura do pacote até à apresentação do alerta ao utilizador.

Tabela 71 – Componentes do monitor de rede em tempo real

Componente	Tecnologia	Função
Monitor <i>scapy</i>	<i>scapy</i> + threading	Captura pacotes TCP porta 80, reconstrói URL HTTP, classifica e publica alertas
Fila de alertas	queue.Queue (Python)	Estrutura thread-safe que transporta alertas do monitor para os subscritores SSE
SSE /alertas/stream	Flask + Response SSE	Mantém ligação HTTP aberta com cada cliente e empurra alertas assim que chegam à fila
SSE /alertas/injetar	Flask POST	Endpoint auxiliar para injetar alertas manualmente, útil para testes sem tráfego real

Monitor na app web	JavaScript EventSource	A página Flask abre ligação SSE e exibe alertas em tempo real com contadores e lista
Monitor na extensão	background.js EventSource	O service worker abre ligação SSE e retransmite alertas para todos os <i>content</i> scripts ativos

9.3. Implementação técnica

9.3.1. Integração no app.py – Arranque Automático

O monitor é iniciado automaticamente quando o Flask arranca, através de uma thread daemon lançada pela função `arrancar_monitor()`. Por ser uma thread daemon, termina automaticamente quando o processo principal do Flask termina. Não é necessário nenhum processo separado ou gestão manual do ciclo de vida.

Tabela 72 -Fluxo de processamento do monitor *scapy*

Passo	Ação	Responsável
1	Flask inicia — <code>arrancar_monitor()</code> é chamado em <code>__main__</code>	app.py
2	Thread daemon <code>MonitorScapy</code> é lançada com <code>target=_iniciar_monitor</code>	<code>threading.Thread</code>
3	<code>scapy.sniff()</code> começa a capturar pacotes TCP na porta 80 da placa de rede	<i>scapy</i>
4	Para cada pacote: <code>_processar_pacote()</code> verifica se é pedido HTTP (GET/POST/HEAD)	<code>_processar_pacote()</code>
5	Reconstrói URL a partir dos campos Host e <i>Path</i> do cabeçalho HTTP	<code>_processar_pacote()</code>
6	Verifica se URL já foi visto (set thread-safe); se sim, descarta	<code>_URL_vistos_monitor</code>
7	Extrai 27 <i>features</i> lexicais com <code>extrair_features()</code> e classifica com o modelo	modelo Random Forest
8	Imprime resultado no terminal (verde=benigno, vermelho=malicioso)	<code>_processar_pacote()</code>
9	Se MALICIOSO: <code>publicar_alerta()</code> coloca alerta na fila de cada subscritor SSE ativo	<code>publicar_alerta()</code>

9.3.2. Server-Sent Events (SSE)

A tecnologia escolhida para propagação de alertas em tempo real foi Server-Sent Events (SSE), uma norma W3C que permite ao servidor empurrar dados para o cliente através de uma ligação HTTP unidirecional mantida aberta. O SSE foi

preferido a WebSockets pela sua simplicidade de implementação: não requer biblioteca adicional, é suportado nativamente pelo browser através da API EventSource, e é suficiente para o caso de uso de alertas unidirecionais (servidor → cliente).

Tabela 73 – Comparação entre SSE e WebSockets

Aspeto	SSE (escolhido)	WebSockets
Direção	Unidirecional (servidor → cliente)	Bidirecional
Biblioteca	Nenhuma (nativo no browser)	Requer biblioteca adicional
Reconexão	Automática pelo browser	Manual
Protocolo	HTTP/1.1 standard	Upgrade para WS
Adequação	✓ Ideal para alertas unidirecionais	Desnecessário para este caso

O endpoint `/alertas/stream` utiliza um gerador Python com `stream_with_context()` do Flask para manter a ligação aberta. Cada cliente que se liga recebe uma fila individual (`queue.Queue`), que é adicionada à lista global de subscritores. Quando o monitor deteta um URL malicioso, `publicar_alerta()` coloca o alerta em todas as filas ativas. Um heartbeat é enviado a cada 25 segundos para manter a ligação viva em proxies e firewalls que fecham ligações inativas.

9.3.3. Integração na Extensão de Browser

A integração SSE na extensão apresentou uma limitação técnica adicional relativamente à aplicação web: os *content* scripts correm no contexto das páginas e estão sujeitos às mesmas restrições de Private Network Access que bloqueiam pedidos de páginas HTTPS para localhost. A solução consistiu em mover a ligação EventSource para o background service worker, que corre num contexto isolado do Chrome sem estas restrições.

Tabela 74 – Fluxo de propagação de alertas SSE na extensão de browser

Passo	Ação	Ficheiro
1	background.js abre EventSource para <code>/alertas/stream</code> ao iniciar	background.js
2	Flask confirma ligação com evento <code>{"tipo": "ligado"}</code>	app.py
3	Monitor deteta URL malicioso → <code>publicar_alerta()</code> → SSE envia evento <code>{"tipo": "alerta"}</code>	app.py
4	background.js recebe o alerta e itera sobre todos os tabs Chrome ativos	background.js

5	Para cada tab: <code>chrome.tabs.sendMessage()</code> com mensagem <code>ALERTA_REDE</code>	<code>background.js</code>
6	<code>content.js</code> recebe <code>ALERTA_REDE</code> : destaca links com domínio coincidente na página	<code>content.js</code>
7	<code>content.js</code> reencaminha <code>ALERTA_REDE_POPUP</code> para o <code>popup</code> aberto (se existir)	<code>content.js</code> → <code>popup.js</code>
8	<code>popup.js</code> exibe o alerta no painel "Alertas de Rede" com URL, probabilidade e hora	<code>popup.js</code>
9	Se Flask reiniciar: <code>background.js</code> reconecta automaticamente após 10 segundos	<code>background.js</code>

9.4. Saída no Terminal

Para além dos alertas SSE enviados aos clientes, o monitor imprime no terminal uma linha por cada URL HTTP capturado, com código de cor ANSI: verde para URL benignos e vermelho para maliciosos, conforme ilustrado na Figura 25. Esta saída permite monitorizar o tráfego em tempo real sem necessidade de abrir o browser ou a aplicação web.

```
[11:17:47] BENIGNO 24.4% http://c.pki.goog/r/r1.crl
[11:17:47] BENIGNO 14.8% http://x1.c.lencr.org/
[11:17:48] BENIGNO 35.3% http://crl3.digicert.com/DigiCertTrustedG4RSA4096SHA256TimeStampingCA.crl
[11:17:49] BENIGNO 14.8% http://x2.c.lencr.org/
[11:17:49] BENIGNO 10.2% http://ye.c.lencr.org/
[11:17:50] BENIGNO 23.4% http://crl3.digicert.com/DigiCertGlobalRootG2.crl
127.0.0.1 - - [03/Jun/2026 11:17:50] "GET /alertas/stream HTTP/1.1" 200 -
[11:17:51] BENIGNO 20.9% http://c.pki.goog/r/gsr1.crl
127.0.0.1 - - [03/Jun/2026 11:17:53] "POST /classificar HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 11:17:53] "POST /classificar HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 11:17:53] "POST /classificar HTTP/1.1" 200 -
[11:17:53] BENIGNO 24.4% http://c.pki.goog/r/r4.crl
[11:17:54] BENIGNO 9.7% http://yr.c.lencr.org/
[11:17:55] MALICIOSO 71.4% http://ctldl.windowsupdate.com/msdownload/update/v3/static/trustedr/en/authrootstl.cab?12f10a0582b91eba
[11:17:55] MALICIOSO 71.4% http://ctldl.windowsupdate.com/msdownload/update/v3/static/trustedr/en/disallowedcertstl.cab?5a75adfc1b5f3aee
```

Figura 25 – Saída do terminal durante a execução do monitor

9.5. Interface de Utilizador

O monitor de rede em tempo real está integrado em duas interfaces distintas: o painel da aplicação Flask, acessível via browser, e o painel da extensão de browser URL Shield, acessível através do `popup`. Ambas as interfaces recebem os alertas SSE em tempo real e apresentam a informação de forma adaptada ao contexto de cada uma, conforme descrito nas secções seguintes.

9.5.1. Painel na Aplicação Flask

A aplicação web foi atualizada com um painel de monitor em tempo real, posicionado abaixo da secção de análise manual de URL. O painel apresenta três contadores (URL detetados, benignos e maliciosos), um indicador de estado da ligação SSE (ativo/desligado/a ligar) e uma lista de alertas maliciosos recebidos

com timestamp, URL completo e botão "✓ Benigno" para correção de falsos positivos. Os alertas são inseridos no topo da lista à medida que chegam, mantendo os mais recentes visíveis sem necessidade de scroll. A Figura 26 apresenta o aspeto do painel de monitor integrado na aplicação web, com os contadores de URL detetados, o indicador de estado da ligação SSE e a lista de alertas maliciosos recebidos.

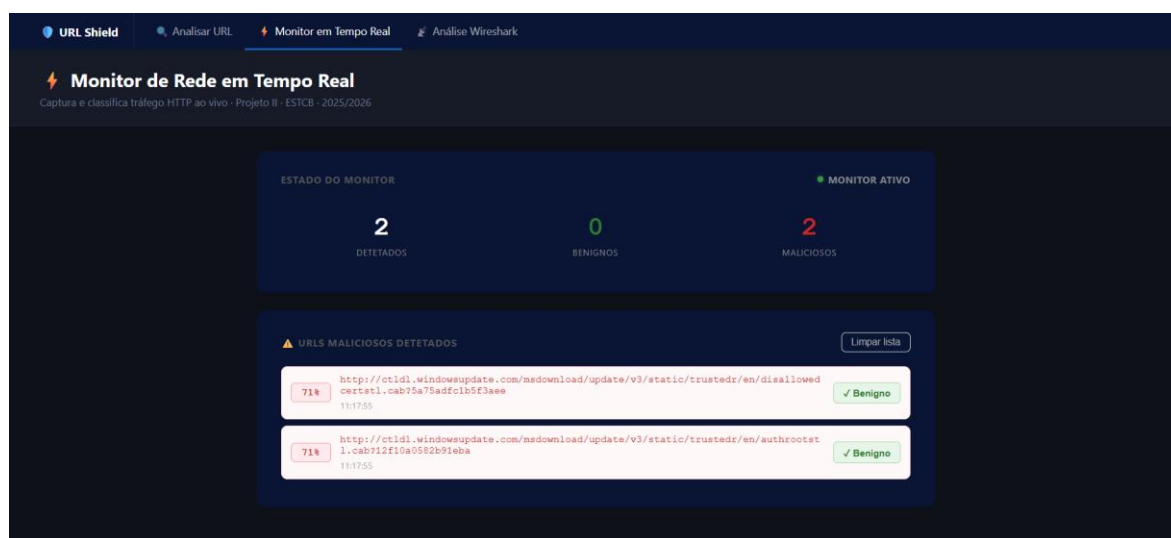


Figura 26 –Monitor de tráfego de rede em tempo real na Aplicação Flask

9.5.2. Painel na Extensão de Browser

O *popup* da extensão URL Shield foi atualizado com um painel "Alertas de Rede" que aparece automaticamente quando chega o primeiro alerta de rede. O painel inclui um contador de alertas, a lista dos 5 alertas mais recentes com probabilidade, URL e hora, e um botão para limpar a lista. Ao chegar um novo alerta, o painel pisca brevemente a amarelo para chamar a atenção do utilizador. O *content* script tenta ainda destacar a vermelho na página atual quaisquer hiperligações cujo domínio coincida com o URL do alerta.

Tabela 75 – Comparação entre o painel de monitor na aplicação Flask e na extensão

Componente	App Flask	Extensão (<i>popup</i>)
Ligação SSE	EventSource direto (/alertas/stream)	Via background.js (sem restrições CSP)
Contadores	Detetados / Benignos / Maliciosos	Total de alertas recebidos
Lista de alertas	Todos os alertas maliciosos, mais recente no topo	Últimos 5 alertas maliciosos
Destaque visual	Não aplicável	Links com domínio coincidente destacados a vermelho

Botão Benigno	Chama diretamente /guardar_feedback	Via chrome.runtime.sendMessage ao background
Animação	Entrada suave (fadeIn + translateY)	Painel pisca a amarelo ao receber novo alerta
Reconexão	Automática após 10 s se Flask reiniciar	Automática após 10 s se Flask reiniciar

9.6. Pré-requisitos e Instalação

O monitor *scapy* requer dois pré-requisitos além das dependências já instaladas nas fases anteriores: a biblioteca Python *scapy* e o driver de captura de pacotes Npcap para Windows. O Npcap é instalado automaticamente com o Wireshark (se o Capítulo 8 foi executado com sucesso, o Npcap já está presente no sistema).

Tabela 76 - Pré-requisitos do monitor *scapy* em Windows

Pré-requisito	Como instalar	Observação
<i>scapy</i>	pip install <i>scapy</i>	Biblioteca Python de captura e manipulação de pacotes de rede
Npcap	https://npcap.com/ ou instalado com Wireshark	Driver de captura de pacotes para Windows; equivalente ao WinPcap moderno
Privilégios de administrador	Executar terminal como Administrador	O <i>scapy</i> requer acesso à placa de rede, que exige privilégios elevados no Windows

O arranque do monitor pode resultar em diferentes estados consoante as condições do sistema. A Tabela 77 descreve as mensagens apresentadas no terminal em cada situação e o impacto que cada uma tem no funcionamento da aplicação, clarificando que a ausência do monitor não impede o funcionamento normal do Flask nem das restantes funcionalidades da aplicação.

Tabela 77 – Mensagens de arranque do monitor e respetivo impacto

Situação	Mensagem no terminal	Impacto
<i>scapy</i> não instalado	[Monitor] X <i>Scapy</i> não instalado. Execute: pip install <i>scapy</i>	Flask funciona; monitor inativo
Sem privilégios de administrador	[Monitor] X Sem permissões para capturar tráfego.	Flask funciona; monitor inativo

Npcap não instalado	[Monitor] X Npcap não encontrado. Instale em: https://npcap.com/	Flask funciona; monitor inativo
Tudo correto	[Monitor] ✓ <i>Scapy</i> carregado — a monitorizar tráfego HTTP (porta 80)...	Monitor ativo e a capturar

9.7. Testes e Validação

O monitor foi testado em condições de uso real, com o Flask a correr como administrador e tráfego HTTP gerado pelo sistema operativo e aplicações instaladas. Para além do tráfego real, o endpoint `/alertas/injetar` permitiu testar o pipeline SSE completo sem necessidade de tráfego HTTP (útil para validar a integração com a extensão e a aplicação web de forma controlada).

Tabela 78 – Resultados dos testes funcionais do monitor de rede em tempo real

Cenário de teste	Resultado esperado	Resultado obtido	✓
Arranque automático com python app.py	Monitor inicia sem intervenção manual	Thread <i>MonitorScapy</i> lançada; mensagem de confirmação no terminal	✓
Tráfego HTTP gerado (navegação, atualizações Windows)	URL capturados e classificados no terminal	URL classificados com label e probabilidade; benignos a verde, maliciosos a vermelho	✓
Injeção manual via <code>/alertas/injetar</code>	Alerta aparece na app Flask e extensão	Alerta exibido imediatamente no painel da app e no <i>popup</i> da extensão	✓
URL malicioso detetado em tráfego real	Alerta SSE propagado a todos os clientes	Alerta recebido na app Flask e no <i>popup</i> da extensão em menos de 1 segundo	✓
Destaque visual de link na página (extensão)	Link com domínio coincidente destacado a vermelho	Links com domínio correspondente ao alerta destacados a vermelho na página atual	✓
Flask reiniciado com SSE ativo	Reconexão automática após reinício	<code>background.js</code> e <code>index.html</code> reconectam automaticamente após 10 segundos	✓
<i>scapy</i> não instalado	Flask funciona sem monitor	Mensagem de erro explicativa no terminal; aplicação e extensão funcionam normalmente	✓

Mesmo URL capturado múltiplas vezes	Apenas um alerta gerado	Set de URL vistos evita duplicados; apenas um alerta por URL único	✓
-------------------------------------	-------------------------	--	---

9.8. Limitações Identificadas

O monitor de rede em tempo real apresenta um conjunto de limitações técnicas que decorrem essencialmente dos requisitos de captura de pacotes ao nível do sistema operativo e da restrição ao protocolo HTTP. A Tabela 79 documenta cada limitação identificada durante o desenvolvimento e os testes, o seu impacto no funcionamento do sistema e as possíveis soluções a considerar em trabalho futuro.

Tabela 79 – Limitações do monitor de rede em tempo real e possíveis soluções

Limitação	Descrição	Impacto	Possível solução
Apenas tráfego HTTP (porta 80)	O HTTPS encripta o payload TLS — os URL não são visíveis ao <i>scapy</i>	Cobertura limitada — a maioria do tráfego moderno é HTTPS	Proxy TLS com inspeção SSL (mitmproxy) em contexto empresarial
Requer privilégios de administrador	O <i>scapy</i> necessita de acesso direto à placa de rede no Windows	Não funciona sem reiniciar o terminal como administrador	Criar serviço Windows com privilégios próprios
Dependência do Npcap	O Npcap tem de estar instalado no sistema (instalado com Wireshark)	Falha silenciosa se Npcap não estiver presente	Verificar presença do Npcap no arranque e instruir instalação
Falsos positivos do modelo	O modelo classifica como maliciosos alguns URL legítimos de sistemas Windows (ctldl.windo wsupdate.com)	Alertas incorretos que degradam a experiência	Lista branca de domínios de sistema; re-treino com feedback
Reconexão SSE com perda de alertas	Se o Flask reiniciar durante uma captura, os alertas gerados antes da reconexão são perdidos	Lacunas nos alertas durante reinícios	Persistir alertas entre páginas via <code>sessionStorage</code> — implementado no Capítulo 10 através do mecanismo de persistência do monitor

9.9. Conclusão de Capítulo

Este capítulo descreveu o desenvolvimento do monitor de rede em tempo real, a quarta camada de detecção do sistema. Ao substituir a leitura retrospectiva de ficheiros `.pcap` pela captura direta de pacotes HTTP através da biblioteca *scapy*, este componente elimina o intervalo entre a ocorrência do tráfego malicioso e a sua detecção, propagando alertas instantâneos via Server-Sent Events para todos os clientes ligados (tanto a aplicação Flask como a extensão de browser).

A integração do monitor no arranque automático do Flask, sem necessidade de qualquer configuração adicional, e a sua degradação elegante na ausência do *scapy* ou dos privilégios necessários, garantem que a aplicação funciona corretamente em qualquer ambiente. Os testes realizados confirmaram o correto funcionamento de todos os cenários previstos, incluindo a propagação de alertas em menos de um segundo e a reconexão automática após reinício do Flask. A limitação principal desta componente (a restrição ao tráfego HTTP na porta 80, partilhada com o Capítulo 8) mantém-se como a oportunidade de melhoria mais relevante para trabalho futuro. O Capítulo 10 descreve a reestruturação da interface web em três páginas independentes e o desenvolvimento do dashboard de análise Wireshark, que integra numa única aplicação todas as modalidades de análise desenvolvidas ao longo do projeto.

10. Dashboard de Tráfego – Aplicação Web

Com o modelo de Machine Learning, a aplicação Flask, a extensão de browser e o monitor de rede em tempo real concluídos nos capítulos anteriores, procedeu-se à reestruturação da interface web em três secções independentes e ao desenvolvimento de um dashboard completo para análise de capturas Wireshark. Esta fase transforma a aplicação Flask de uma ferramenta de classificação pontual num sistema integrado de cibersegurança com múltiplas modalidades de análise acessíveis através de uma navegação unificada.

10.1. Motivação e Objetivos

A aplicação Flask desenvolvida nos capítulos anteriores concentrava todas as funcionalidades numa única página, o que criava uma experiência de utilizador confusa à medida que o sistema crescia. A análise manual de URL, o monitor de tráfego em tempo real e a análise de capturas Wireshark são casos de uso distintos, com audiências e contextos de utilização diferentes, e beneficiam de páginas dedicadas com foco visual e funcional próprio.

Adicionalmente, a análise de ficheiros .pcap estava confinada ao ambiente Jupyter Notebook do Capítulo 8, exigindo conhecimentos técnicos de Python para ser utilizada. O dashboard Wireshark resolve esta limitação expondo a mesma capacidade analítica através de uma interface web acessível a qualquer utilizador, sem necessidade de instalar ou configurar qualquer ferramenta adicional.

Tabela 80 – Objetivos do dashboard de tráfego

Objetivo	Descrição
Separação de páginas	Dividir a aplicação em três páginas independentes com navegação unificada, melhorando a clareza e foco de cada secção
Dashboard Wireshark	Permitir upload de ficheiros .pcap diretamente na interface web e visualizar os resultados classificados sem recurso a código Python
Tabela interativa	Filtrar por resultado (benigno/malicioso), pesquisar por URL ou domínio, e ordenar qualquer coluna por clique no cabeçalho
Visualização gráfica	Gráfico donut com distribuição benigno/malicioso e gráfico de barras horizontais com os top 10 domínios maliciosos detetados
Exportação CSV	Descarregar os resultados da análise (com o filtro ativo) em ficheiro CSV compatível com Excel e outras ferramentas de análise

Histórico de análises	Registrar automaticamente cada análise .pcap realizada com nome do ficheiro, data, total de URL e contagem de maliciosos
Persistência do monitor	Os alertas do monitor de rede persistem na sessão do browser ao navegar entre páginas, sem necessidade de reconexão ou reload

10.2. Reestruturação da Aplicação – Três Páginas Independentes

A aplicação foi reestruturada em três rotas Flask distintas, cada uma servindo uma página HTML independente com cabeçalho, estilo e funcionalidade próprios. Uma barra de navegação comum ao topo de todas as páginas permite transitar entre secções com um único clique, mantendo sempre visível a secção ativa através de um indicador visual (sublinhado colorido). A Tabela 81 apresenta as três rotas Flask criadas, a página correspondente a cada uma e o ficheiro de template HTML utilizado para a sua renderização.

Tabela 81 – Rotas e páginas da aplicação Flask reestruturada

Rota	Página	Ficheiro template
GET /	Analisar URL	index.html
GET /monitor	Monitor em Tempo Real	monitor.html
GET /wireshark	Análise Wireshark	wireshark.html

10.3. Dashboard de Análise Wireshark

O dashboard de análise Wireshark é composto por quatro componentes principais que cobrem todo o ciclo de análise de uma captura de tráfego: o upload do ficheiro .pcap, o processamento e classificação dos URL no backend, a visualização gráfica dos resultados e a exportação dos dados. As secções seguintes descrevem cada um destes componentes em detalhe.

10.3.1. Upload de Ficheiro .pcap

A página de análise Wireshark apresenta uma zona de upload com suporte a dois modos de seleção de ficheiro: clique para abrir o seletor de ficheiros do sistema operativo, ou arrastar e largar o ficheiro diretamente na zona demarcada. Apenas ficheiros com extensão .pcap são aceites. Qualquer outro formato é rejeitado com uma mensagem de erro. Após a seleção, o nome do ficheiro é exibido e o botão "Analisar captura" torna-se visível. A Figura 27 ilustra o aspeto da página de análise Wireshark, apresentando a zona de upload com suporte a clique e arrastar e largar, o botão de análise e o histórico das análises anteriores.

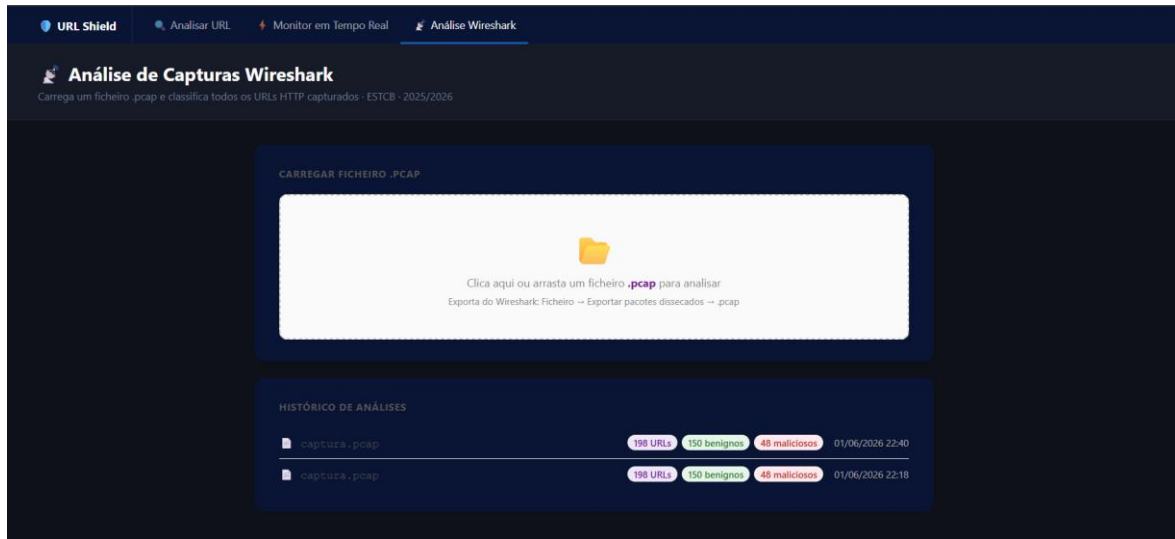


Figura 27 –Página daAplicação Flask para análise de capturas .pcap

10.3.2. Processamento no Backend - /analisar_pcap

O endpoint `/analisar_pcap` recebe o ficheiro, guarda-o temporariamente na pasta `uploads_tmp/`, lê-o com `scapy.rdpcap()` e itera sobre todos os pacotes. Para cada pacote TCP com payload Raw, verifica se o conteúdo começa com um método HTTP válido (GET, POST ou HEAD), extrai o Host e o URI do cabeçalho HTTP, reconstrói o URL completo e classifica-o com o modelo Random Forest. URL duplicados são descartados através de um set de URL já vistos.

Tabela 82 – Fluxo de processamento do endpoint `/analisar_pcap`

Passo	Ação	Detalhe
1	Receber ficheiro	Validar extensão .pcap; guardar em <code>uploads_tmp/</code> temporariamente
2	Ler pacotes	<code>scapy.rdpcap()</code> carrega todos os pacotes do ficheiro .pcap
3	Filtrar pacotes HTTP	Verificar presença de camada TCP + Raw; extrair método HTTP da primeira linha do payload
4	Extrair URL	Combinar campo Host com URI para reconstituir o URL completo (<code>http://host/uri</code>)
5	Deduplica URL	Set de URL vistos — cada URL único é classificado apenas uma vez
6	Classificar	<code>extrair_features()</code> → 27 <i>features</i> lexicais → <code>modelo.predict_proba()</code> → label + probabilidades
7	Limpar ficheiro	<code>os.remove()</code> apaga o .pcap temporário após processamento

8	Guardar histórico	guardar_historico() regista nome, data, total, benignos e maliciosos em historico_pcap.json
9	Devolver resultado	JSON com lista de resultados, totais e contadores para renderização na página

10.3.3. Sumário e Visualização Gráfica

Após a análise, a página apresenta quatro métricas em destaque: total de URL únicos identificados, número de benignos, número de maliciosos e confiança média do modelo em todas as classificações. Abaixo das métricas, dois gráficos gerados com Chart.js oferecem uma visão visual imediata dos resultados.

Tabela 83 – Gráficos de visualização dos resultados da análise .pcap

Gráfico	Tipo	Conteúdo
Distribuição de resultados	Donut	Proporção entre URL benignos (verde) e maliciosos (vermelho)
Top domínios maliciosos	Barras horizontais	Os 10 domínios com maior número de URL classificados como maliciosos, ordenados por contagem

10.3.4. Tabela Interativa de Resultados

A tabela de resultados apresenta todos os URL classificados com cinco colunas: resultado (badge colorido BENIGNO/MALICIOSO), probabilidade de ser malicioso (com barra visual), método HTTP, domínio e URL completo. A tabela suporta três modos de interação: filtragem por categoria, pesquisa livre e ordenação por coluna.

Tabela 84 – Funcionalidades interativas da tabela de resultados

Funcionalidade	Descrição
Filtro por categoria	Botões "Todos", "Maliciosos" e "Benignos" filtram a tabela instantaneamente sem recarregar a página
Pesquisa livre	Campo de texto filtra por URL ou domínio em tempo real à medida que o utilizador escreve
Ordenação por coluna	Clique no cabeçalho de qualquer coluna ordena ascendente; segundo clique inverte para descendente
Paginação	Resultados divididos em páginas de 20 entradas com botões de navegação numérica
Exportação CSV	Botão "■ Exportar CSV" descarrega os resultados atualmente visíveis (filtro aplicado) em ficheiro .csv

10.3.5. Exportação para CSV

O botão "■ Exportar CSV" gera e descarrega automaticamente um ficheiro CSV com os resultados atualmente visíveis na tabela, ou seja, com o filtro e a pesquisa aplicados no momento da exportação. O ficheiro inclui seis colunas: Resultado, Probabilidade de ser Malicioso (%), Probabilidade de ser Benigno (%), Método HTTP, Domínio e URL. O ficheiro é gerado inteiramente no browser com a API Blob, sem qualquer pedido ao servidor, e inclui um BOM UTF-8 para garantir compatibilidade com o Microsoft Excel em versões portuguesas.

10.3.6. Histórico de Análises

Cada análise de ficheiro .pcap é automaticamente registada no ficheiro `historico_pcap.json`, persistido na pasta Aplicação/ juntamente com o modelo e o ficheiro de feedback. O histórico é carregado pelo Flask ao servir a página `/wireshark` e renderizado com Jinja2, garantindo que o histórico está disponível mesmo após reiniciar o servidor. São mantidas as últimas 20 análises; entradas mais antigas são automaticamente descartadas.

10.4. Persistência do Monitor entre Páginas

Uma limitação identificada durante os testes do monitor de rede foi a perda dos alertas ao navegar para outra página e regressar. Como os dados viviam exclusivamente em variáveis JavaScript, eram descartados quando o browser descarregava a página. A solução implementada recorre ao `sessionStorage` do browser, uma área de armazenamento de sessão disponível nativamente em todos os browsers modernos.

Tabela 85 - Mecanismo de persistência dos alertas do monitor de rede

Mecanismo	Comportamento
Guardar estado	Sempre que um novo alerta é recebido ou a lista é limpa, o estado completo (alertas, contadores) é serializado em JSON e guardado em <code>sessionStorage</code> com a chave <code>URLhield_alertas_monitor</code>
Carregar estado	Ao entrar na página <code>/monitor</code> , o JavaScript lê o <code>sessionStorage</code> e reconstrói a lista de alertas e os contadores antes de iniciar a ligação SSE
Botão "✓ Benigno"	O estado "guardado" de cada alerta é também persistido, evitando que o botão volte ao estado inicial ao regressar à página
Limpar lista	O botão "Limpar lista" apaga os dados do <code>sessionStorage</code> além de limpar a interface — os alertas não reaparecem ao regressar
Fim de sessão	O <code>sessionStorage</code> é automaticamente limpo quando o utilizador fecha o separador ou o browser — comportamento esperado e correto

10.5. Novos Endpoints do Flask

A reestruturação da aplicação em três páginas independentes e a integração do dashboard Wireshark implicaram a adição de novos endpoints ao ficheiro `app.py`, complementando os já existentes nas fases anteriores do projeto. A Tabela 86 descreve cada um dos novos endpoints adicionados, o método HTTP utilizado e a função que desempenha na aplicação.

Tabela 86 – Novos endpoints adicionados ao Flask

Endpoint	Método	Descrição
GET /monitor	GET	Serve a página <code>monitor.html</code> — monitor de rede em tempo real
GET /wireshark	GET	Serve a página <code>wireshark.html</code> com o histórico carregado via Jinja2
POST /analisar_pcap	POST	Recebe ficheiro <code>.pcap</code> (multipart), processa com <i>scapy</i> , classifica URL e devolve JSON com resultados
GET /historico_pcap	GET	Devolve o <code>historico_pcap.json</code> em formato JSON (endpoint auxiliar para consulta externa)

10.6. Testes e Validação

O dashboard de tráfego foi testado com um conjunto de cenários representativos das principais funcionalidades implementadas, cobrindo a navegação entre páginas, o upload e análise de ficheiros `.pcap`, as funcionalidades interativas da tabela de resultados e o mecanismo de persistência do monitor. A Tabela 87 resume os cenários testados, o resultado esperado, o resultado obtido e a confirmação do correto funcionamento de cada funcionalidade.

Tabela 87 – Resultados dos testes funcionais do dashboard de tráfego

Cenário de teste	Resultado esperado	Resultado obtido	✓
Navegação entre as três páginas	Página correta carregada; separador ativo destacado	Navegação funcional; indicador visual correto em todas as páginas	✓
Upload de ficheiro <code>.pcap</code> válido	Análise concluída com tabela e gráficos	Resultados exibidos com métricas, donut, barras e tabela paginada	✓
Upload de ficheiro com extensão errada	Mensagem de erro; análise não iniciada	Erro "Apenas ficheiros <code>.pcap</code> são aceites" exibido corretamente	✓

Arrastar e largar ficheiro .pcap	Ficheiro aceite e botão "Analisar" visível	Drag & drop funcional; zona muda de cor ao arrastar	✓
Filtrar tabela por "Maliciosos"	Apenas linhas MALICIOSO visíveis	Filtro aplicado instantaneamente; paginação recalculada	✓
Pesquisar URL na tabela	Resultados filtrados em tempo real	Pesquisa funcional por URL e por domínio	✓
Ordenar coluna "Prob. Malicioso"	Tabela ordenada descendente; segundo clique inverte	Ordenação correta; seta de direção implícita	✓
Exportar CSV sem filtro	CSV com todos os resultados descarregado	Ficheiro gerado com todas as linhas e BOM UTF-8; abre corretamente no Excel	✓
Exportar CSV com filtro "Maliciosos"	CSV com apenas os maliciosos	Ficheiro contém apenas as linhas visíveis no filtro ativo	✓
Navegar para outra página e regressar ao monitor	Alertas anteriores mantidos; contadores corretos	sessionStorage restaura corretamente todos os alertas e contadores	✓
Clicar "Limpar lista" e regressar ao monitor	Lista vazia ao regressar	sessionStorage limpo; monitor começa vazio após limpeza	✓
Histórico após análise .pcap	Entrada adicionada no histórico	Histórico atualizado em tempo real na página e persistido em historico_pcap.json	✓
Reiniciar Flask — histórico mantido	Histórico visível após reinício	historico_pcap.json relido pelo Flask; histórico disponível após reinício	✓

10.7. Resumo do Capítulo

Este capítulo descreveu a reestruturação da aplicação Flask numa interface unificada com três páginas independentes e o desenvolvimento do dashboard de análise de capturas Wireshark. A separação em páginas dedicadas (análise manual de URL, monitor de rede em tempo real e análise de ficheiros .pcap), clarifica o propósito de cada funcionalidade e melhora significativamente a experiência do utilizador face à versão inicial de página única.

O dashboard Wireshark democratiza o acesso à análise de tráfego de rede, expondo através de uma interface web acessível a qualquer utilizador a mesma capacidade analítica que anteriormente estava confinada ao ambiente Jupyter Notebook do Capítulo 8. As funcionalidades de filtragem, pesquisa, ordenação, paginação e exportação CSV, combinadas com a visualização gráfica dos resultados e o histórico de análises persistido entre sessões, tornam o dashboard numa ferramenta completa de auditoria de tráfego HTTP. O mecanismo de persistência do monitor via `sessionStorage` resolve ainda a limitação identificada no Capítulo 9, garantindo que os alertas de rede são preservados ao navegar entre as três páginas da aplicação.

Com este capítulo, o sistema de deteção de URL maliciosos desenvolvido ao longo deste projeto encontra-se completo, integrando numa única aplicação Flask quatro modalidades de análise complementares: a classificação manual de URL, a extensão de browser com análise automática de páginas, o monitor de rede em tempo real e o dashboard de análise de capturas Wireshark.

11. Conclusão

O presente relatório descreveu o desenvolvimento completo de um sistema de detecção de URL maliciosos baseado em Machine Learning, materializando os objetivos definidos no Projeto I e concretizando a transição da componente teórica para uma solução funcional e testada.

O pipeline de Machine Learning implementado percorreu todas as etapas definidas no Projeto I: o pré-processamento e limpeza do *dataset* selecionado, a normalização das variáveis e a extração de características lexicais, o treino e comparação de sete algoritmos de classificação, e a otimização de hiperparâmetros do modelo selecionado. O algoritmo Random Forest confirmou empiricamente a sua superioridade face aos restantes classificadores testados, alcançando **94,63%** de F1-Score e **98,79%** de AUC-ROC sem qualquer otimização, valores consistentes com os reportados na literatura científica analisada no Projeto I.

O processo iterativo de desenvolvimento da aplicação revelou uma limitação fundamental do modelo inicial (a dependência de *features* não calculáveis em tempo real) que foi resolvida através da expansão para 27 características exclusivamente lexicais e do enriquecimento cirúrgico do *dataset* de treino. O modelo final v3, com **94,65%** de Accuracy e **98,40%** de AUC-ROC, classifica corretamente URL do quotidiano que o modelo original falhava sistematicamente, mantendo simultaneamente a capacidade de detetar URL de *phishing* com palavras suspeitas.

O sistema desenvolvido integra quatro modalidades de análise complementares, cada uma cobrindo uma perspetiva distinta da detecção de URL maliciosos. A aplicação Flask disponibiliza a classificação manual de qualquer URL através de uma interface web acessível. A extensão de browser URL Shield integra a detecção diretamente no fluxo de navegação, analisando automaticamente todos os links presentes em qualquer página e incluindo um modo especializado para o Gmail. O monitor de rede em tempo real acrescenta uma camada de análise ao nível dos pacotes HTTP, propagando alertas instantâneos via Server-Sent Events. O dashboard Wireshark completa o sistema com a capacidade de auditar retrospectivamente capturas de tráfego de rede em formato .pcap, através de uma interface web com visualização gráfica, filtragem interativa e exportação CSV. Em conjunto, estas quatro componentes implementam uma arquitetura de defesa em profundidade, cobrindo desde a interação do utilizador com links individuais até à análise do tráfego gerado por qualquer aplicação no sistema.

O sistema desenvolvido apresenta um conjunto de limitações identificadas ao longo do relatório que constituem as oportunidades de melhoria mais relevantes para uma fase seguinte do projeto:

- **Análise de tráfego HTTPS:** A limitação mais significativa do monitor de rede e da integração Wireshark é a impossibilidade de analisar tráfego encriptado por TLS. A integração de um proxy com inspeção SSL, como o mitmproxy, permitiria estender a cobertura à grande maioria do tráfego web moderno.

- **Features de reputação de domínio:** O modelo atual baseia-se exclusivamente em características lexicais. A integração de *features* baseadas em WHOIS e DNS, como a idade do domínio e a data de expiração do registo, permitiria detetar URL de *phishing* curtos e simples que não utilizam palavras suspeitas no texto, uma das principais limitações identificadas nos testes.
- **Expansão do *dataset* com domínios institucionais portugueses:** Os falsos positivos identificados em domínios de universidades, câmaras e hospitais portugueses decorrem da ausência destes padrões no *dataset* de treino. A recolha e inclusão de URL institucionais portugueses benignos reduziria significativamente esta categoria de erros.
- **Publicação da aplicação Flask:** A dependência do Flask a correr localmente em localhost:5000 limita a utilidade da extensão de browser a utilizadores com conhecimentos técnicos para executar a aplicação. A publicação da aplicação num serviço cloud eliminaria esta dependência, tornando a extensão utilizável por qualquer pessoa sem configuração adicional.
- **Suporte ao Outlook Web:** A implementação do modo email para o Outlook Web não foi possível devido à geração dinâmica de nomes de classes CSS pela aplicação. A utilização da Microsoft Graph API, que expõe o conteúdo dos emails através de uma API REST estruturada, constituiria uma solução robusta e independente da estrutura do DOM.
- **Atualização automática do modelo na aplicação:** Atualmente, após o re-treino automático semanal pelo GitHub Actions, o utilizador tem de executar git pull manualmente para obter o modelo atualizado. A implementação de um mecanismo de verificação e download automático da versão mais recente do modelo eliminaria esta dependência.

12. Referências Bibliográficas

1. E. -S. El-Metwaly et al., "Detection of *Phishing* URL Based on Machine Learning and Cybersecurity," 2024 International Telecommunications Conference (ITC-Egypt), Cairo, Egypt, 2024, pp. 394-398, doi: 10.1109/ITC-Egypt61547.2024.10620574
2. J. Hariri, L. Batouq and A. Al-Jarf, "Developing a Model to Detect Malicious URL using Different Classification Algorithms," 2024 21st Learning and Technology Conference (L&T), Jeddah, Saudi Arabia, 2024, pp. 47-51, doi: 10.1109/LT60077.2024.10468929
3. Shantanu, B. Janet and R. Joshua Arul Kumar, "Malicious URL Detection: A Comparative Study," 2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS), Coimbatore, India, 2021, pp. 1147-1151, doi: 10.1109/ICAIS50930.2021.9396014
4. T. M. Saravanan, A. Muthusamy, C. P. Thamil Selvi, M. Senthil Kumar and D. Maheshwari, "Improving Cybersecurity Measures Through the Revelation of Malignant URL using Machine Learning Techniques," 2024 5th International Conference on Data Intelligence and Cognitive Informatics (ICDICI), Tirunelveli, India, 2024, pp. 471-475, doi: 10.1109/ICDICI62993.2024.10810759
5. H. Febrianto and A. Kurniawan, "Evaluation *Feature* Selection in Machine Learning Models for Malicious URL Detection," 2025 6th International Conference on Artificial Intelligence and Data Sciences (AiDAS), West Java, Indonesia, 2025, pp. 595-600, doi: 10.1109/AiDAS67696.2025.11213608
6. S. Kailas and R. Roopalakshmi, "'Think Before You Click"—Malicious URL Detection in Cybersecurity: A Systematic Review and Research Roadmap," in IEEE Access, vol. 13, pp. 154305-154325, 2025, doi: 10.1109/ACCESS.2025.3601387
7. Y. Ari Kustiawan and K. Imran Ghauth, "*Feature* Engineering for *Phishing* Website Detection Using Machine Learning: A Systematic Review," in IEEE Access, vol. 13, pp. 192080-192104, 2025, doi: 10.1109/ACCESS.2025.3630334

8. Y. Yang, H. Li and D. Jing, "Detection of Malicious URL Based on BERT CNN," 2023 International Conference on Computer Science and Automation Technology (CSAT), Shanghai, China, 2023, pp. 284-288, doi: 10.1109/CSAT61646.2023.00079
9. Sahoo, D., Liu, C. e Hoi, S. C. H. (2017). Malicious URL Detection using Machine Learning: A Survey. [Online]. Disponível em: <https://arxiv.org/pdf/1701.07179>